

**O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Himoyaga ruxsat
“ATDT” kafedra mudiri
_____ Raximov N.O.
«_____» _____ 2026-yil

BITIRUV MALAKAVIY ISHI

**Mavzu: Real vaqtda xabar almashish imkonini beruvchi “Milliy
chat” dasturini ishlab chiqish**

Bitiruvchi (315-21 Dlo‘
guruh)

(imzo)

S.O. Sohinazarov

Ilmiy rahbar

(imzo)

Y.Sh. Yuldoshev

Taqrizchi

(imzo)

HFX bo‘yicha maslahatchi

(imzo)

S. Abdullayeva

Toshkent – 2026

**O‘ZBEKISTON RESPUBLIKASI RAQAMLI TEXNOLOGIYALAR
VAZIRLIGI
MUHAMMAD AL-XORAZMIY NOMIDAGI
TOSHKENT AXBOROT TEXNOLOGIYALARI UNIVERSITETI**

Fakultet: Dasturiy injiniring

Kafedra: Axborot texnologiyalarining dasturiy ta’minoti

Ta’lim yo‘nalishi: 60610600 – Dasturiy injiniring

TASDIQLAYMAN

“ATDT” kafedra mudiri

_____ Raximov N.O.

« ____ » _____ 2026-yil

**315-21 DIo‘ guruh talabasi Sohinazarov Sardorbek O‘rinboy o‘g‘lining
bitiruv malakaviy ishiga**

T O P S H I R I Q

1. BMI mavzusi: Real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini ishlab chiqish.
2. BMI tasdiqlangan buyruq: « ____ » _____ 2026-yil № _____.
3. BMIning himoyaga topshirish muddati: _____.
4. BMIning boshlang‘ich ma’lumotlari: mavjud messengerlar (Telegram, WhatsApp, Signal) tahlili, real vaqtda xabar almashish texnologiyalari, .NET 8, ASP.NET Core, PostgreSQL, SignalR va Angular texnologiyalariga oid adabiyotlar hamda amaliyot materiallari.
5. Hisob-tushuntirish yozuvining mazmuni: sohaning tahlili va masalaning qo‘yilishi, tizim arxitekturasini va ma’lumotlar bazasini loyihalash, server va klient qismlarini ishlab chiqish, xavfsizlik choralari, foydalanish yo‘riqnomasi, hayot faoliyati xavfsizligi, xulosa.
6. Grafik materiallar ro‘yxati: tizim arxitekturasini, ER-diagramma, UML (use-case, sequence) diagrammalari, dastur interfeysi skrinshotlari va PowerPoint taqdimoti.
7. Topshiriq berilgan sana: _____.

Rahbar:

(imzo)

Y.Sh. Yuldoshev

Topshiriqni oldim:

(imzo)

S.O. Sohinazarov

8. BMIning alohida bo‘limlari bo‘yicha maslahatchilar:

№	Bo‘lim nomi	Maslahatchi	Topshiriq berdi	Topshiriq oldi
1	Asosiy qism	Y.Sh. Yuldoshev		
2	Hayot faoliyati xavfsizligi	S. Abdullayeva		

9. BMIning bajarish bo‘yicha kalendar grafik:

№	Malakaviy ish bo‘limlari	Bajarish muddati	Rahbar imzosi
1	Kirish, masalaning qo‘yilishi		
2	Soha tahlili, mavjud messenjerlarni o‘rganish		
3	Tizimni loyihalash (arxitektura, ma’lumotlar bazasi, UML)		
4	Dasturning server va klient qismlarini ishlab chiqish		
5	Hayot faoliyati xavfsizligi		
6	Xulosa, adabiyotlar ro‘yxati, ilovalar		
7	Taqdimot slaydlarini tayyorlash		
8	Dastlabki himoya		

Bitiruv malakaviy ish rahbari:

(imzo)

Y.Sh. Yuldoshev

Bitiruvchi:

(imzo)

S.O. Sohinazarov

ANNOTATSIYA

Ushbu bitiruv malakaviy ishi real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini ishlab chiqishga bag‘ishlangan. Ishda zamonaviy messenjerlar va real vaqtda aloqa texnologiyalari tahlil qilingan, tizimning arxitekturası, ma’lumotlar bazasi va UML modellari loyihalangan. Dasturning server qismi .NET 8 platformasida ASP.NET Core, Entity Framework Core, PostgreSQL va SignalR texnologiyalari asosida Clean Architecture tamoyillari bo‘yicha, klient qismi esa Angular freymvorkida ishlab chiqilgan. Tizim elektron pochta orqali bir martalik kod va Google hisobi yordamida autentifikatsiya, shaxsiy va guruh chatlari, real vaqtda xabar almashish, “yozmoqda” va o‘qilganlik belgilari, rasm, video va fayllar yuborish, hikoyalar (stories) hamda onlayn holatni kuzatish imkoniyatlarini taqdim etadi. Tizimning o‘ziga xos imkoniyatlari — xabarlarini lotin yoki kirill yozuvida ko‘rsatish va yozuvdan qat’i nazar qidiruv, e-pochta domeni orqali tashkilot a’zolicini tasdiqlash va yopiq tashkilot guruhları hamda Web Crypto API asosida uchdan-uchgacha shifrlangan maxfiy chatlar. Ilova Docker konteyneri ko‘rinishida bulutli serverga joylashtirilgan.

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке национального мессенджера «Milliy chat», обеспечивающего обмен сообщениями в реальном времени. В работе проанализированы современные мессенджеры и технологии обмена данными в реальном времени, спроектированы архитектура системы, база данных и UML-модели. Серверная часть разработана на платформе .NET 8 с использованием ASP.NET Core, Entity Framework Core, PostgreSQL и SignalR в соответствии с принципами Clean Architecture, клиентская часть — на фреймворке Angular. Система поддерживает аутентификацию по одноразовому коду, отправляемому на электронную почту, и через аккаунт Google, личные и групповые чаты, обмен сообщениями в реальном времени, индикаторы набора текста и прочтения, отправку изображений, видео и файлов, истории (stories), а также

отслеживание статуса «в сети». Особенности системы — отображение сообщений на латинице или кириллице по выбору пользователя и поиск независимо от алфавита, подтверждение принадлежности к организации по домену электронной почты с закрытыми группами организаций, а также секретные чаты со сквозным шифрованием на основе Web Crypto API. Приложение развёрнуто в облаке в виде Docker-контейнера.

ANNOTATION

This graduation qualification work is devoted to the development of “Milliy chat”, a national messenger that allows for real-time messaging. The work analyses modern messengers and real-time communication technologies and designs the system architecture, the database and the UML models. The server side is built on .NET 8 with ASP.NET Core, Entity Framework Core, PostgreSQL and SignalR following Clean Architecture principles, and the client side is built with the Angular framework. The system provides sign-in with a one-time code sent by e-mail or with a Google account, private and group chats, real-time messaging, typing and read indicators, sending images, videos and files, stories, and online presence tracking. Its distinctive features are showing messages in the Latin or Cyrillic Uzbek script chosen by each user with script-independent search, organization membership verified by the e-mail domain with private organization groups, and end-to-end encrypted secret chats built on the Web Crypto API. The application is deployed to the cloud as a Docker container.

MUNDARIJA

KIRISH	8
I-BOB. ZAMONAVIY MESSENERLAR VA MILLIY MESSENER YARATISH ZARURIYATI.....	11
1.1. Messenjerlarning rivojlanishi va hozirgi holati	11
1.2. Mavjud messenjerlar tahlili.....	12
1.3. Axborot xavfsizligi va ma'lumotlar lokalizatsiyasi.....	14
1.4. Real vaqtda aloqa texnologiyalari.....	15
1.5. Masalaning qo'yilishi	16
1-bob bo'yicha xulosa	18
II-BOB. "MILLIY CHAT" TIZIMINI LOYIHALASH	19
2.1. Texnologiyalarni tanlash va asoslash.....	19
2.2. Tizim arxitekturasini.....	21
2.3. Ma'lumotlar bazasini loyihalash	26
2.4. Tizimning UML modellari.....	34
2.5. Xavfsizlikni loyihalash	42
2.6. Maxfiy chatlar kriptografik protokolini loyihalash.....	47
2-bob bo'yicha xulosa	53
III-BOB. "MILLIY CHAT" DASTURINI ISHLAB CHIQISH.....	55
3.1. Server qismini ishlab chiqish	55
3.2. Real vaqtda xabar almashishni amalga oshirish	60
3.3. Media fayllar bilan ishlash.....	62
3.4. Guruh chatlari va hikoyalar	64
3.5. Lotin–kirill transliteratsiyasi va yozuvdan qat'i nazar qidiruv	67
3.6. Tashkilot rejimini amalga oshirish	71

3.7. Maxfiy chatlarni amalga oshirish.....	74
3.8. Klient qismini ishlab chiqish	78
3.9. Joylashtirish va testlash	81
3.10. Foydalanish yo‘riqnomasi.....	85
3-bob bo‘yicha xulosa	93
IV-BOB. HAYOT FAOLİYATI XAVFSIZLIGI	95
4.1. Dasturchining ergonomik ish joyini tashkil etish	95
4.2. Evakuatsiya tadbirlarini rejalashtirish hamda avariya va yong‘inlar sodir bo‘lganda xodimlarning harakatlari	103
4-bob bo‘yicha xulosa	112
XULOSA	114
FOYDALANILGAN ADABIYOTLAR	117
ILOVALAR	121
1-ilova. SignalR habi (ChatHub)	121
2-ilova. Guruh chatlari servisi (GroupService)	123
3-ilova. Bir martalik kodlarni xeshlash (PBKDF2).....	130
4-ilova. Klientdagi real vaqt ulanishi (ChatRealtimeService)	130
5-ilova. Guruh chatlari uchun unit testlar (parcha).....	132
6-ilova. O‘zbek yozuvlari o‘rtasida transliteratsiya (UzbekTransliterator).....	133
7-ilova. Maxfiy chatlar kriptografiyasi: kalitlarni chiqarish, shifrlash va ratchet (secret-crypto.ts, parcha)	137

KIRISH

Bugungi kunda odamlar o'rtasidagi muloqotning katta qismi telefon qo'ng'iroqlari va elektron pochtdan messenjerlarga ko'chdi. Messenjerlarda o'quv guruhlari va ish jamoalari tashkil etiladi, yangiliklar tarqatiladi, davlat va tijorat xizmatlari ko'rsatiladi. O'zbekistonda aholining asosiy qismi xorijiy kompaniyalarga tegishli Telegram va WhatsApp ilovalaridan foydalanadi. Natijada fuqarolarning yozishmalari va shaxsga doir ma'lumotlari mamlakatdan tashqaridagi, milliy nazoratga bo'ysunmaydigan serverlarda saqlanadi.

Mavzuning dolzarbligi.

“Raqamli O'zbekiston – 2030” strategiyasi (PF-6079-son Farmon, 2020-yil 5-oktabr) mahalliy dasturiy mahsulotlarni rivojlantirish va axborot xavfsizligini ta'minlashni ustuvor vazifa sifatida belgilaydi. “Shaxsga doir ma'lumotlar to'g'risida”gi Qonunga 2021-yilda kiritilgan o'zgartishlarga ko'ra fuqarolarning shaxsga doir ma'lumotlari respublika hududidagi texnik vositalarda saqlanishi lozim. Bu talablar ma'lumotlarni o'z serverida saqlaydigan milliy messenjer yaratishni dolzarb qiladi.

Texnik jihatdan ham masala murakkab: xabar, “yozmoqda” belgisi va onlayn holat real vaqtda, sahifani yangilamasdan yetkazilishi, katta yozishmalar tarixi tez yuklanishi, autentifikatsiya esa ishonchli bo'lishi kerak. Bularni hal etish qatlamli arxitektura, real vaqt protokollari va ma'lumotlar bazasini optimallashtirishni amalda qo'llashni talab qiladi.

Bitiruv malakaviy ishining maqsadi

Ishning maqsadi — foydalanuvchi ma'lumotlarini o'z serverida saqlaydigan, shaxsiy va guruh yozishmalarida real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini loyihalash va ishlab chiqishdir.

Qo'yilgan maqsadga erishish uchun quyidagi vazifalar belgilandi:

- mavjud messenjerlarni va real vaqtda ma'lumot almashish texnologiyalarini tahlil qilib, tizim uchun eng maqbul yechimni tanlash;

- funksional va nofunktional talablarni aniqlash, Clean Architecture asosida tizim arxitekturasini, ma'lumotlar bazasini va UML modellarini loyihalash;
- bir martalik kod, Google hisobi, JWT va refresh token asosida xavfsiz autentifikatsiyani ishlab chiqish;
- .NET 8 va SignalR asosida shaxsiy va guruh chatlari, media fayllar, hikoyalar va onlayn holatni amalga oshirish, Angular'da moslashuvchan interfeys yaratish;
- lotin–kirill transliteratsiyasi, e-pochta domeni orqali tashkilot rejimi va uchdan-uchgacha shifrlangan maxfiy chatlarni ishlab chiqish;
- dasturni Docker konteynerida bulutli serverga joylashtirish va sinovdan o'tkazish;
- dasturchi ish joyining ergonomikasi va favqulodda vaziyatlarda evakuatsiya tadbirlarini ishlab chiqish.

Tadqiqot obyekti va predmeti.

Tadqiqot obyekti — foydalanuvchilar o'rtasida matnli va multimedia xabarlar almashishni ta'minlaydigan onlayn muloqot tizimlari. Tadqiqot predmeti — real vaqtda ishlaydigan messenjerning arxitekturasini, ma'lumotlar modeli va xabarlarni yetkazish mexanizmlarini .NET va Angular texnologiyalari yordamida amalga oshirish usullari.

Tadqiqot usullari.

Ishda qiyosiy tahlil, obyektga yo'naltirilgan loyihalash, UML yordamida modellashtirish, relatsion ma'lumotlar bazasini normallashtirish hamda iterativ ishlab chiqish va avtomatlashtirilgan testlash usullaridan foydalanildi.

Ishning ilmiy-amaliy yangiligi.

- o'zbek tilining lotin va kirill yozuvlari o'rtasida kontekstga bog'liq qoidalarni hisobga oluvchi transliteratsiya algoritmi ishlab chiqildi. Uning asosida har bir foydalanuvchi xabarlarni o'zi tanlagan yozuvda ko'radi, qidiruv esa yozuvga bog'liq bo'lmagan kalit bo'yicha ishlaydi;

- tashkilotni oldindan ro'yxatga olmasdan, tasdiqlangan e-pochta domeni orqali a'zolikni tasdiqlash va yopiq tashkilot guruhlarini avtomatik shakllantirish usuli taklif etildi;
- faqat brauzerning standart kriptografik primitivlari (X25519, HKDF, HMAC, AES-256-GCM) asosida oldinga maxfiylikka ega maxfiy chat protokoli ishlab chiqildi; server xabar mazmunini bilmasligi ma'lumotlar bazasi darajasida ko'rsatib berildi.

Ishning amaliy ahamiyati.

“Milliy chat” to'liq ishlaydigan veb-ilova bo'lib, uni tashkilotlar, ta'lim muassasalari va davlat idoralari o'z serverlarida ichki muloqot vositasi sifatida qo'llashi mumkin. Xodimlar korporativ pochtasi bilan kirishi bilanoq tashkilotning yopiq guruhiga qo'shiladi, maxfiy chatlar esa nozik yozishmalarni serverdan ham himoya qiladi.

Bitiruv malakaviy ishining tuzilishi.

Ish kirish, to'rt bob, xulosa, foydalanilgan adabiyotlar ro'yxati va ilovalardan iborat. Birinchi bobda mavjud messenjerlar va real vaqtda aloqa texnologiyalari tahlil qilinib, masala qo'yilgan. Ikkinchi bobda tizim arxitekturasini, ma'lumotlar bazasi, UML modellari, xavfsizlik va maxfiy chat protokoli loyihalangan. Uchinchi bobda dasturni ishlab chiqish, joylashtirish, sinash va foydalanish yo'riqnomasi keltirilgan. To'rtinchi bob hayot faoliyati xavfsizligiga bag'ishlangan. Ish 46 ta rasm, 19 ta jadval, 10 ta kod listingi va 7 ta ilovani o'z ichiga oladi.

I-BOB. ZAMONAVIY MESSENERLAR VA MILLIY MESSENER YARATISH ZARURIYATI

1.1. Messenjerlarning rivojlanishi va hozirgi holati

Messenjer (ingl. instant messenger) — foydalanuvchilar o‘rtasida matnli, ovoqli, video va boshqa xabarlarni Internet orqali deyarli bir zumda yetkazadigan dasturiy ta‘minot. Elektron pochtdan farqli ravishda messenjerda muloqot dialog ko‘rinishida kechadi: xabar darhol yetkaziladi, jo‘natuvchi esa suhbatdosh onlayn ekanini, xabarni o‘qiganini yoki javob yozayotganini ko‘radi.

Birinchi ommaviy chat tizimi 1988-yilda yaratilgan IRC protokoli edi. 1996-yilda chiqqan ICQ kontaktlar ro‘yxati va onlayn holat tushunchalarini ommalashtirdi, 1999-yilda boshlangan Jabber loyihasi esa IETF standarti XMPP’ga asos bo‘ldi. Smartfonlar paydo bo‘lgach, 2009-yilda WhatsApp foydalanuvchini telefon raqami orqali identifikatsiya qilishni joriy etdi va messenjerni SMS’ning bepul muqobiliga aylantirdi. 2013-yilda ishga tushirilgan Telegram yozishmalarni serverda saqlaydigan “bulutli” yondashuvni, 2014-yilda chiqqan Signal esa keyinchalik WhatsApp’da ham qo‘llangan uchdan-uchgacha shifrlash protokolini taklif etdi.

Hozirgi messenjerlar quyidagi yo‘nalishlarda rivojlanmoqda:

- **xavfsizlik** — uchdan-uchgacha shifrlash, faol sessiyalarni boshqarish, o‘z-o‘zidan o‘chadigan xabarlar;
- **ko‘p qurilmalilik** — bitta hisobdan telefon, kompyuter va brauzerda bir vaqtda foydalanish;
- **ommaviy kommunikatsiya** — katta guruhlar, kanallar va 24 soatlik hikoyalar (stories);
- **platformaga aylanish** — botlar, mini-ilovalar va to‘lov tizimlari (WeChat kabi “super-ilova”lar).

WhatsApp foydalanuvchilari soni 2 milliarddan, Telegramning oylik faol foydalanuvchilari 1 milliarddan oshgan. Shu bilan birga, ayrim davlatlarda mahalliy messenjerlar ustunlik qiladi: Janubiy Koreyada KakaoTalk, Yaponiyada LINE,

Vyetnamda Zalo, Xitoyda WeChat. Bu misollar mahalliy ehtiyojlarni yaxshi hisobga olgan milliy messenjer xalqaro raqobatchilar bilan muvaffaqiyatli raqobatlasha olishini ko'rsatadi.

1.2. Mavjud messenjerlar tahlili

Tizimga qo'yiladigan talablarni aniqlash uchun O'zbekistonda keng qo'llaniladigan Telegram va WhatsApp, maxfiylik bo'yicha etalon hisoblangan Signal hamda WeChat tahlil qilindi.

Telegram.

Telegram o'zining MTProto 2.0 protokolidan foydalanadi. Oddiy chatlarda xabarlar mijoz va server o'rtasida shifrlanib, serverda saqlanadi, shu tufayli yozishmalar har qanday qurilmadan ochiladi. Uchdan-uchgacha shifrlash faqat alohida "maxfiy chat"larda qo'llanadi. 200 000 a'zoli guruhlar, kanallar, botlar va 2 GB gacha fayllar qo'llab-quvvatlanadi. Kuchli tomonlari — tezlik, bulutli sinxronizatsiya va boy funksionallik; kamchiliklari — server kodining yopiqligi va serverlarning xorijda joylashgani.

WhatsApp.

WhatsApp Meta kompaniyasiga tegishli. 2016-yildan barcha yozishmalar Signal Protocol asosida uchdan-uchgacha shifrlanadi va serverda faqat yetkazilgunga qadar turadi. Guruhlar 1024 a'zogacha bo'ladi. Kamchiliklari — kodi yopiq, metama'lumotlar (kim, kim bilan, qachon yozishgani) esa kompaniya ixtiyorida qoladi.

Signal va WeChat.

Signal'ning klient va server kodlari to'liq ochiq, barcha yozishmalar sukut bo'yicha shifrlanadi, metama'lumotlar minimal saqlanadi. Biroq u funksional jihatdan kambag'alroq: kanallar, botlar va katta guruhlar yo'q. Tencent'ga tegishli WeChat esa messenjer, to'lov tizimi va mini-ilovalarni birlashtirgan "super-ilova" bo'lib, uchdan-uchgacha shifrlashni qo'llamaydi. Messenjerlarning taqqoslanishi 1.1-jadvalda keltirilgan.

Messenjerlarning qiyosiy tahlili

Mezon	Telegram	WhatsApp	Signal	WeChat	Milliy chat
Egasi (davlat)	Telegram (BAA)	Meta (AQSh)	Signal Foundation (AQSh)	Tencent (Xitoy)	Mahalliy ishlab chiquvchi
Ro'yxatdan o'tish	Telefon raqami	Telefon raqami	Telefon raqami	Telefon raqami	E-pochta kodi, Google
E2E shifrlash	Faqat maxfiy chatlarda	Sukut bo'yicha	Sukut bo'yicha	Yo'q	Maxfiy chatlarda (X25519, AES-GCM)
Yozishmalar saqlanishi	Bulutda (serverda)	Qurilmada	Qurilmada	Serverda	O'z serverida (bulutli)
Guruh hajmi	200 000	1024	1000	500	200
Kanallar, botlar	Bor	Kanallar bor	Yo'q	Bor	Kelajakda (sxemasi tayyor)
Hikoyalar	Bor	Bor (Status)	Bor	Bor	Bor
Server kodi	Yopiq	Yopiq	Ochiq	Yopiq	Mahalliy nazoratda
Ma'lumotlar joylashuvi	Xorijda	Xorijda	Xorijda	Xitoyda	Istalgan serverda, jumladan O'zbekistonda
Lotin/kirill yozuvida ko'rsatish	Yo'q	Yo'q	Yo'q	Yo'q	Bor (qidiruv ham)
Tashkilot tomonidan tasdiqlash	Yo'q	Yo'q	Yo'q	Yo'q	E-pochta domeni orqali

Mavjud messenjerlar funksionallik va qulaylik bo'yicha yuqori darajada, ammo barchasi xorijiy kompaniyalar tomonidan boshqariladi va ma'lumotlarni mamlakatdan tashqarida saqlaydi. Funksionallik va qulaylik muvozanati eng yaxshi bo'lgani uchun foydalanuvchi tajribasi bo'yicha namuna sifatida Telegram tanlandi. Ro'yxatdan o'tish esa pullik SMS-shlyuz o'rniga elektron pochta va Google hisobi orqali amalga oshiriladi.

1.3. Axborot xavfsizligi va ma'lumotlar lokalizatsiyasi

Messenjer foydalanuvchi haqidagi eng shaxsiy ma'lumotlarni — yozishmalar, fayllar, kontaktlar, kim bilan qachon muloqot qilingani, IP-manzillar va qurilmalarni saqlaydi. Xorijiy messenjerlardan foydalanishda quyidagi xavflar mavjud:

- **yurisdiksiya xavfi** — ma'lumotlar boshqa davlat qonunlariga bo'ysunadi va ularni o'sha davlat organlari talab qilishi mumkin;
- **metama'lumotlar xavfi** — xabar mazmuni shifrlangan bo'lsa ham, muloqot grafigi xizmat egasida qoladi;
- **xizmatning uzilishi** — shartlar o'zgarishi, hisob bloklanishi yoki mintaqaviy cheklov foydalanuvchini aloqadan mahrum qiladi;
- **nazorat qilib bo'lmazlik** — yopiq server kodi ma'lumotlar qanday qayta ishlanishini tekshirishga imkon bermaydi.

“Shaxsga doir ma'lumotlar to'g'risida”gi O'RQ-547-son Qonun (2019) shaxsga doir ma'lumotlarni yig'ish, saqlash va himoya qilish tartibini belgilaydi. 2021-yilgi qo'shimchalarga ko'ra fuqarolarning shaxsga doir ma'lumotlarini Internet orqali qayta ishlovchi operatorlar ularni respublika hududidagi ma'lumotlar bazalarida saqlashi shart. Milliy messenjer bu talabga tabiiy ravishda javob beradi: uning serverlari mamlakat ichidagi data-markazda joylashadi, kodi mahalliy mutaxassislar nazoratida bo'ladi.

Milliy messenjer mahalliy ehtiyojlarni ham hisobga oladi. O'zbekistonda lotin yozuvi rasmiy bo'lsa-da, kirill yozuvi hanuz keng qo'llanadi: katta avlod kirillda, yoshlar lotinda yozadi. Xorijiy messenjerlarda bu yozuvlar o'rtasida bog'lanish yo'q — lotincha qidiruv kirillcha xabarni topmaydi. Shuningdek, ta'lim muassasalari va tashkilotlar uchun a'zoning haqiqatan shu tashkilotga tegishli ekanini tasdiqlash muhim. Telefon raqamiga asoslangan messenjerlar buni ta'minlay olmaydi, tashkilotning e-pochta domeni esa bunday tasdiq uchun tabiiy vositadir.

1.4. Real vaqtda aloqa texnologiyalari

Messenjering asosiy texnik vazifasi — yangi xabarni qabul qiluvchiga imkon qadar tez yetkazish. HTTP protokoli “so‘rov–javob” modeliga asoslangan va server o‘z tashabbusi bilan mijozga ma’lumot yubora olmaydi. Shu sababli serverdan mijozga ma’lumot “itarish” (push) uchun bir necha usul ishlab chiqilgan.

Short polling usulida mijoz har bir necha soniyada “yangi xabar bormi?” deb so‘raydi: so‘rovlarning aksariyati bo‘sh qaytadi, xabar esa interval qadar kechikadi. **Long polling**’da server yangi ma’lumot paydo bo‘lguncha javobni ushlab turadi; kechikish kamayadi, lekin har bir xabar uchun yangi HTTP so‘rovi kerak. **Server-Sent Events** (SSE) bitta HTTP ulanish orqali hodisalarni uzluksiz yuboradi, ammo faqat serverdan mijozga yo‘nalishda ishlaydi.

WebSocket (RFC 6455) ulanishi oddiy HTTP so‘rovi bilan boshlanib, “101 Switching Protocols” javobidan so‘ng ikki tomonlama (full-duplex) kanalga aylanadi. Shundan keyin mijoz ham, server ham istalgan vaqtda kichik freymmlar yuboradi. Minimal kechikish va qo‘shimcha trafik tufayli WebSocket zamonaviy veb-messenjrlarning asosiy transporti hisoblanadi.

ASP.NET Core SignalR — Microsoft kutubxonasi bo‘lib, transport darajasini abstraksiya qiladi: avval WebSocket orqali ulanadi, u ishlamasa avtomatik ravishda SSE yoki long polling’ga o‘tadi. Dasturchi “hab” (Hub) abstraksiyasi bilan ishlaydi: mijoz server metodlarini, server esa mijoz funksiyalarini masofadan chaqiradi. Ulanishlarni guruhlash, ma’lum foydalanuvchiga xabar yuborish, JWT autentifikatsiyasi va qayta ulanish tayyor holda beriladi. Texnologiyalar 1.2-jadvalda taqqoslangan.

1.2-jadval

Real vaqtda aloqa texnologiyalarining taqqoslanishi

Texnologiya	Yo‘nalish	Kechikish	Server yuklamasi	Brauzerda qo‘llab-quvvatlash
Short polling	Mijoz → server	Yuqori (interval)	Yuqori	Barcha brauzerlar
Long polling	Mijoz → server	O‘rtacha	O‘rtacha	Barcha brauzerlar
Server-Sent Events	Server → mijoz	Past	Past	Zamonaviy brauzerlar

Texnologiya	Yo‘nalish	Kechikish	Server yuklamasi	Brauzerda qo‘llab-quvvatlash
WebSocket	Ikki tomonlama	Juda past	Past	Zamonaviy brauzerlar
SignalR	Ikki tomonlama (WebSocket, SSE yoki long polling)	Juda past	Past	Barcha brauzerlar (fallback hisobiga)

Tahlil asosida “Milliy chat”da real vaqt aloqasi uchun SignalR tanlandi: u WebSocket samaradorligini saqlagan holda cheklangan tarmoqlarda ham ishlaydi, .NET server bilan tabiiy integratsiyalashadi, brauzer uchun esa rasmiy @microsoft/signalr kutubxonasi mavjud.

1.5. Masalaning qo‘yilishi

Tahlil natijalariga ko‘ra, Telegram namunasi asosida veb-brauzerda ishlaydigan, ma’lumotlarni o‘z serverida saqlaydigan va real vaqtda xabar almashish imkonini beruvchi “Milliy chat” dasturini ishlab chiqish talab etiladi. Tizimda uch turdagi aktor bor: tizimga hali kirmagan **mehmon**, ro‘yxatdan o‘tgan **foydalanuvchi** va **guruh administratori (egasi)**.

Funksional talablar. Tizim quyidagi imkoniyatlarni ta’minlashi kerak:

1. elektron pochtaga yuboriladigan 6 xonali bir martalik kod yoki Google hisobi orqali ro‘yxatdan o‘tish va kirish;
2. profilni (ism, username, bio, rasm) tahrirlash;
3. faol sessiyalar (qurilmalar) ro‘yxatini ko‘rish va ularni yakunlash;
4. foydalanuvchilarni ism yoki username bo‘yicha qidirish;
5. shaxsiy chat yaratish, chatlar ro‘yxatini oxirgi xabar va o‘qilmaganlar soni bilan ko‘rish;
6. guruh yaratish, a’zolar va administratorlarni boshqarish, guruh ma’lumotlarini o‘zgartirish;
7. xabar yuborish, javob berish (reply), tahrirlash, o‘chirish va chat tarixini tozalash;
8. rasm, video va fayllar yuborish, chatdagi media fayllarni ko‘rish;

9. chat ichida xabarlarni matn bo'yicha qidirish;
10. yangi xabarlar, "yozmoqda" holati, o'qilganlik belgilari va onlayn holatni real vaqtda ko'rsatish;
11. 24 soatlik hikoyalar (stories) joylash va ko'rish;
12. yorug' va qorong'i mavzular, mobil qurilmalarga moslashuvchan interfeys;
13. xabarlarni tanlangan yozuvda (asl holida, lotin yoki kirill) ko'rsatish va yozuvdan qat'i nazar qidirish;
14. e-pochta domeni orqali tashkilot a'zolarini tasdiqlash, yopiq tashkilot guruhlari va taklif havolalari;
15. uchdan-uchgacha shifrlangan maxfiy chat: kalitni tekshirish, o'z-o'zini o'chirish taymeri, shifrlangan fayllar.

Nofunksional talablar:

- **unumdorlik** — ro'yxatlarni kursorli sahifalash, real vaqt xabarlarini 1 soniyadan kam kechikish bilan yetkazish;
- **xavfsizlik** — HTTPS, qisqa muddatli access token, xavfsiz cookie'dagi refresh token, kod va tokenlarni xesh ko'rinishida saqlash, fayllarni antivirus orqali tekshirish;
- **ishonchlilik** — xatolarni markazlashgan qayta ishlash, ulanish uzilganda avtomatik qayta ulanish;
- **kengaytiriluvchanlik** — holatsiz API, yangi funktsiyani boshqa qismlarga ta'sir qilmasdan qo'shish;
- **qo'llab-quvvatlanuvchanlik** — Clean Architecture qatlamlari, yagona kod uslubi, versiyalar nazorati;
- **ko'chiriluvchanlik** — Docker konteynerida istalgan serverga joylashtirish;
- **qulaylik** — o'zbek tilidagi, Telegram foydalanuvchilariga tanish interfeys.

1-bob bo'yicha xulosa

Birinchi bobda messenjerlarning rivojlanishi ko'rib chiqildi va mavjud messenjerlar funktsionallik, xavfsizlik hamda ma'lumotlarni saqlash joyi bo'yicha taqqoslandi. Ularning barchasi ma'lumotlarni mamlakatdan tashqarida saqlashi aniqlandi, bu esa lokalizatsiya talablari nuqtai nazaridan milliy messenjer yaratish zaruriyatini asoslaydi. Real vaqt aloqasi uchun WebSocket va zaxira transportlarni avtomatik tanlaydigan SignalR tanlandi. Bob yakunida 15 ta funktsional va 7 guruh nofunktsional talab aniqlanib, masala qo'yildi.

II-BOB. “MILLIY CHAT” TIZIMINI LOYIHALASH

2.1. Texnologiyalarni tanlash va asoslash

Birinchi bobda aniqlangan talablar texnologiyalarni tanlashda quyidagi mezonlarni belgilab berdi: real vaqtda ikki tomonlama aloqani qo‘llab-quvvatlash; yuqori unumdorlik va ko‘p sonli bir vaqtdagi ulanishlarga chidamlilik; xavfsizlik vositalarining (autentifikatsiya, kriptografiya) tayyor va ishonchli bo‘lishi; ekotizimning yetukligi va uzoq muddatli qo‘llab-quvvatlanishi; ochiq kodli va bepul litsenziya; konteynerlarda joylashtirish qulayligi.

Server qismi.

Server qismi uchun Microsoft kompaniyasining **.NET 8** platformasi va **ASP.NET Core 8** veb-freymvorki tanlandi. .NET 8 uzoq muddatli qo‘llab-quvvatlanadigan (LTS) versiya bo‘lib, kross-platformali, ochiq kodli va Linux konteynerlarida ishlaydi. ASP.NET Core mustaqil sinovlarda eng tezkor veb-freymvorklar qatorida turadi, unda bog‘liqliklarni kiritish (Dependency Injection), konfiguratsiya, loglash va middleware konveyeri tayyor holda mavjud. C# tilining qat’iy tiplanishi, async/await modeli va LINQ so‘rovlari murakkab biznes-mantiqni ishonchli yozish imkonini beradi.

Muqobil sifatida Node.js (Express, NestJS) va Java (Spring Boot) ko‘rib chiqildi. Node.js real vaqt ilovalari uchun qulay, lekin dinamik tiplash katta loyihada xatolar ehtimolini oshiradi. Spring Boot esa kuchli, ammo ko‘proq resurs talab qiladi va konfiguratsiyasi murakkabroq. .NET tanlovining hal qiluvchi sababi — **SignalR** kutubxonasi platformaning tarkibiy qismi ekanligi: WebSocket, SSE va long polling transportlari, hablar, guruhlar va JWT autentifikatsiyasi qo‘shimcha kutubxonalarsiz ishlaydi.

Ma’lumotlar bazasi sifatida **PostgreSQL** tanlandi. U ochiq kodli, ACID tranzaksiyalarni to‘liq qo‘llab-quvvatlaydigan, sxemalar, izohlar (comment), qisman va murakkab indekslar, matnni registrga bog‘liq bo‘lmagan holda qidirish (ILIKE) kabi imkoniyatlarga ega obyekt-relatsion MBBT. Ma’lumotlar bazasi bilan ishlash **Entity Framework Core 8** ORM va Npgsql provayderi orqali amalga

oshiriladi. EF Core LINQ so'rovlarini SQLga tarjima qiladi, sxema o'zgarishlarini migratsiyalar ko'rinishida boshqaradi va obyektlarni kuzatish (change tracking) mexanizmini taqdim etadi.

Klient qismi.

Foydalanuvchi interfeysi Google kompaniyasining **Angular 22** freymvorkida ishlab chiqildi. Angular to'liq freymvork bo'lib, marshrutlash, formalar, HTTP klient, bog'liqliklarni kiritish va sinov vositalarini bitta ekotizimda beradi. Loyihada Angularning zamonaviy imkoniyatlari qo'llanildi: mustaqil (standalone) komponentlar, holatni boshqarish uchun **signals** mexanizmi, shablonlardagi yangi boshqaruv bloklari (@if, @for) va server tomonida render qilish (SSR). Kod **TypeScript 6** tilida yozilgan, asinxron oqimlar **RxJS** orqali boshqariladi, real vaqt aloqasi esa rasmiy **@microsoft/signalr** kutubxonasi yordamida o'rnatiladi.

React va Vue.js muqobil variantlar sifatida ko'rib chiqildi. Ular kutubxona sifatida yengilroq, lekin marshrutlash, formalar va holatni boshqarish uchun uchinchi tomon kutubxonalarini tanlash va birlashtirishni talab qiladi. Angularning qat'iy tuzilmasi va TypeScriptni birinchi darajali qo'llab-quvvatlashi server qismidagi C# kodi bilan uslubiy uyg'unlikni ta'minlaydi.

Infratuzilma.

Server ilovasi **Docker** konteyneriga ko'p bosqichli (multi-stage) usulda yig'iladi: birinchi bosqichda .NET SDK obrazida loyiha kompilyatsiya qilinadi, ikkinchi bosqichda esa faqat kompilyatsiya natijasi yengil ASP.NET runtime obraziga ko'chiriladi. Konteyner **Render** bulutli platformasida ishga tushirilgan. Kod **Git** tizimida, GitHub'dagi ikkita repozitoriyda (server va klient) yuritiladi. Har bir funksiya alohida tarmoqda ishlab chiqilib, Pull Request orqali asosiy tarmoqqa birlashtiriladi. Tanlangan texnologiyalar 2.1-jadvalda umumlashtirilgan.

2.1-jadval

Tizimda qo'llanilgan texnologiyalar

Qatlam	Texnologiya	Versiya	Vazifasi
Server	.NET, ASP.NET Core	8.0 (LTS)	Web API, middleware, DI, konfiguratsiya

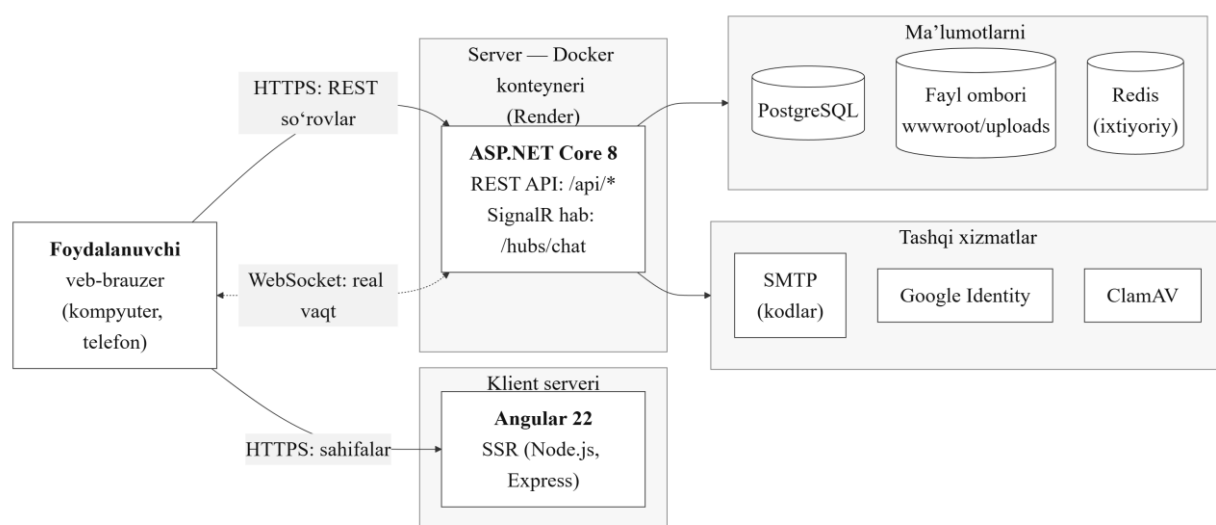
Qatlam	Texnologiya	Versiya	Vazifasi
Server	ASP.NET Core SignalR	8.0	Real vaqtda ikki tomonlama aloqa (hab)
Server	Entity Framework Core, Npgsql	8.0.8	ORM, LINQ so‘rovlari, migratsiyalar
Server	FluentValidation	11.11	Kiruvchi ma’lumotlarni validatsiya qilish
Server	System.IdentityModel.Tokens.Jwt	8.3.1	JWT access tokenlarni yaratish va tekshirish
Server	Google.Apis.Auth	1.68	Google ID tokenini tekshirish
Server	SixLabors.ImageSharp	3.1.12	Rasmlarni tekshirish va thumbnail yaratish
Server	StackExchange.Redis	2.8.16	Onlayn holatni taqsimlangan saqlash
Server	Swashbuckle (Swagger)	6.6.2	API hujjatlari va sinov interfeysi
Ma’lumotlar	PostgreSQL	16+	Relatsion ma’lumotlar bazasi
Klient	Angular, Angular SSR	22	SPA freymvork, server tomonida render
Klient	TypeScript, RxJS	6.0, 7.8	Tiplashtirilgan til, reaktiv oqimlar
Klient	@microsoft/signalr	10.0	Brauzerdagi SignalR klienti
Klient	Web Crypto API (X25519, HKDF, HMAC, AES-GCM)	brauzer	Maxfiy chatlarda uchdan-uchgacha shifrlash
Klient	IndexedDB	brauzer	Maxfiy chat kalitlari va tarixini faqat qurilmada saqlash
Infratuzilma	Docker, Render, Git/GitHub	—	Konteynerlash, joylashtirish, versiyalar nazorati

2.2. Tizim arxitekturası

Umumiy arxitektura.

“Milliy chat” klient-server arxitekturası asosida qurilgan. Foydalanuvchi brauzerida Angular ilovasi ishlaydi. U server bilan ikki kanal orqali aloqa qiladi: HTTPS ustidagi **REST API** (JSON formatidagi so‘rov va javoblar) va WebSocket ustidagi **SignalR** ulanishi. REST API holatni o‘zgartiruvchi barcha amallar uchun ishlatiladi: xabar yuborish, profilni tahrirlash, guruh yaratish va hokazo. SignalR ulanishi esa serverdan mijozga hodisalarni tezkor yetkazish uchun xizmat qiladi: yangi xabar, o‘qilganlik, “yozmoqda” holati, onlayn holat, guruh o‘zgarishlari.

Server qismi holatsiz (stateless) qilib loyihalangan: foydalanuvchi sessiyasi haqidagi barcha ma'lumotlar ma'lumotlar bazasida, autentifikatsiya esa har bir so'rovdagi JWT tokenda saqlanadi. Shu sababli bir nechta server nusxasini ishga tushirib, yuklamani ular o'rtasida taqsimlash mumkin. Bunda faqat SignalR ulanishlari holati va onlayn foydalanuvchilar ro'yxatini umumiy joyda (Redis) saqlash talab etiladi. Tizimning umumiy arxitekturasini va joylashtirish sxemasi 2.1-rasmda keltirilgan.



2.1-rasm. Tizimning umumiy arxitekturasini va joylashtirish sxemasi

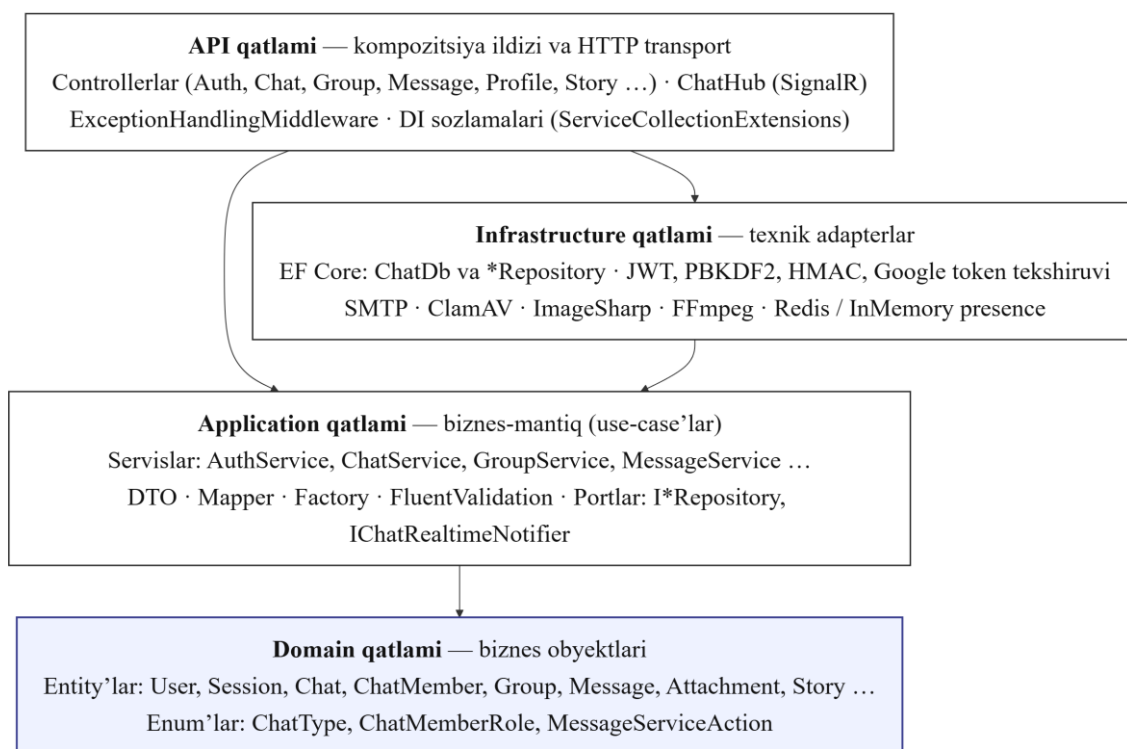
Rasmdan ko'rinib turibdiki, tizim tashqi xizmatlar bilan ham ishlaydi. SMTP server bir martalik kodlarni elektron pochtaga yetkazadi. Google Identity xizmati Google hisobi orqali kirishda ID tokenini beradi, server esa uni Google'ning ochiq kalitlari yordamida tekshiradi. ClamAV antivirusi yuklanayotgan fayllarni tekshiradi. Yuklangan fayllar serverning fayl omborida (wwwroot/uploads katalogi) sana bo'yicha ajratilgan papkalarda saqlanadi, ma'lumotlar bazasida esa faqat ularning metama'lumotlari va yo'li yoziladi.

Server qismining qatlamli arxitekturasini.

Server kodi Robert Martin tomonidan taklif etilgan **toza arxitektura** (Clean Architecture) tamoyillari asosida tashkil etilgan. Uning asosiy g'oyasi — biznes-mantiqni texnik tafsilotlardan (ma'lumotlar bazasi, veb-freymvork, tashqi xizmatlar) ajratish. Bog'liqliklar faqat tashqi qatlamlardan ichki qatlamlarga

yo‘nalgan bo‘lishi kerak. Natijada biznes-qoidalar ma’lumotlar bazasi yoki freymvork almashtirilganda o‘zgarmaydi, ularni esa alohida sinash mumkin bo‘ladi.

Yechim (solution) to‘rtta loyihadan iborat. Ularning o‘zaro bog‘liqligi 2.2-rasmda ko‘rsatilgan.



2.2-rasm. Server qismining Clean Architecture qatlamlari

- **Domain** — tizimning biznes obyektlari (entity): User, Session, Chat, ChatMember, Group, Message, Attachment, File, Photo, Story, Organization, OrganizationMember, SecretChat, SecretMessage, SecretFile va boshqalar, shuningdek ChatType, ChatMemberRole, MessageServiceAction, ScriptPreference, SecretChatStatus kabi sanab o‘tiladigan turlar. Tashqi bog‘liqligi yo‘q biznes-qoida sifatida o‘zbek yozuvlari o‘rtasidagi transliteratsiya (UzbekTransliterator) ham shu qatlamda joylashgan. Bu qatlam boshqa hech bir loyihaga bog‘liq emas;
- **Application** — foydalanish holatlari (use-case) amalga oshiriladigan qatlam. Unda servislar (AuthService, ChatService, GroupService, MessageService, StoryService, OrganizationMembershipService, SecretChatService...), ma’lumot uzatish obyektlari (DTO),

FluentValidation validatorlari, entity'larni DTOga o'giruvchi Mapper va yangi obyektlarni yaratuvchi Factory sinflari hamda tashqi dunyo bilan aloqa uchun "portlar" — interfeyslar (IMessageRepository, IChatRealtimeNotifier, IEmailSender, IFileStorage, ISecretChatRepository, ISecretFileStorage...) joylashgan. Application faqat Domain'ga bog'liq;

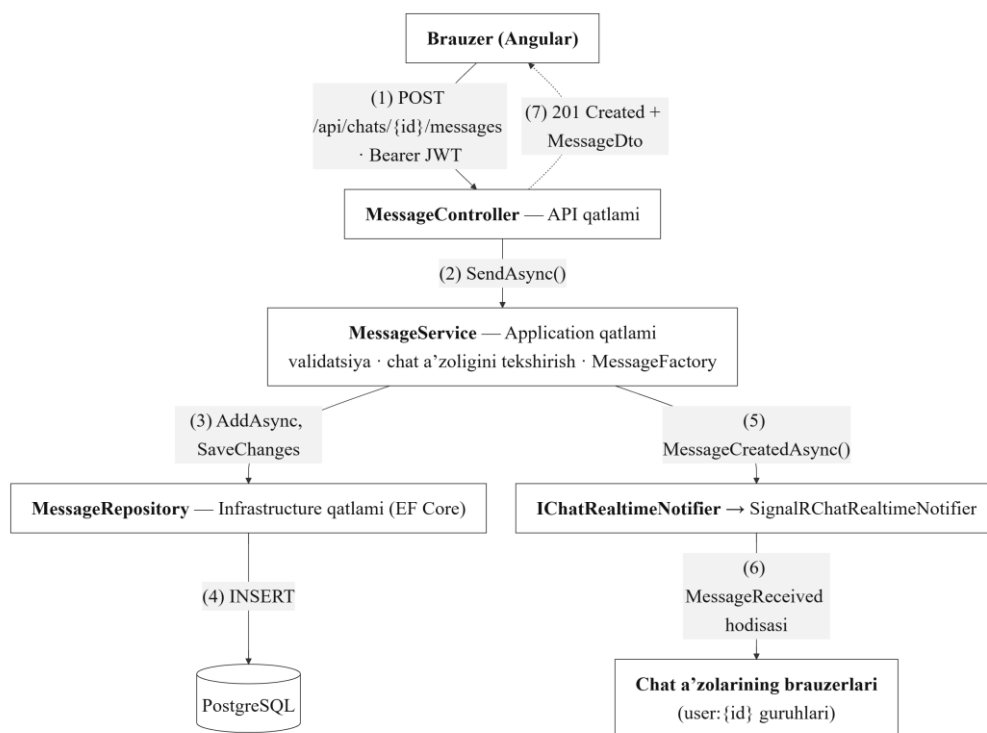
- **Infrastructure** — Application interfeyslarining texnik amalga oshirilishi: EF Core ma'lumotlar konteksti (ChatDb) va repozitoriylar, JWT tokenlar chiqaruvchisi, PBKDF2 va HMAC xeshlovchilari, Google token tekshiruvchisi, SMTP orqali xat yuboruvchi, ClamAV skaneri, ImageSharp va FFmpeg asosidagi media ishlov beruvchilar, onlayn holat kuzatuvchilari, shifrlangan fayllar ombori (LocalSecretFileStorage) va fon ishlari (SearchTextBackfill, SecretChatCleanup);
- **API** — kompozitsiya ildizi va HTTP transport qatlami: REST controllerlar, SignalR habi (ChatHub), xatolarni markazlashgan holda qayta ishlovchi va xavfsizlik sarlavhalarini qo'shuvchi middleware'lar hamda barcha bog'liqliklarni ro'yxatga oluvchi kengaytma metodlar.

Application qatlami ma'lumotlar bazasi yoki SignalR haqida hech narsa bilmaydi. Masalan, xabar yuborilgandan so'ng MessageService faqat IChatRealtimeNotifier interfeysining MessageCreatedAsync metodini chaqiradi. Bu interfeysning SignalR orqali ishlaydigan amalga oshirilishi (SignalRChatRealtimeNotifier) API qatlamida joylashgan va bog'liqliklar konteynerida ro'yxatdan o'tkazilgan. Bu yondashuv "bog'liqliklarni teskari aylantirish" (Dependency Inversion) tamoyilining amaliy ko'rinishidir.

Application qatlami ichida kod texnik vazifalar bo'yicha emas, **funksiyalar (feature) bo'yicha** guruhlangan: Authentication, Chats, Groups, Messages, Contacts, Profiles, Stories, Users, Files, Organizations, SecretChats. Har bir funksiya papkasi o'z servislari, DTOlari, interfeyslari, validatorlari, mapper va factory sinflarini o'z ichiga oladi. Bu tuzilma yangi funktsiyani qo'shishda o'zgarishlarni bitta joyda to'plash va kodni tushunishni osonlashtiradi. Loyihada

qabul qilingan qoidalar (qatlam chegaralari, UTC vaqt, sahifalash, validatsiya, nomlash) repozitoriydagi AGENTS.md hujjatida qayd etilgan.

HTTP so‘rov qatlamlar bo‘ylab qanday o‘tishi xabar yuborish misolida 2.3-rasmda ko‘rsatilgan. Controller so‘rovni qabul qilib, foydalanuvchi identifikatorini JWT token dan oladi va servisni chaqiradi. Servis ma’lumotlarni validatsiya qiladi, foydalanuvchining chat a’zosi ekanligini tekshiradi, Factory yordamida yangi Message obyektini yaratadi va repozitoriy orqali saqlaydi. Tranzaksiya muvaffaqiyatli yakunlangach, real vaqt xabarnomasi chat a’zolariga yuboriladi. Bu tartib mijozlar hech qachon bazaga yozilmagan xabar haqida bildirishnoma olmasligini kafolatlaydi.



2.3-rasm. Xabar yuborish so‘rovining qatlamlar bo‘ylab o‘tishi

Klient qismining arxitekturası.

Angular ilovasi ham qatlamlarga ajratilgan. **core** papkasida butun ilova uchun umumiy xizmatlar joylashgan: autentifikatsiya servisi (`AuthService`), har bir so‘rovga token qo‘shuvchi va muddati o‘tgan tokenni avtomatik yangilovchi HTTP interceptor, himoyalangan sahifalarni qo‘riqlovchi guard, mavzuni (yorug‘ va qorong‘i) boshqaruvchi servis. **features** papkasida funksional modullar joylashgan:

kirish sahifasi (auth), taklif havolasi sahifasi (invite) va chat moduli. Chat modulining data-access qatlamida ChatApiService (REST so‘rovlar va ilova holati signallari) hamda ChatRealtimeService (SignalR ulanishi va hodisalar oqimi) servislari bor. Komponentlar faqat shu servislar bilan ishlaydi. Maxfiy chatlar alohida **secret** papkasida joylashgan: kriptografik qism (secret-crypto.ts) Angular’ga umuman bog‘liq emas, kalitlar va tarixni IndexedDB’da saqlovchi ombor, protokol holatini boshqaruvchi servis va suhbat komponenti esa uning ustiga quriladi.

shared/text papkasida serverdagi transliteratorning TypeScript’dagi aynan nusxasi va xabarlarni foydalanuvchi tanlagan yozuvda ko‘rsatuvchi uzScript pipe’i joylashgan. **shared/ui** papkasida loyihaning dizayn tizimi (design system) komponentlari joylashgan: tugma, ikonkali tugma, matn maydoni, checkbox, banner, tasdiqlash oynasi, tashkilot belgisi (org-badge) va boshqalar. Barcha ranglar CSS o‘zgaruvchilari (design token) orqali berilgan. Shu sababli yorug‘ va qorong‘i mavzular o‘rtasida o‘tish faqat o‘zgaruvchilar qiymatini almashtirish orqali amalga oshadi.

2.3. Ma’lumotlar bazasini loyihalash

Ma’lumotlar bazasi “kod birinchi” (code-first) yondashuvda loyihalandi: jadvallar tuzilmasi Domain qatlamidagi entity sinflari va Infrastructure qatlamidagi IEntityConfiguration konfiguratsiyalari asosida EF Core tomonidan yaratiladi. Sxemadagi har bir o‘zgarish migratsiya ko‘rinishida saqlanadi va versiyalar nazorati tizimida kuzatiladi.

Jadvallar mantiqiy vazifasiga ko‘ra PostgreSQL’ning o‘nta sxemasiga (schema) ajratilgan. Bu yondashuv katta bazada yo‘nalishni osonlashtiradi, kelajakda esa turli xizmatlarga faqat kerakli sxemalarga kirish huquqini berish imkonini yaratadi. Jami 36 ta jadval loyihalangan. Ularning bir qismi (kanallar, qo‘ng‘iroqlar, stikerlar, botlar, papkalar, so‘rovnomalar, reaksiyalar) kelajakdagi funksiyalar uchun oldindan tayyorlangan. Sxemalar tarkibi 2.2-jadvalda keltirilgan.

Ma'lumotlar bazasi sxemalari va jadvallari

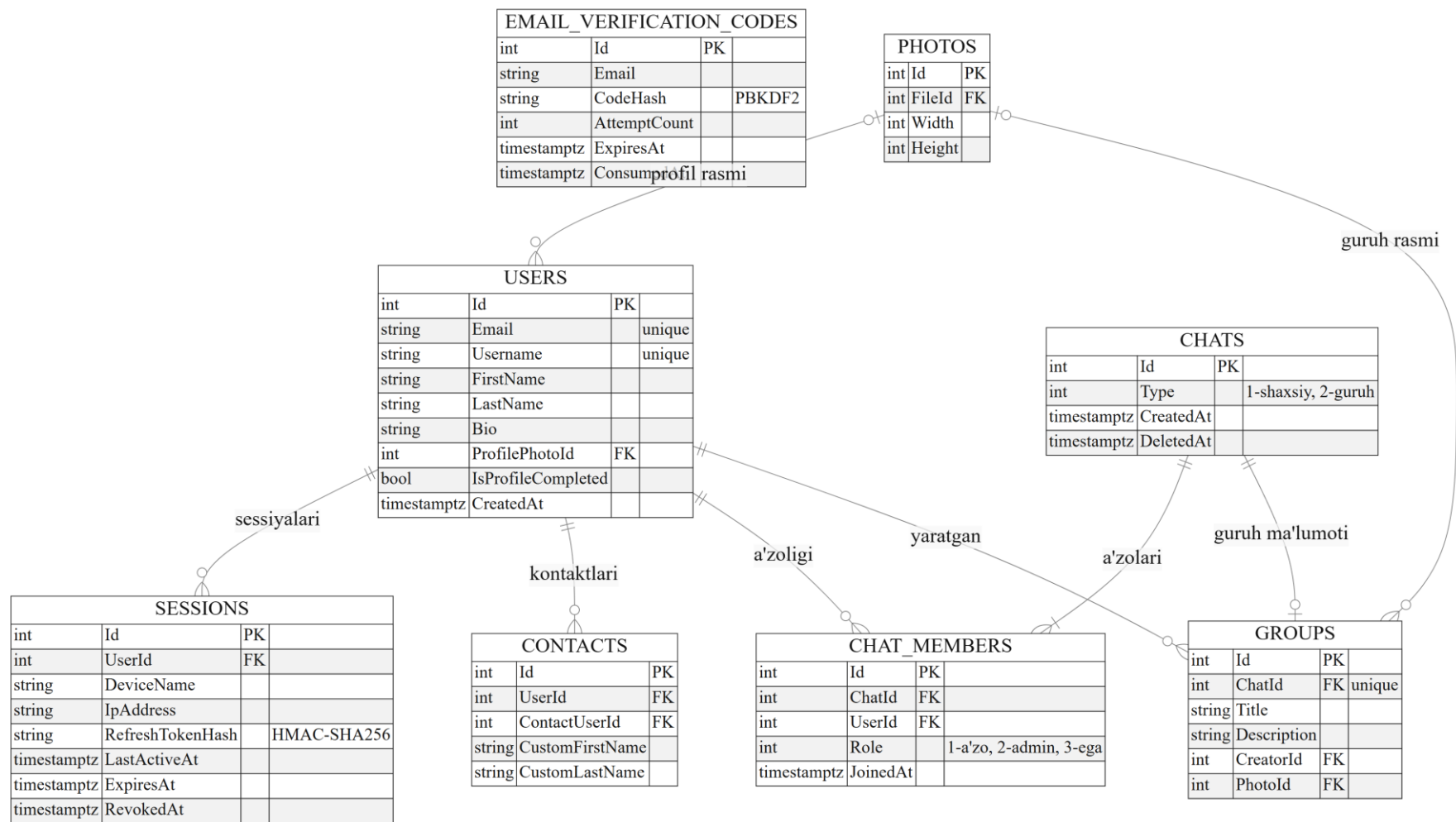
Sxema	Jadvallar	Vazifasi
identity	users, contacts, blocked_users	Foydalanuvchilar va ular o'rtasidagi aloqalar
security	sessions, email_verification_codes, two_factor_auth	Sessiyalar, bir martalik kodlar, qo'shimcha himoya
chat	chats, chat_members, groups, channels, channel_subscribers, secret_chats	Chatlar, a'zolik va rollar, guruh va kanallar, maxfiy chatlar
messaging	messages, attachments, message_views, reactions, polls, poll_options, poll_votes, secret_messages, secret_files	Xabarlar, biriktirmalar, o'qilganlik, reaksiya va so'rovnomalar, yetkazilmagan shifrlangan xabar va fayllar
organizations	organizations, organization_members	Tashkilotlar (e-pochta domenlari) va tasdiqlangan a'zolik
storage	files, photos, stickers, sticker_sets	Fayllar metama'lumotlari, rasmlar, stikerlar
story	stories, story_views	Hikoyalar va ularni ko'rishlar
personal	folders, folder_chats, saved_messages, subscriptions	Shaxsiy papkalar, saqlangan xabarlar, obunalar
call	calls, call_participants	Ovozli va video qo'ng'iroqlar (kelajak uchun)
bot	bots	Botlar platformasi (kelajak uchun)

Tizimning joriy funksiyalarini ta'minlovchi asosiy jadvallar va ular o'rtasidagi bog'lanishlar uchta ER-diagrammada ko'rsatilgan. 2.4-rasmda foydalanuvchilar, sessiyalar, chatlar va guruhlar, 2.5-rasmda xabarlar, fayllar va hikoyalar, 2.6-rasmda esa tizimning o'ziga xos imkoniyatlari — transliteratsiya, tashkilotlar va maxfiy chatlar uchun qo'shilgan jadval va ustunlar tasvirlangan.

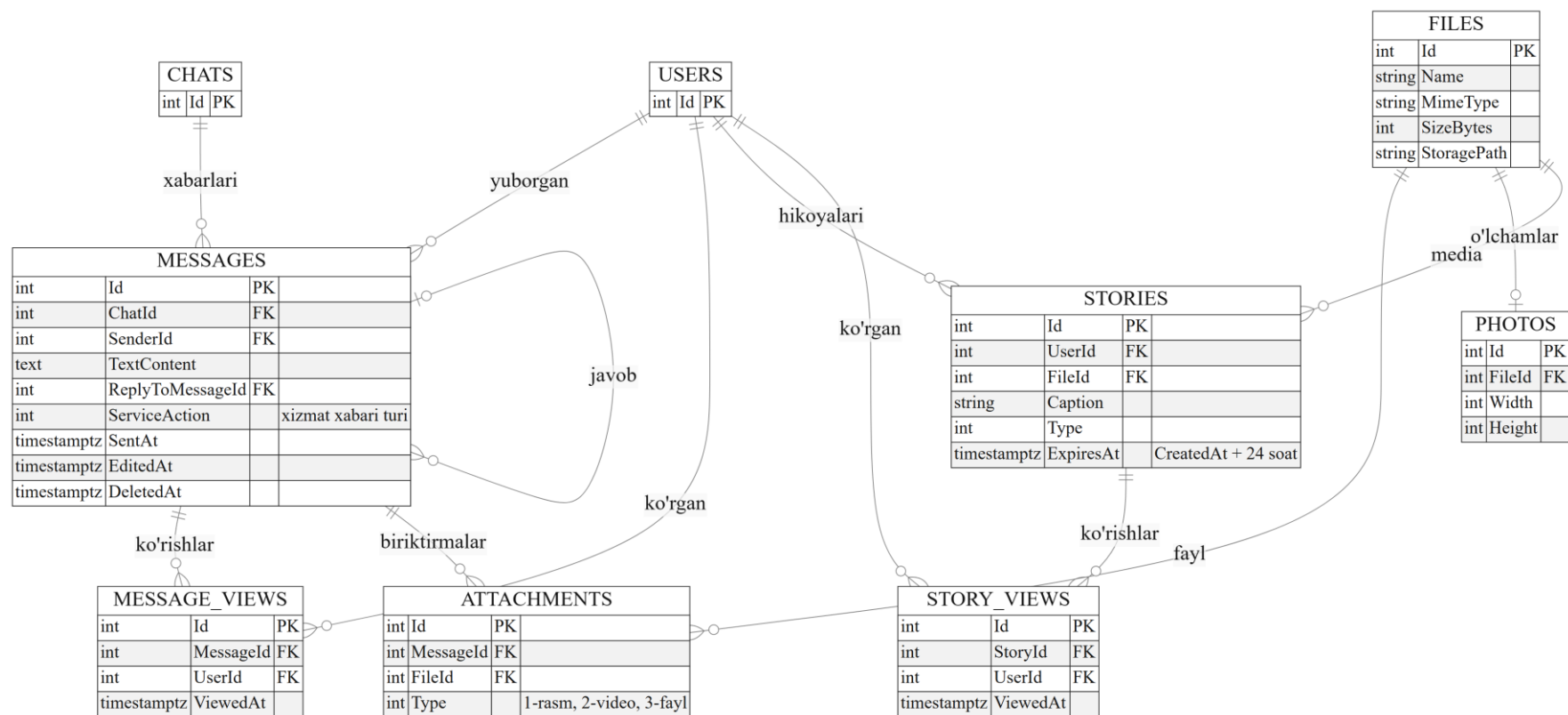
Modelning markazida **chats** jadvali turadi. Shaxsiy chat ham, guruh ham bitta chats yozuvi bilan ifodalanadi va ular Type maydoni (1 — shaxsiy, 2 — guruh) bilan farqlanadi. Chat ishtirokchilari **chat_members** jadvalida saqlanadi. Bu jadval users va chats o'rtasidagi “ko'pdan-ko'pga” bog'lanishni amalga oshiradi va har bir a'zoning rolini (1 — a'zo, 2 — administrator, 3 — ega) hamda qo'shilgan vaqtini saqlaydi. Guruhga xos ma'lumotlar (nomi, tavsifi, rasmi, yaratuvchisi) alohida **groups** jadvaliga chiqarilgan va chats bilan “birga-bir” bog'langan. Buni ChatId

ustunidagi unique indeks ta'minlaydi. Bu yechim tufayli xabarlar, biriktirmalar, qidiruv va real vaqt mexanizmlari shaxsiy chatlar va guruhlar uchun bir xil ishlaydi.

users jadvalida Email va Username ustunlari unique indekslar bilan himoyalangan. Foydalanuvchi sessiyalari **sessions** jadvalida saqlanadi: qurilma nomi, IP-manzil, oxirgi faollik vaqti, amal qilish muddati, bekor qilingan vaqti (RevokedAt) va refresh tokenning xeshi. Refresh token xeshi ustunida ham unique indeks bor, bu token bo'yicha sessiyani tez topish imkonini beradi. Bir martalik kodlar **email_verification_codes** jadvalida faqat xesh ko'rinishida, urinishlar soni va amal qilish muddati bilan birga saqlanadi.



2.4-rasm. ER-diagramma: foydalanuvchilar, sessiyalar, chatlar va guruhlar

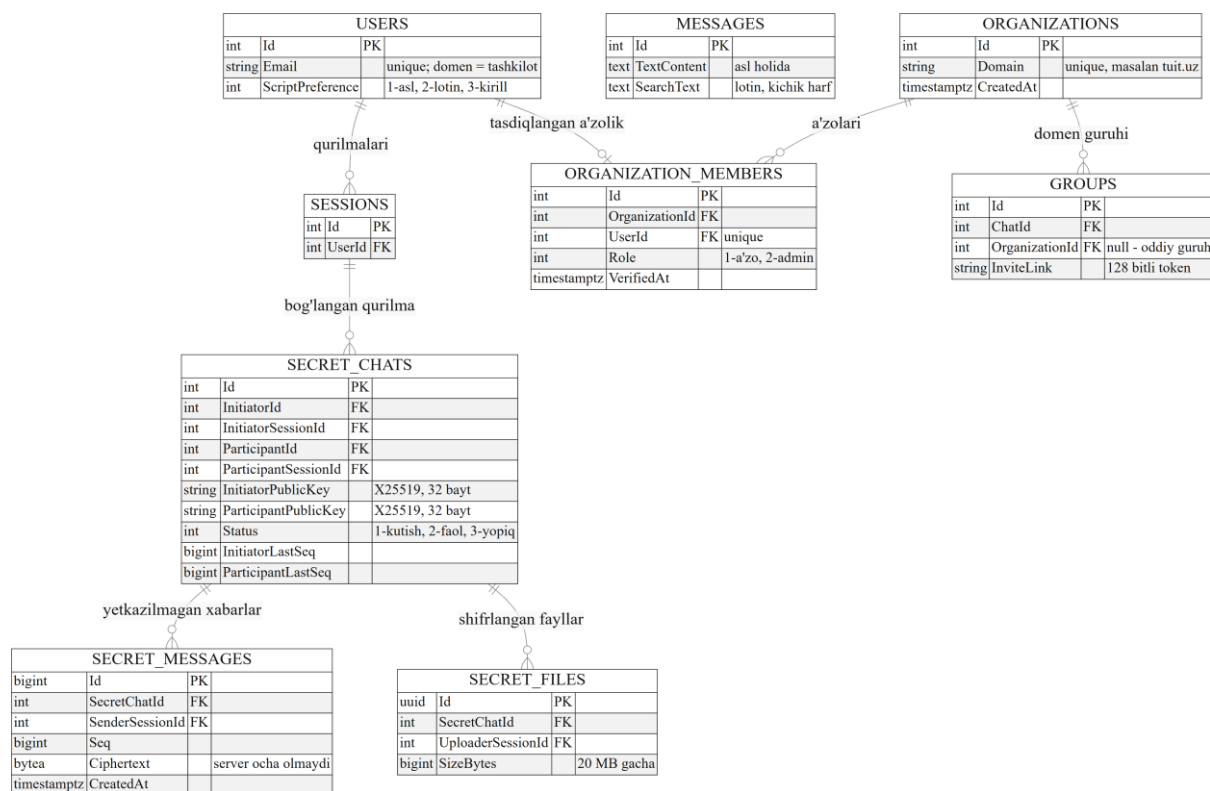


2.5-rasm. ER-diagramma: xabarlar, fayllar va hikoyalar

organizations jadvalida har bir tashkilot bitta e-pochta domeni bilan ifodalanadi (Domain ustuni unique). **organization_members** jadvali foydalanuvchining tashkilotga a'zolicini va rolini (1 — a'zo, 2 — admin) saqlaydi. UserId ustunidagi unique indeks bitta foydalanuvchi faqat bitta tashkilotga tegishli bo'lishini kafolatlaydi: e-pochta manzili bitta, demak domen ham bitta. **groups** jadvaliga OrganizationId ustuni qo'shildi: u to'ldirilgan guruh shu tashkilotning yopiq domen guruhi hisoblanadi. InviteLink ustunida esa taklif havolasining maxfiy tokeni saqlanadi.

Maxfiy chatlar uchun **secret_chats** jadvali qayta loyihalandi. Avvalgi loyihada unda shifrlash kalitining o'zi (EncryptionKey) saqlanishi ko'zda tutilgan edi. Bu uchdan-uchgacha shifrlash tamoyiliga zid: kalit serverda bo'lsa, server yozishmani o'qiy oladi. Yangi loyihada jadvalda faqat ikkala tomonning **ochiq** kalitlari, chat bog'langan qurilmalar (sessiyalar), holat va har bir tomon yuborgan oxirgi xabar tartib raqami saqlanadi. **secret_messages** va **secret_files** jadvallari esa faqat hali yetkazilmagan shifrlangan bloblarni vaqtincha saqlaydi: qabul qiluvchi qurilma ularni olganini tasdiqlashi bilan yozuvlar o'chiriladi. (SecretChatId, SenderSessionId, Seq) ustunlaridagi unique indeks bitta xabarning ikki marta saqlanishiga yo'l qo'ymaydi.

Transliteratsiya uchun **users** jadvaliga ScriptPreference (1 — asl holida, 2 — lotin, 3 — kirill), **messages** jadvaliga esa SearchText ustuni qo'shildi. SearchText xabar matnining lotin yozuviga o'girilgan, kichik harfli va tutuq belgilari birxillashtirilgan nusxasi bo'lib, faqat qidiruv uchun ishlatiladi.



2.6-rasm. ER-diagramma: tashkilotlar, maxfiy chatlar va transliteratsiya uchun qo‘shilgan jadval va ustunlar

messages jadvali tizimdagi eng katta hajmli jadval hisoblanadi, shuning uchun uning tuzilmasiga alohida e’tibor berildi (2.3-jadval). Xabar boshqa xabarga javob bo‘lishi mumkin, buning uchun jadval o‘ziga o‘zi ReplyToMessageId orqali bog‘langan. Asl xabar o‘chirilsa, bu maydon avtomatik NULL qiymatga o‘tadi (ON DELETE SET NULL). ServiceAction maydoni oddiy xabarni guruhdagi xizmat xabaridan (“guruh yaratildi”, “a’zo qo‘shildi”, “nom o‘zgartirildi” va hokazo) farqlaydi.

2.3-jadval

“messaging.messages” jadvalining tuzilmasi

Ustun	Tur	Izoh
Id	integer, PK	Avtomatik oshuvchi identifikator, sahifalash kursori
ChatId	integer, FK → chats	Xabar tegishli chat (chat o‘chirilsa, kaskad)
SenderId	integer, FK → users	Jo‘natuvchi (foydalanuvchi o‘chirilishi cheklangan)
TextContent	text, NULL	Xabar matni (faqat fayldan iborat xabarda bo‘sh)

Ustun	Tur	Izoh
SearchText	text, NULL	Qidiruv kaliti: matnning lotin, kichik harfli, tutuq belgilari birxillashtirilgan nusxasi
ReplyToMessageId	integer, FK, NULL	Javob berilgan xabar (ON DELETE SET NULL)
ServiceAction	integer, NULL	Xizmat xabari turi: 1 — guruh yaratildi, 2 — a’zolar qo’shildi, 3 — chiqarildi, 4 — chiqib ketdi, 5 — nom, 6 — rasm o’zgardi
SentAt	timestampz	Yuborilgan vaqt (UTC)
EditedAt	timestampz, NULL	Oxirgi tahrir vaqti
DeletedAt	timestampz, NULL	Yumshoq o’chirish belgisi

Loyihalash qarorlari.

- **Vaqt.** Barcha vaqt belgilari UTC’da saqlanadi va PostgreSQL’da “timestamp with time zone” turiga ega. Server kodida joriy vaqt TimeProvider abstraksiyasi orqali olinadi, bu esa vaqtga bog’liq mantiqni sinashni osonlashtiradi. Foydalanuvchining mahalliy vaqtiga o’girish faqat brauzerda amalga oshiriladi;
- **Yumshoq o’chirish (soft delete).** Chat yoki xabar o’chirilganda yozuv jismonan o’chirilmaydi, balki DeletedAt maydoniga vaqt yoziladi. EF Core’ning global so’rov filtrlari (HasQueryFilter) bunday yozuvlarni barcha so’rovlardan avtomatik chiqarib tashlaydi;
- **O’qilganlik.** Xabar kim tomonidan va qachon o’qilgani alohida message_views jadvalida saqlanadi. Bu yechim guruh chatlarida har bir a’zo uchun o’qilmagan xabarlar sonini alohida hisoblash imkonini beradi;
- **Fayllar.** Fayl ma’lumotlar bazasida emas, diskda saqlanadi. files jadvalida faqat nomi, MIME turi, hajmi va saqlash yo’li yoziladi, rasmlar uchun esa photos jadvalida o’lchamlari qo’shimcha saqlanadi. Bitta fayl xabarga biriktirma, profil rasmi, guruh rasmi yoki hikoya sifatida qayta ishlatilishi mumkin;
- **Hujjatlashtirilgan enum’lar.** Sanab o’tiladigan turlar bazada butun son sifatida saqlanadi, har bir qiymatning ma’nosi esa ustun izohiga (masalan, “1 = Member, 2 = Admin, 3 = Creator”) avtomatik yoziladi;

- **Indekslar.** Tez-tez bajariladigan so‘rovlar uchun indekslar yaratilgan: users(Email), users(Username), sessions(RefreshTokenHash) — unique; sessions(UserId, RevokedAt, ExpiresAt), email_verification_codes(Email, Purpose, ExpiresAt) — tarkibiy; groups(ChatId), organizations(Domain), organization_members(UserId), secret_messages(SecretChatId, SenderSessionId, Seq) — unique.

Kursorli sahifalash.

Chatlar ro‘yxati va xabarlar tarixi doimiy o‘zgarib turadigan, tartiblangan ma’lumotlardir. Klassik “sahifa raqami” (OFFSET/LIMIT) usulida yangi xabar qo‘shilganda sahifalar siljiydi, natijada foydalanuvchi bir xabarni ikki marta ko‘radi yoki ayrim xabarlarni o‘tkazib yuboradi. Katta OFFSET qiymatlarida esa MBBT o‘tkazib yuboriladigan qatorlarni ham o‘qishga majbur bo‘ladi va so‘rov sekinlashadi.

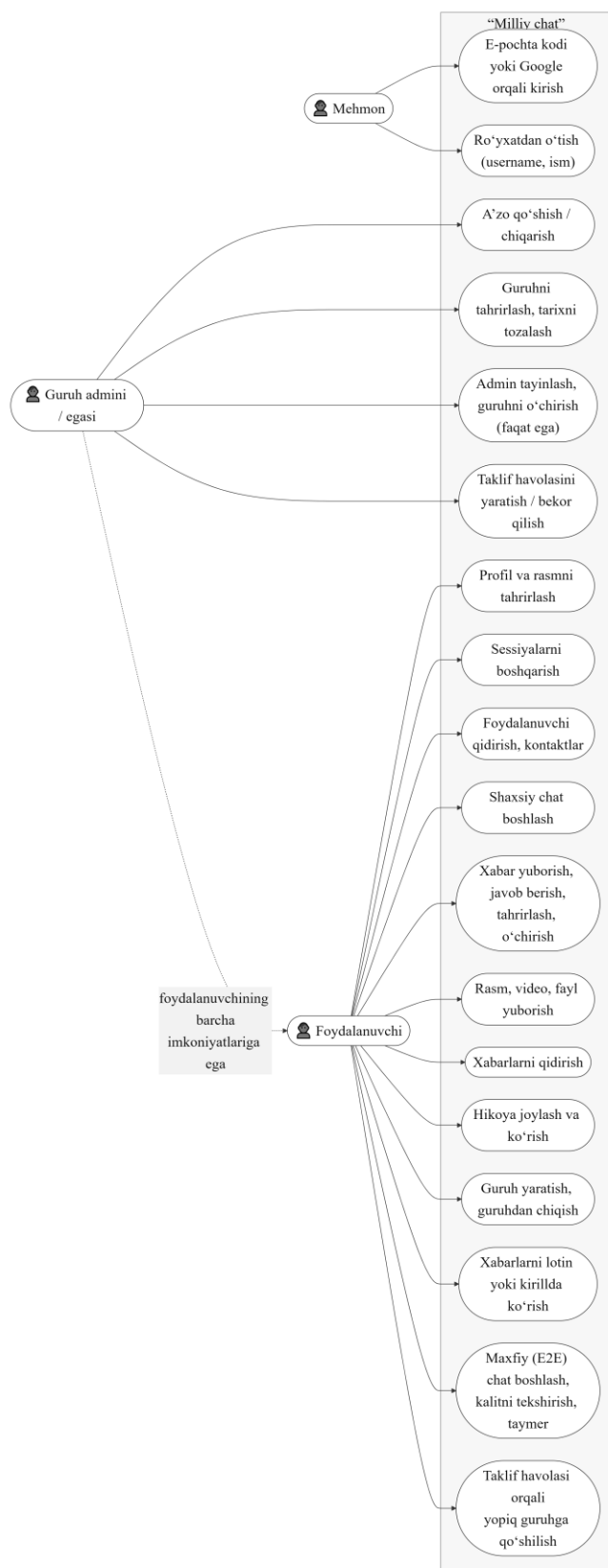
Shu sababli tizimda **kursorli (keyset) sahifalash** qo‘llanildi. Mijoz “beforeId” (shu identifikatordan oldingi yozuvlar) va “limit” (1 dan 100 gacha) parametrlarini yuboradi. Server yozuvlarni identifikator bo‘yicha kamayish tartibida saralaydi, limit+1 ta yozuvni o‘qiydi va javobda items (yozuvlar), nextCursor (keyingi so‘rov uchun kursor) va hasMore (yana yozuvlar bormi) maydonlarini qaytaradi. Bu so‘rov birlamchi kalit indeksi bo‘yicha bajariladi, shuning uchun uning tezligi tarix hajmiga bog‘liq emas. Chatlar ro‘yxatida kursor sifatida chatdagi oxirgi xabarning identifikatori olinadi. Shu tufayli yangi xabar kelgan chat avtomatik ravishda ro‘yxat boshiga chiqadi.

2.4. Tizimning UML modellari

Tizimning funksional imkoniyatlari va komponentlar o‘rtasidagi o‘zaro ta’sir UML (Unified Modeling Language) tilida modellashtirildi. Foydalanish holatlari diagrammasi (use-case) tizim nima qilishini, ketma-ketlik diagrammalari (sequence) esa asosiy jarayonlar qanday bajarilishini ko‘rsatadi.

Tizimda uchta aktor mavjud. **Mehmon** — hali tizimga kirmagan shaxs: u faqat ro‘yxatdan o‘tishi yoki kirishi mumkin. **Foydalanuvchi** — autentifikatsiyadan

o'tgan shaxs: u profilini boshqaradi, boshqa foydalanuvchilar bilan yozishadi, fayl va hikoyalar almashadi, guruh yaratadi. Bundan tashqari, u xabarlarni o'zi tanlagan yozuvda (lotin yoki kirill) ko'radi, boshqa foydalanuvchi bilan maxfiy chat ochadi va taklif havolasi orqali yopiq guruhga qo'shiladi. **Guruh administratori yoki egasi** foydalanuvchining barcha imkoniyatlariga ega, qo'shimcha ravishda esa guruh a'zolari, sozlamalari va taklif havolasini boshqaradi. Foydalanish holatlari diagrammasi 2.7-rasmda keltirilgan.



2.7-rasm. Foydalanish holatlari (use-case) diagrammasi

Guruh chatlarida harakatlar rolga qarab cheklanadi. Ruxsatlar Telegram'dagi mantiqqa yaqin qilib loyihalangan va 2.4-jadvalda keltirilgan. Barcha ruxsatlar

server tomonida GroupService, ChatService va MessageService servislarida tekshiriladi. Klient interfeysi esa ruxsat berilmagan tugmalarni ko'rsatmaydi.

2.4-jadval

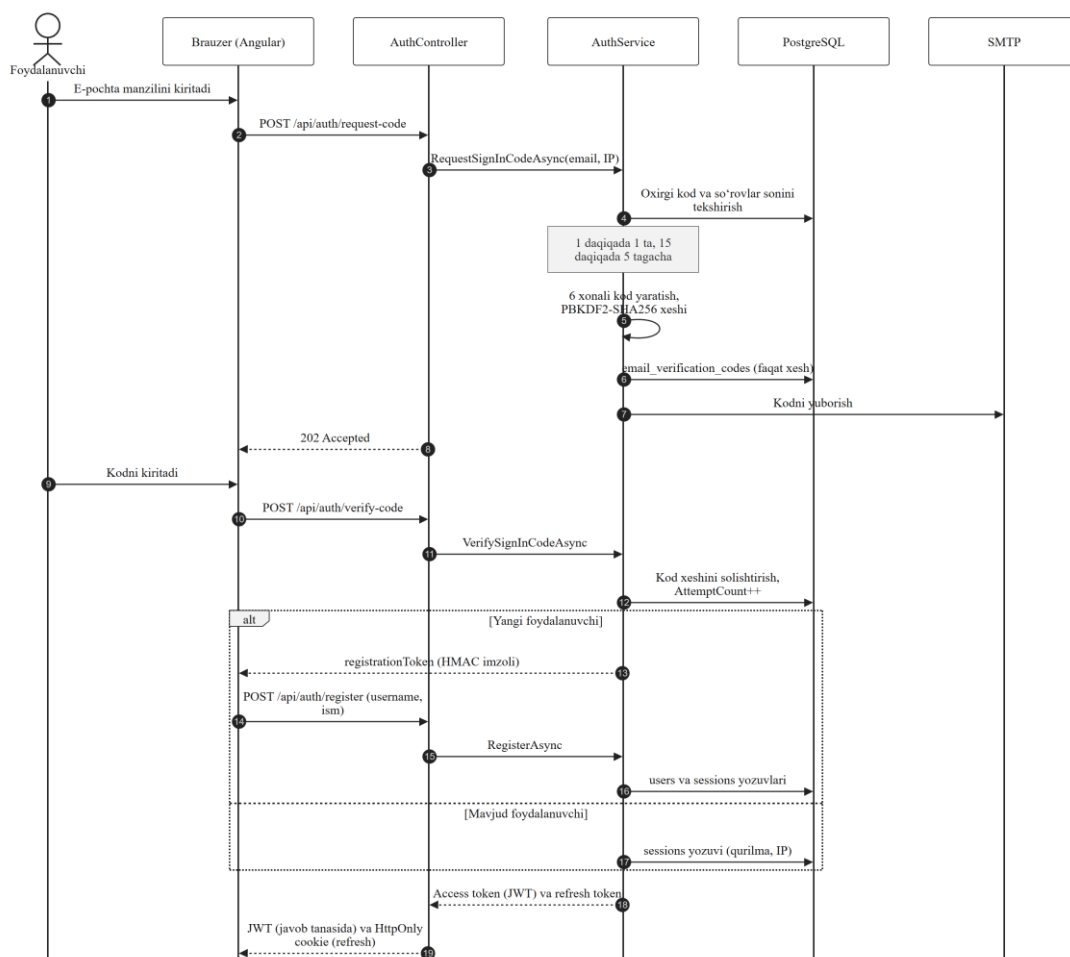
Guruh chatidagi rollar va ruxsatlar

Amal	A'zo	Administrator	Ega
Xabar yozish, o'qish, fayl yuborish	✓	✓	✓
A'zo qo'shish	—	✓	✓
A'zoni guruhdan chiqarish	—	Faqat oddiy a'zoni	✓ (hammani)
Administrator tayinlash yoki olib tashlash	—	—	✓
Guruh nomi, tavsifi va rasmini o'zgartirish	—	✓	✓
Taklif havolasini yaratish, yangilash, bekor qilish	—	✓	✓
Chat tarixini tozalash	—	✓	✓
Guruhni butunlay o'chirish	—	—	✓
Guruhdan chiqish	✓	✓	✓ (egalik boshqa a'zoga o'tadi)

Elektron pochta orqali kirish jarayoni.

Tizimga kirish parolsiz amalga oshiriladi: foydalanuvchi elektron pochta manzilini kiritadi va unga yuborilgan 6 xonali kodni tasdiqlaydi (2.8-rasm). Server avval so'rovlar chastotasini tekshiradi: bitta manzilga daqiqada bir martadan ko'p kod yuborilmaydi, 15 daqiqalik oynada esa bitta manzil yoki bitta IP-manzildan beshtadan ortiq so'rov qabul qilinmaydi. Kod kriptografik tasodifiy sonlar generatori yordamida yaratiladi va bazaga faqat PBKDF2 xeshi ko'rinishida yoziladi. Kod 10 daqiqa amal qiladi, uni kiritishga esa beshtagacha urinish beriladi.

Agar bu manzil bilan birinchi marta kirilayotgan bo'lsa, server HMAC bilan imzolangan vaqtinchalik ro'yxatdan o'tish tokenini qaytaradi. Foydalanuvchi username va ismini kiritib, ro'yxatdan o'tishni yakunlaydi. Mavjud foydalanuvchi uchun esa darhol yangi sessiya yaratiladi. Har ikki holatda ham mijoz javob tanasida qisqa muddatli access tokenni, HttpOnly cookie'da esa refresh tokenni oladi.



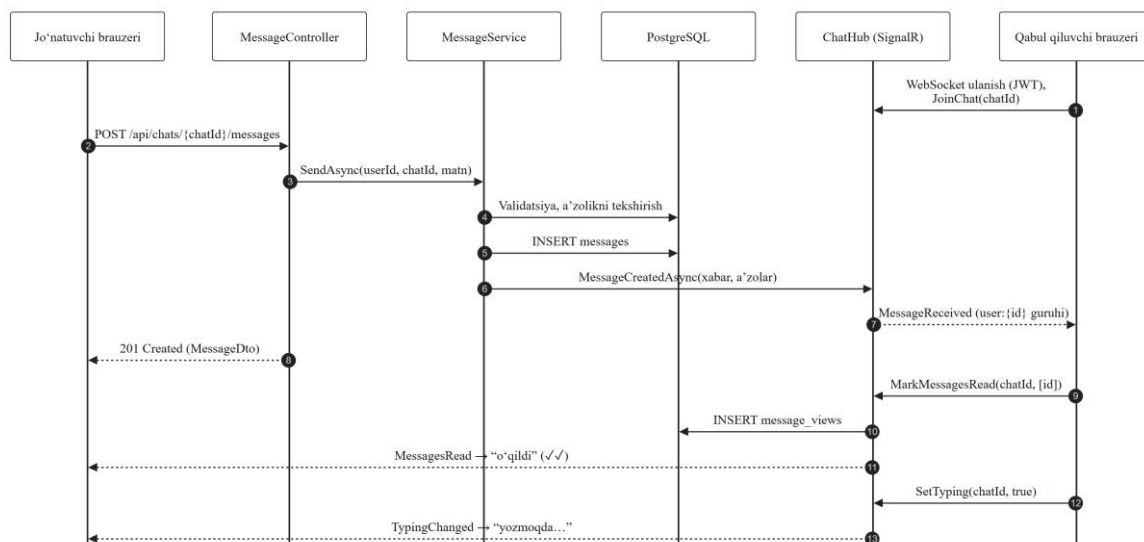
2.8-rasm. Elektron pochtaga yuborilgan kod orqali kirish ketma-ketlik diagrammasi

Real vaqtda xabar yuborish jarayoni.

Xabar almashish jarayoni 2.9-rasmda ko‘rsatilgan. Brauzer tizimga kirgach, JWT token bilan SignalR habiga WebSocket ulanishini o‘rnatadi. Server har bir ulanishni “user:{id}” nomli guruhga qo‘shadi. Shu tufayli foydalanuvchining barcha qurilmalari va brauzer oynalari hodisalarni bir vaqtda oladi. Foydalanuvchi chatni ochganda ulanish qo‘shimcha ravishda “chat:{id}” guruhiga qo‘shiladi. Bu guruh faqat “yozmoqda” kabi tez-tez keladigan va faqat chatni ochib turganlar uchun dolzarb bo‘lgan hodisalar uchun ishlatiladi.

Xabarning o‘zi REST so‘rovi orqali yuboriladi. Bu yondashuv xabarni saqlash va validatsiya qilishni oddiy HTTP semantikasi bilan (status kodlar, xatolar) bajarish imkonini beradi. Xabar bazaga yozilgach, u chat a‘zolarining “user:{id}” guruhlariga MessageReceived hodisasi sifatida yuboriladi. Qabul qiluvchining brauzeri chat ochiq bo‘lsa, darhol MarkMessagesRead hab metodini chaqiradi.

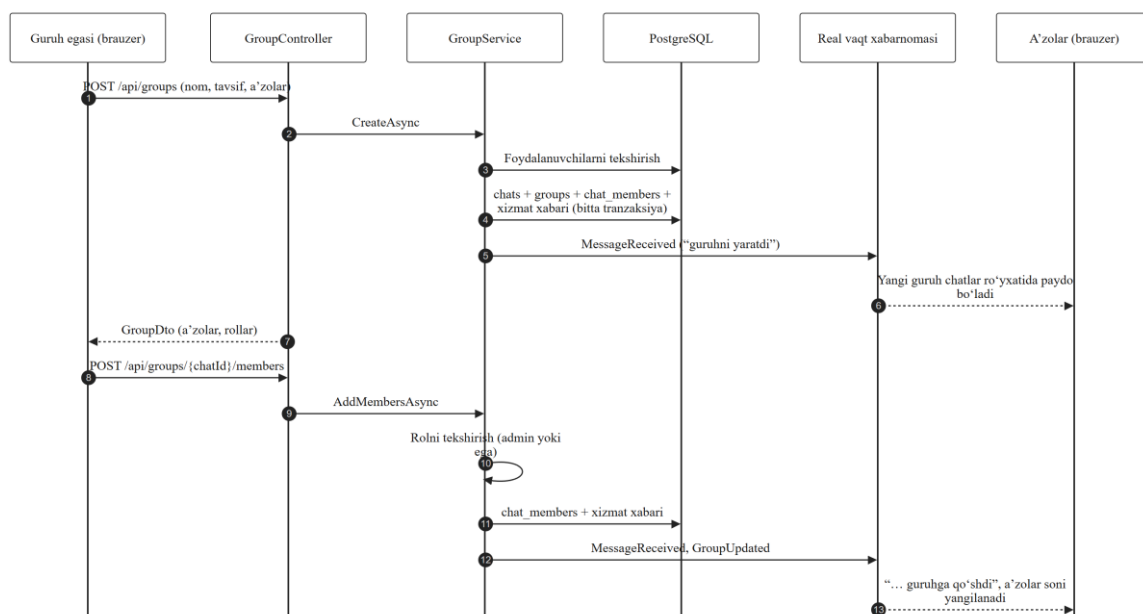
Server message_views jadvaliga yozuv qo‘shadi va jo‘natuvchiga MessagesRead hodisasini yuboradi. Natijada jo‘natuvchida xabar yonidagi bitta belgi ikki belgiga (✓✓) almashadi.



2.9-rasm. Real vaqtda xabar yuborish va o‘qilganlik belgisi ketma-ketlik diagrammasi

Guruh yaratish jarayoni.

Guruh yaratilganda chats, groups, chat_members jadvallariga yozuvlar va birinchi xizmat xabari (“... guruhini yaratdi”) bitta tranzaksiyada saqlanadi (2.10-rasm). Xizmat xabari muhim vazifani bajaradi: u barcha a’zolariga MessageReceived hodisasi sifatida yetib boradi va mijoz noma’lum chatdan xabar kelganda chatlar ro‘yxatini qayta yuklaydi. Shunday qilib, yangi guruh alohida hodisa turisiz, barcha a’zolarida bir zumda paydo bo‘ladi. A’zo qo‘shish, chiqarish, nom yoki rasmni o‘zgartirish ham xizmat xabarlari bilan qayd etiladi. Guruh ma’lumotlari o‘zgarganda esa qo‘shimcha GroupUpdated hodisasi yuboriladi.



2.10-rasm. Guruh yaratish va a'zo qo'shish ketma-ketlik diagrammasi

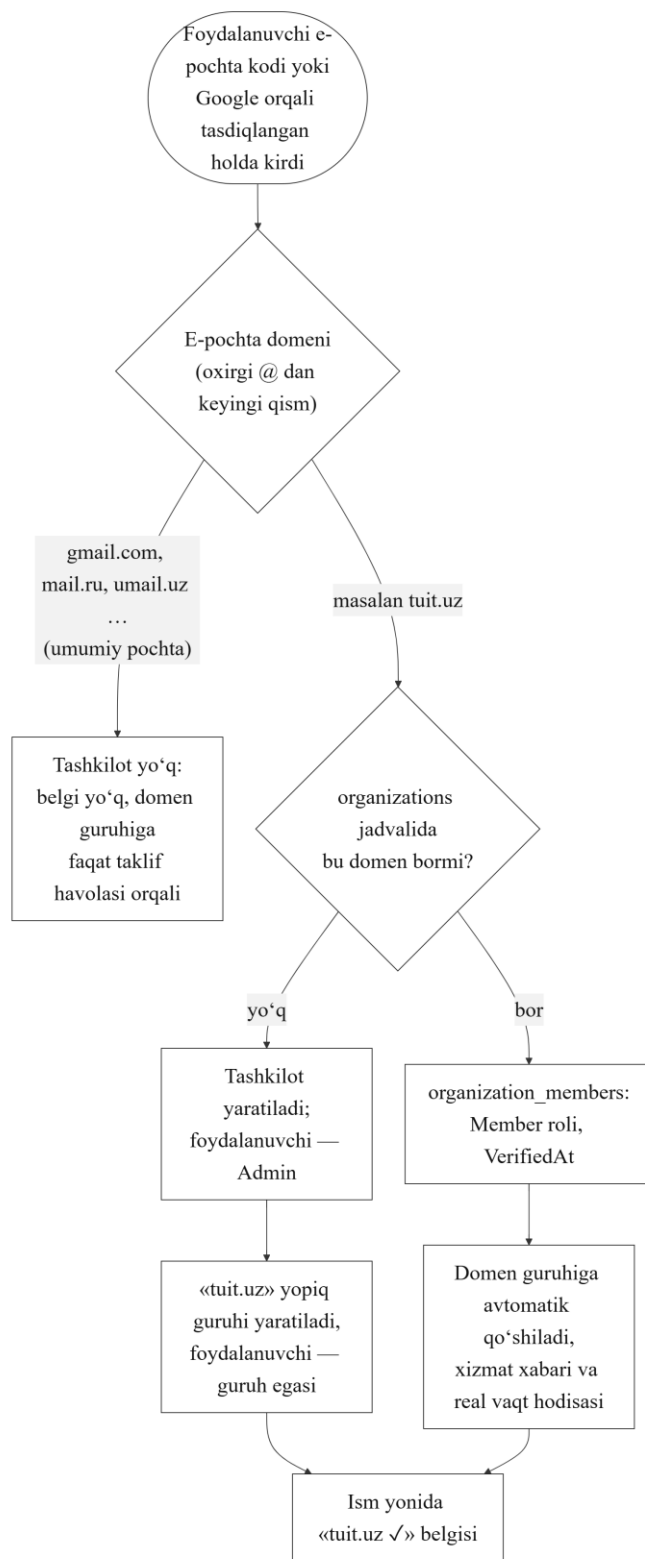
Tashkilotga avtomatik qo'shilish jarayoni.

Tashkilot rejimi alohida ro'yxat yoki sozlamalarni talab qilmaydi: **tashkilot — bu e-pochta domenining o'zi**. Foydalanuvchi tasdiqlangan pochta bilan (bir martalik kod yoki Google hisobi orqali) kirganda server manzilning oxirgi “@” belgisidan keyingi qismini ajratib oladi va uni normallashtiradi: kichik harflarga o'tkazadi, oxiridagi nuqtani olib tashlaydi, domen formatini tekshiradi. “tuit.uz” va “student.tuit.uz” ikki alohida tashkilot hisoblanadi, “evil-tuit.uz” kabi o'xshash domenlar esa hech qachon “tuit.uz” bilan mos kelmaydi, chunki domenlar faqat to'liq tengligi bo'yicha solishtiriladi.

Hamma ro'yxatdan o'ta oladigan umumiy pochta xizmatlari (gmail.com, mail.ru, yandex.ru, umail.uz, vaqtinchalik pochta xizmatlari va boshqalar — jami 40 ga yaqin domen) tashkilot hisoblanmaydi. Ularning foydalanuvchilari tashkilot belgisini olmaydi va domen guruhlariga avtomatik qo'shilmaydi. Jarayon 2.11-rasmda ko'rsatilgan: domendan birinchi bo'lib kirgan foydalanuvchi tashkilotning admini va uning yopiq guruhining egasi bo'ladi, keyingilar esa a'zo sifatida guruhga avtomatik qo'shiladi. Har bir qo'shilish guruhda “... @tuit.uz pochta bilan guruhga qo'shildi” xizmat xabari bilan qayd etiladi.

Domen guruhi **yopiq**: uni qidiruv orqali topib bo'lmaydi. Tashqaridan (masalan, gmail.com foydalanuvchisi) unga faqat admin qo'shganda yoki taklif

havolasi orqali kirish mumkin. Havola tokeni 128 bitli kriptografik tasodifiy son bo'lib, uni taxmin qilib topish amalda imkonsiz. Admin havolani istalgan vaqtda yangilashi yoki bekor qilishi mumkin, shunda eski havola darhol ishlamay qoladi. Tashkilot guruhlari uchun a'zolar chegarasi oddiy guruhlardagi 200 o'rniga 5000 qilib belgilangan, chunki bunday guruhga butun fakultet yoki kompaniya kirishi mumkin.



2.11-rasm. Foydalanuvchini tashkilotga va uning domen guruhiga qo‘shish algoritmi

2.5. Xavfsizlikni loyihalash

Messenjer shaxsiy yozishmalarni saqlagani uchun xavfsizlik loyihalashning boshidan hisobga olindi. Himoya bir necha darajada tashkil etilgan: autentifikatsiya,

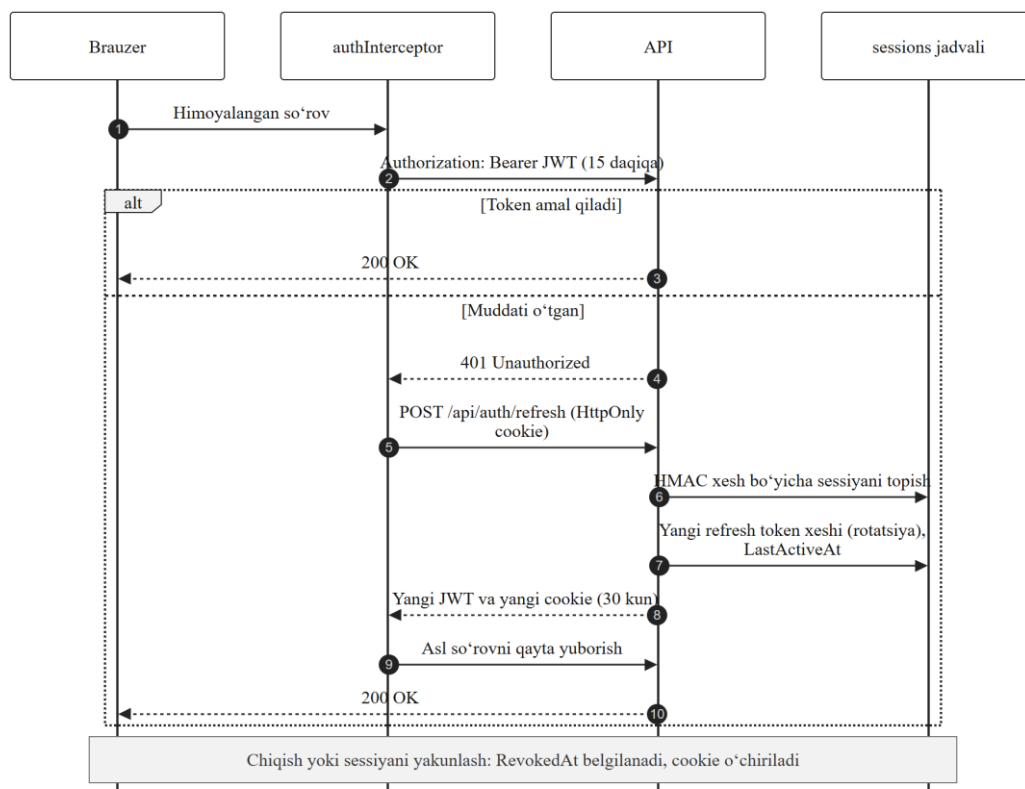
sessiyalarni boshqarish, avtorizatsiya, kiruvchi ma'lumotlar va fayllarni tekshirish, veb-ilovaning o'zini himoyalash. Serverning o'zidan ham himoyalinishi kerak bo'lgan yozishmalar uchun esa maxfiy chatlar protokoli loyihalandi (2.6-bo'lim).

Autentifikatsiya va tokenlar.

Tizim parollardan foydalanmaydi, shuning uchun parollar bazasining o'g'irlanishi yoki zaif parollar muammosi yo'q. Bir martalik kodlar PBKDF2 algoritmi (HMAC-SHA256, 210 000 iteratsiya, 16 baytli tasodifiy tuz) bilan xeshlanadi. Kodni tekshirishda vaqtga bog'liq bo'lmagan (constant-time) taqqoslash qo'llanadi. Google orqali kirishda server brauzerdan kelgan ID tokenning imzosini, muddatini va auditoriyasini (loyihaning Client ID'si) Google.Apis.Auth kutubxonasi yordamida tekshiradi.

Muvaffaqiyatli kirishdan so'ng ikki turdagi token beriladi. **Access token** — HS256 algoritmi bilan imzolangan JWT. U foydalanuvchi identifikatori va sessiya identifikatorini (sid) o'z ichiga oladi va 15 daqiqa amal qiladi. U har bir API so'rovining Authorization sarlavhasida, SignalR ulanishida va media oqimlarida (video) esa access_token parametrda yuboriladi. **Refresh token** — 30 kun amal qiladigan kriptografik tasodifiy qator. U JavaScript'dan o'qib bo'lmaydigan cookie'da (HttpOnly, ishlab chiqarish muhitida Secure, SameSite=Strict, faqat /api/auth yo'li uchun) saqlanadi. Ma'lumotlar bazasiga refresh tokenning o'zi emas, faqat HMAC-SHA256 xeshi yoziladi.

Access token muddati tugaganda klientdagi HTTP interceptor 401 javobini ushlaydi, /api/auth/refresh so'rovini yuboradi va asl so'rovni yangi token bilan takrorlaydi (2.12-rasm). Bir vaqtda bir nechta so'rov 401 olsa, ular bitta refresh so'rovini baham ko'radi. Har bir yangilashda refresh token almashtiriladi (rotatsiya). Shu sababli o'g'irlangan eski token qayta ishlatilsa, u endi hech qanday sessiyaga mos kelmaydi.



2.12-rasm. Access va refresh tokenlarning hayotiy sikli

Sessiyalarni boshqarish.

Har bir kirish alohida sessiya yaratadi. Unda qurilma nomi, tizim versiyasi, IP-manzil, yaratilgan va oxirgi faollik vaqti saqlanadi. Foydalanuvchi profil sozlamalarida barcha faol qurilmalarni ko‘rishi, begona qurilmani chiqarib yuborishi yoki joriy qurilmadan tashqari barcha sessiyalarni bir vaqtda yakunlashi mumkin. Sessiya yakunlanganda uning RevokedAt maydoni to‘ldiriladi va u bilan bog‘liq refresh token darhol yaroqsiz bo‘ladi. Access token esa muddati tugashi bilan, ya’ni ko‘pi bilan 15 daqiqada o‘z kuchini yo‘qotadi.

Avtorizatsiya.

Barcha controllerlar va SignalR habi [Authorize] atributi bilan himoyalangan. Ammo tokenning mavjudligi yetarli emas: har bir chat bilan bog‘liq amalda server foydalanuvchining shu chat a‘zosi ekanligini ma’lumotlar bazasidan tekshiradi. Masalan, xabarlarni o‘qish, yuborish, qidirish, media yuklash, JoinChat, SetTyping va MarkMessagesRead hab metodlari chaqirilganda shunday tekshiruv bajariladi. Foydalanuvchi faqat o‘z xabarini tahrirlashi yoki o‘chirishi mumkin. Guruhda esa

2.4-jadvaldagi rolga asoslangan qoidalar qo'llanadi. Guruhdan chiqarilgan foydalanuvchi keyingi so'rovdayoq chat ma'lumotlariga kirish huquqini yo'qotadi.

Kiruvchi ma'lumotlar va fayllar.

API orqali keladigan har bir DTO uchun FluentValidation validatori yozilgan (matn uzunligi, identifikatorlar, sahifalash chegaralari va hokazo). Validatsiya ikki darajada bajariladi: ASP.NET Core'ning avtomatik tekshiruv va servis ichidagi qayta tekshiruv. Kutilmagan xatolar ExceptionHandlingMiddleware tomonidan ushlanadi. Mijozga ichki tafsilotlarsiz, bir xil formatdagi javob qaytariladi, to'liq ma'lumot esa server logiga yoziladi.

Yuklanadigan fayllar hajmi cheklangan: rasm uchun 10 MB, boshqa fayllar uchun 50 MB. Rasm faylning kengaytmasiga emas, uning haqiqiy tarkibiga qarab ImageSharp kutubxonasi yordamida tekshiriladi. Faqat JPEG, PNG va WebP formatlari qabul qilinadi, rasm qayta kodlanadi, ya'ni yashirin metama'lumotlar va zararli qo'shimchalar olib tashlanadi. Katta rasmlar uchun 256 pikseli kichik nusxa (thumbnail) yaratiladi. Fayllar diskda tasodifiy (GUID) nom bilan saqlanadi. Chatga yuborilgan rasm, video va fayllar faqat shu chat a'zosiga beriladi: server fayl birlashtirilgan xabar chatining a'zolari orasida so'rovchi borligini tekshiradi. Sozlamalarda yoqilgan bo'lsa, har bir fayl saqlashdan oldin ClamAV antivirusi bilan tekshiriladi.

Veb-ilovani himoyalash.

Brauzerda ishlaydigan ilovaning eng xavfli zaifligi — begona skriptning sahifaga kirib olishi (XSS). Maxfiy chatlarda bu ayniqsa muhim: sahifada ishlayotgan har qanday skript shifrlash kalitlaridan foydalana oladi. Shu sababli klient ilovasi qat'iy **Content Security Policy** (CSP) bilan himoyalangan. Skriptlar faqat ilovaning o'z manzilidan va Google Identity xizmatidan yuklanadi, sahifa ichidagi skriptlarga esa faqat SHA-256 xeshi oldindan ro'yxatga olinganlari ruxsat etiladi. Tarmoq so'rovlari faqat ilova serveriga yuboriladi, pluginlar (object-src) taqiqlangan, sahifani boshqa sayt ichiga joylashtirish (frame-ancestors) man etilgan.

API javoblariga ham xavfsizlik sarlavhalari qo'shiladi: X-Content-Type-Options: nosniff (brauzer javob turini "taxmin qilib", uni sahifa sifatida bajarmasligi

uchun), Content-Security-Policy: default-src none (API javobi hech qachon sahifa sifatida ishlamasligi uchun) va Referrer-Policy: no-referrer. Refresh token cookie'si boshqa saytdan faqat ruxsat etilgan manzillar (Cors:AllowedOrigins) ro'yxati sozlangandagina yuboriladi.

Loyihalash jarayonida aniqlangan asosiy tahdidlar va ularga qarshi ko'rilgan choralar 2.5-jadvalda umumlashtirilgan.

2.5-jadval

Asosiy tahdidlar va himoya choralari

Tahdid	Himoya choralari
Kodni tanlab topish (brute-force)	Kod 10 daqiqa amal qiladi, 5 tagacha urinish; kod so'rash chastotasi e-pochta va IP bo'yicha cheklangan
Ma'lumotlar bazasining sizib chiqishi	Kodlar PBKDF2, refresh tokenlar HMAC-SHA256 xeshi ko'rinishida saqlanadi; parollar yo'q
Tokenning o'g'irlanishi	Access token 15 daqiqa; refresh token HttpOnly cookie'da va har safar rotatsiya qilinadi; sessiyani masofadan yakunlash mumkin
CSRF (so'rovni soxtalashtirish)	Refresh cookie SameSite=Strict va faqat /api/auth yo'li uchun; boshqa so'rovlar Authorization sarlavhasini talab qiladi
XSS (skript kiritish)	Angular shablonlari ma'lumotlarni avtomatik ekranlaydi; qat'iy CSP; refresh token JavaScript'ga ko'rinmaydi; access token qisqa muddatli
Begona chatga kirish	Har bir REST va hab chaqiruvida a'zolik tekshiruvi; guruhda rolga asoslangan ruxsatlar
Zararli yoki soxta fayl	Hajm cheklovlari, tarkib bo'yicha format tekshiruvi, rasmni qayta kodlash, ClamAV skaneri
Tinglab olish (MITM)	Barcha trafik HTTPS/WSS orqali; ishlab chiqarish muhitida cookie faqat Secure rejimida
Server yoki ma'lumotlar bazasi buzilishi	Maxfiy chatlarda uchdan-uchgacha shifrlash: serverda faqat ochiq kalitlar va vaqtincha shifrlangan bloblar
Xabarni takrorlash (replay) yoki almashtirish	Maxfiy chatda har xabar tartib raqami (seq) bilan: takrori 409 bilan rad etiladi; AES-GCM AAD maydoni xabarni chat, yo'nalish va raqamga bog'laydi
Yopiq guruhga ruxsatsiz kirish	Guruh qidiruvda ko'rinmaydi; 128 bitli taklif tokeni; havolani yangilash yoki bekor qilish mumkin
Begona domen bilan tashkilotga kirish	Domen faqat tasdiqlangan pochtdan olinadi va to'liq tengligi bo'yicha solishtiriladi; umumiy pochta domenlari chiqarib tashlangan

Kelajakda xavfsizlikni kuchaytirish uchun ikki bosqichli autentifikatsiya (bazada two_factor_auth jadvali tayyor), foydalanuvchilarni bloklash

(blocked_users jadvali) va OneID tizimi orqali shaxsni davlat darajasida tasdiqlash imkoniyatlarini qo‘shish rejalashtirilgan.

2.6. Maxfiy chatlar kriptografik protokolini loyihalash

Oddiy chatlarda xabarlar serverga TLS orqali shifrlangan holda yetib keladi, lekin serverda ochiq matn ko‘rinishida saqlanadi. Bu Telegram’ning bulutli chatlari modeliga mos keladi va barcha qurilmalarda sinxronizatsiya, qidiruv, transliteratsiya kabi imkoniyatlarni beradi. Ammo ayrim yozishmalar serverning o‘zidan ham himoyalaniishi kerak: server buzilganda, ma’lumotlar bazasi sizib chiqqanda yoki administrator ruxsatsiz kirganda ham ularni o‘qib bo‘lmasligi lozim. Buning uchun **maxfiy chatlar** — uchdan-uchgacha (end-to-end, E2E) shifrlangan suhbatlar loyihalandi.

Tahdid modeli va model tanlovi.

Protokolda server “halol, lekin qiziquvchan” ishtirokchi deb qaraladi: u xabarlarni to‘g‘ri yetkazadi, lekin o‘zidan o‘tgan har qanday ma’lumotni o‘qishga urinishi mumkin. Maqsad — server faqat quyidagilarni bilishi: kim kimga, qachon va qancha hajmdagi xabar yubordi. Xabarlarning mazmuni, fayllar, ularning nomi va turi, taymer sozlamalari esa serverga ko‘rinmasligi kerak.

Ikki model ko‘rib chiqildi. **Qurilmaga bog‘langan model** (Telegram maxfiy chatlari) — chat ikki aniq qurilma o‘rtasida ochiladi va boshqa qurilmalarda ko‘rinmaydi. **Ko‘p qurilmali model** (Signal, WhatsApp) — har bir xabar foydalanuvchining barcha qurilmalari uchun alohida shifrlanadi. Ikkinchi model ancha murakkab: har bir qurilmaning kalitlarini boshqarish va yangi qurilmani xavfsiz qo‘shish mexanizmi kerak bo‘ladi. Shu sababli birinchi versiya uchun qurilmaga bog‘langan model tanlandi: oddiy chatlar bulutda qoladi, maxfiy chatlar esa faqat qurilmada yashaydi.

Kriptografik primitivlar.

Kriptografiyani noldan yozish xavfli, shuning uchun protokol faqat brauzerga o‘rnatilgan, standartlashtirilgan **Web Crypto API** primitivlaridan quriladi. Uchinchi tomon kutubxonalari ishlatilmaydi. Barcha maxfiy kalitlar “extractable:

false” rejimida yaratiladi: sahifadagi skript ular yordamida shifrlashi mumkin, lekin kalitning baytlarini o‘qiy olmaydi. Qo‘llanilgan primitivlar 2.6-jadvalda keltirilgan.

2.6-jadval

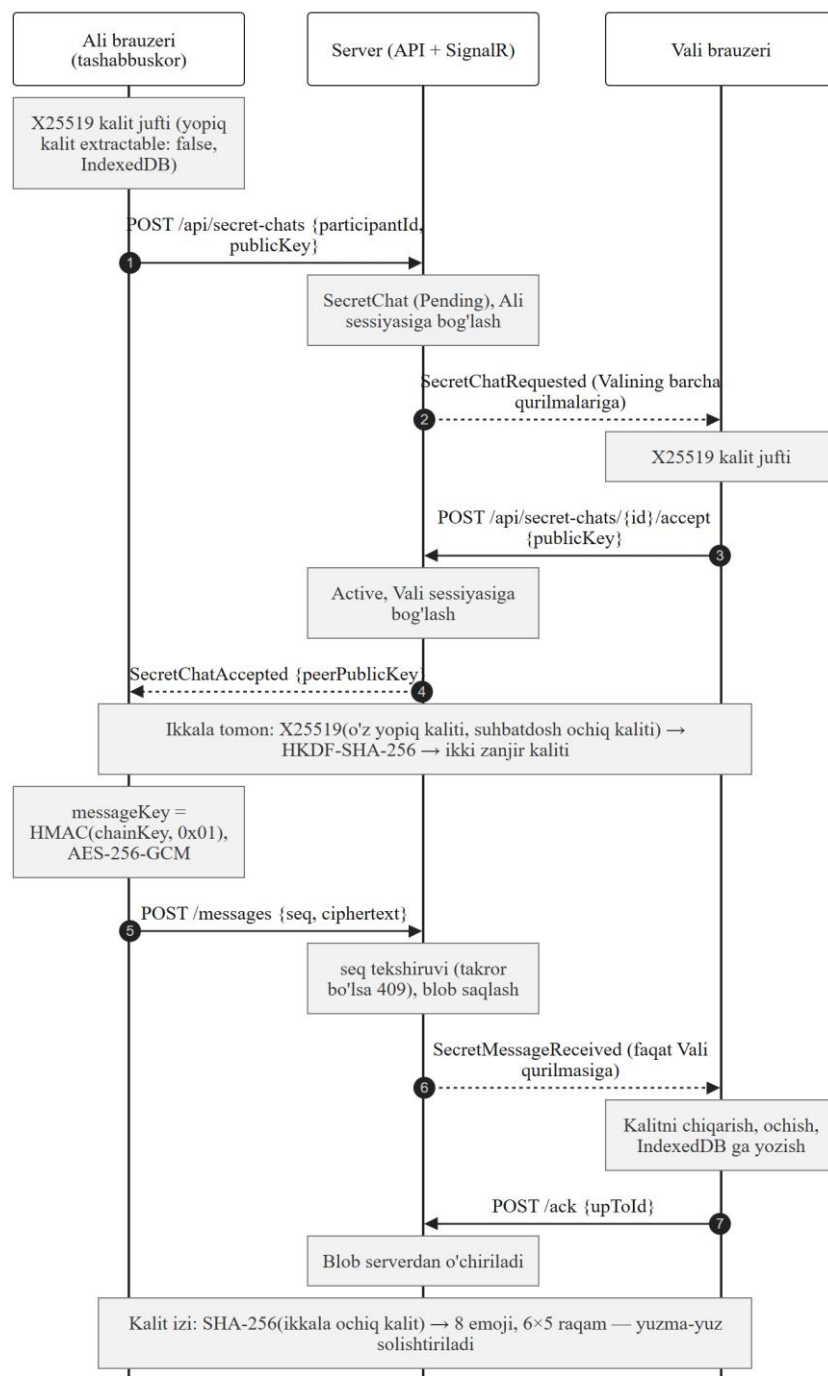
Maxfiy chatlarda qo‘llanilgan kriptografik primitivlar

Primitiv	Standart	Vazifasi
X25519	RFC 7748	Diffi-Xellman kalit kelishuvi: har bir qurilma har bir chat uchun yangi kalit jufti yaratadi, umumiy sir hech qachon tarmoq orqali uzatilmaydi
HKDF-SHA-256	RFC 5869	Umumiy sirdan ikki yo‘nalish uchun ikkita zanjir kalitini chiqarish
HMAC-SHA-256	RFC 2104	Zanjir kalitidan har bir xabar kaliti va keyingi zanjir kalitini hosil qilish (ratchet)
AES-256-GCM	NIST SP 800-38D	Xabar va fayllarni shifrlash va butunligini tekshirish; AAD maydoni xabarni chatga, yo‘nalishga va tartib raqamiga bog‘laydi
SHA-256	FIPS 180-4	Kalit izi (fingerprint): ikkala ochiq kalitdan emoji va raqamlar ko‘rinishida

Kalit almashish (handshake).

Maxfiy chat ochish jarayoni 2.13-rasmda ko‘rsatilgan. Tashabbuskorning brauzeri shu chat uchun X25519 kalit juftini yaratadi va serverga faqat 32 baytli ochiq kalitni yuboradi. Server Pending holatidagi chatni yaratadi, uni tashabbuskorning joriy sessiyasiga bog‘laydi va suhbatdoshning barcha onlayn qurilmalariga SecretChatRequested hodisasini yuboradi. Suhbatdosh qaysi qurilmada taklifni qabul qilsa, chat o‘sha qurilmaga bog‘lanadi: u ham o‘z kalit juftini yaratadi va ochiq kalitini qaytaradi. Server chatni Active holatiga o‘tkazadi va tashabbuskorga SecretChatAccepted hodisasi orqali suhbatdosh ochiq kalitini yetkazadi.

Shundan so‘ng ikkala tomon mustaqil ravishda bir xil umumiy sirni hisoblaydi: X25519(o‘z yopiq kaliti, suhbatdosh ochiq kaliti). Serverda faqat ikkita ochiq kalit bor, ulardan umumiy sirni hisoblash esa diskret logarifm masalasini yechishni talab qiladi, ya’ni amalda imkonsiz.



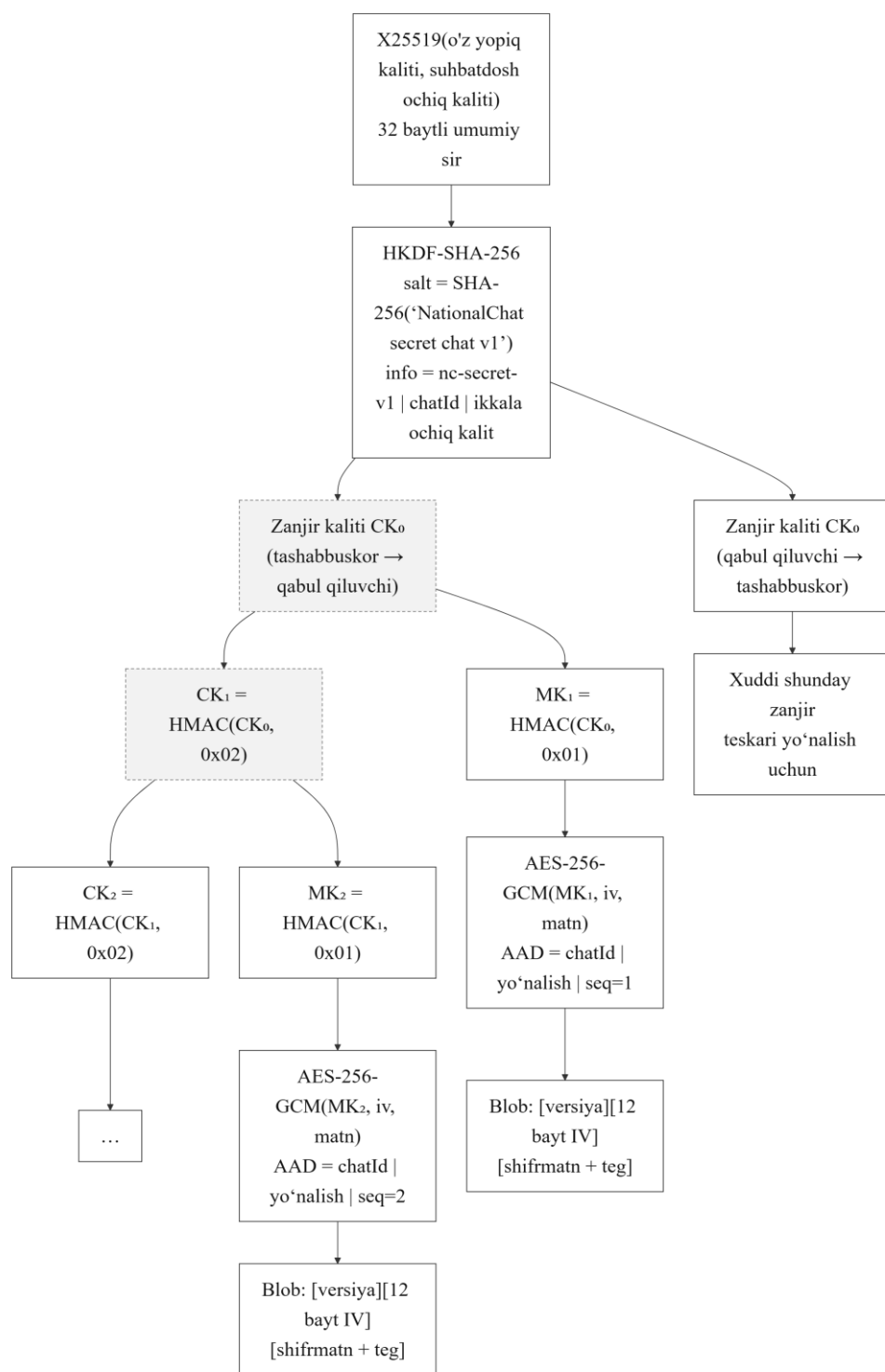
2.13-rasm. Maxfiy chatni ochish va shifrlangan xabar yetkazish ketma-ketlik diagrammasi

Xabar kalitlari zanjiri (ratchet).

Umumiy sir HKDF funksiyasi yordamida ikkita zanjir kalitiga kengaytiriladi: biri tashabbuskordan qabul qiluvchiga, ikkinchisi teskari yoʻnalishdagi xabarlar uchun (2.14-rasm). HKDF'ning info parametriga protokol versiyasi, chat identifikatori va ikkala ochiq kalit kiritiladi. Shu sababli har bir chatning kalitlari boshqa chatlarnikidan kriptografik jihatdan ajratilgan.

Har bir xabar uchun zanjir bir qadam oldinga suriladi: xabar kaliti $MK = \text{HMAC}(CK, 0x01)$, keyingi zanjir kaliti $CK' = \text{HMAC}(CK, 0x02)$. Eski zanjir kaliti darhol unutiladi, oraliq baytlar esa xotirada nollanadi. Bu usul **oldinga maxfiylikni** (forward secrecy) ta'minlaydi: agar qurilma keyinroq buzilib, joriy zanjir kaliti o'g'irlansa ham, HMAC bir tomonlama funksiya bo'lgani uchun undan avvalgi xabarlarning kalitlarini tiklab bo'lmaydi.

Xabar AES-256-GCM bilan tasodifiy 12 baytli IV yordamida shifrlanadi. Qo'shimcha autentifikatsiyalangan ma'lumot (AAD) sifatida "chatId | yo'nalish | tartib raqami" qatori ishlatiladi. U shifrlanmaydi, lekin GCM tegi unga bog'lanadi. Natijada serverdagi xabarni boshqa chatga ko'chirish, uni yuboruvchining o'ziga qaytarish yoki xabarlar o'rnini almashtirish mumkin bo'lmaydi: har qanday bunday urinishda teg tekshiruvdan o'tmaydi va xabar rad etiladi. Tarmoq orqali uzatiladigan blob quyidagi tuzilishga ega: versiya bayti, IV, shifrlangan matn va 16 baytli teg. Ochiq matn ichida esa xabar turi, matn, javob berilgan xabar raqami, yuborilgan vaqt, taymer va fayl ma'lumotlari JSON ko'rinishida saqlanadi.



2.14-rasm. Maxfiy chatda kalitlarni chiqarish va xabar kalitlari zanjiri

Yetkazish, tartib va taymerlar.

Server maxfiy xabarlar uchun faqat **saqlab-uzatish** (store-and-forward) vazifasini bajaradi. Shifrlangan blob qabul qiluvchi qurilmaga SignalR orqali (faqat shu qurilmaning “session: {sid}” guruhiga) yuboriladi. Qurilma xabarni ochib, o‘z xotirasiga (IndexedDB) yozgach, serverga tasdiq (ack) yuboradi va server blobni

o‘chiradi. Yetkazilmagan bloblar 7 kundan keyin fon ishi tomonidan tozalanadi. Shunday qilib, serverda maxfiy yozishmalar tarixi umuman saqlanmaydi.

Har bir tomon o‘z xabarlariga 1 dan boshlab o‘svuchi tartib raqami (seq) beradi. Server kichik yoki takroriy raqamli xabarni 409 Conflict javobi bilan rad etadi: bu qayta yuborish (replay) hujumidan himoyalaydi. Javobi yo‘qolgan so‘rovni qayta yuborgan klient esa shu javobdan xabar allaqachon serverda ekanini biladi. Xabarlar turli sabablarga ko‘ra tartibsiz kelishi mumkin. Shuning uchun qabul qiluvchi 200 tagacha o‘tkazib yuborilgan xabarning kalitini vaqtincha saqlaydi va har birini faqat bir marta ishlatadi. Zanjir holati faqat xabar muvaffaqiyatli ochilgandan keyin yangilanadi, shuning uchun soxta xabar zanjirni buza olmaydi.

Maxfiy chatda **o‘z-o‘zini o‘chirish taymeri** (10 soniyadan 1 haftagacha) va **tarixni tozalash** buyrug‘i bor. Ular ham oddiy xabar kabi shifrlanadi, shuning uchun server ularni oddiy xabardan ajrata olmaydi. Qabul qiluvchida xabarning muddati u ekranda birinchi marta ko‘ringan paytdan boshlab hisoblanadi, muddati o‘tgan xabar esa IndexedDB‘dan butunlay o‘chiriladi. Foydalanuvchi logout qilsa yoki sessiya yakunlansa, shu qurilmadagi barcha maxfiy chatlar serverda avtomatik yopiladi, qurilmadagi kalitlar va tarix esa o‘chiriladi.

Shifrlangan fayllar va kalit izi.

Fayl brauzerda har bir fayl uchun alohida tasodifiy AES-256-GCM kalit bilan shifrlanadi. Keyin serverga faqat shifrlangan baytlar yuklanadi (20 MB gacha). Fayl kaliti, IV, nomi, turi va hajmi esa E2E shifrlangan xabar ichida suhbatdoshga yuboriladi. Server fayl mazmunini ko‘rmagani uchun unga ClamAV tekshiruvi va rasmni qayta kodlash qo‘llanmaydi. Buning o‘rniga klient xavfsizlik qoidasiga amal qiladi: faqat PNG, JPEG, WebP va GIF rasmlar sahifada ko‘rsatiladi, boshqa har qanday fayl (HTML, SVG va boshqalar) faqat yuklab olinadi. Chunki sahifada ochilgan begona fayl ilova nomidan skript bajarishi mumkin edi.

Kalit izi (fingerprint) ikkala ochiq kalitning SHA-256 xeshidan hisoblanadi va 8 ta emoji hamda 6 guruh besh xonali raqam ko‘rinishida ko‘rsatiladi. Foydalanuvchilar uni yuzma-yuz yoki telefon orqali solishtiradi. Agar server o‘rtada

turib o'z kalitlarini almashtirgan bo'lsa (MITM hujumi), ikki tomondagi belgilar farq qiladi.

Protokol cheklovlari.

- protokol Signal'dagi to'liq Double Ratchet emas: oldinga maxfiylik bor, lekin buzilgandan keyin tiklanish (post-compromise security) yo'q, chunki suhbat davomida yangi Diffi-Hellman almashinuvi bajarilmaydi;
- metama'lumotlar (kim, kimga, qachon, qancha hajmda yozgani) serverga ko'rinadi;
- veb-ilovada shifrlash kodini ham server yuboradi: server buzilsa, u zararli skript yuborishi mumkin. Bu xavf qat'iy CSP va kriptografik kodni alohida modulga ajratish bilan kamaytirilgan, to'liq bartaraf etish esa alohida mobil yoki desktop ilova yaratishni talab qiladi;
- maxfiy chat faqat bitta qurilmada ishlaydi va boshqa qurilmaga ko'chirilmaydi. Vebda skrinshot olishni taqiqlash yoki aniqlash imkoni yo'q.

2-bob bo'yicha xulosa

Ikkinchi bobda "Milliy chat" tizimi loyihalandi. Server qismi uchun .NET 8, ASP.NET Core, SignalR, Entity Framework Core va PostgreSQL, klient qismi uchun Angular 22 texnologiyalari tanlandi va asoslandi. Server kodi Domain, Application, Infrastructure va API qatlamlaridan iborat Clean Architecture asosida tashkil etildi. Bu biznes-mantiqni texnik tafsilotlardan ajratish va tizimni kengaytirishni osonlashtirish imkonini berdi.

Ma'lumotlar bazasi o'nta sxemaga ajratilgan 36 ta jadvaldan iborat qilib loyihalandi. Shaxsiy va guruh chatlari yagona chats modeli orqali ifodalandi, katta hajmli ma'lumotlar uchun kursorli sahifalash tanlandi. Use-case va ketma-ketlik diagrammalari yordamida tizimga kirish, real vaqtda xabar almashish, guruh yaratish, tashkilotga qo'shilish va tokenlarni yangilash jarayonlari modellashtirildi. Xavfsizlik tizimi parolsiz autentifikatsiya, xeshlangan kod va tokenlar, qisqa

muddatli JWT va rotatsiya qilinadigan refresh token, a'zolik va rolga asoslangan avtorizatsiya, fayllarni ko'p bosqichli tekshirish hamda qat'iy CSP asosida qurildi.

Tizimning o'ziga xos imkoniyatlari ham loyihalandi. Tashkilot rejimi e-pochta domenini tashkilot sifatida qabul qiladi va shu domen egalari uchun yopiq guruhni avtomatik shakllantiradi. Maxfiy chatlar uchun Web Crypto API'dagi standart primitivlar (X25519, HKDF, HMAC, AES-256-GCM) asosida oldinga maxfiylikka ega uchdan-uchgacha shifrlash protokoli ishlab chiqildi, uning xavfsizlik kafolatlari va cheklovlari aniqlandi. Keyingi bobda ushbu loyihaning dasturiy amalga oshirilishi ko'rib chiqiladi.

III-BOB. “MILLIY CHAT” DASTURINI ISHLAB CHIQISH

Ushbu bobda ikkinchi bobda loyihalangan tizimning dasturiy amalga oshirilishi yoritiladi. Server qismi 236 ta C# faylidan (migratsiyalarsiz 9 mingga yaqin qator) iborat, unit testlar esa yana 2,1 ming qatorni tashkil etadi. Klient qismi 30 ta Angular komponenti va 13 mingdan ortiq TypeScript, HTML va SCSS qatoridan tashkil topgan. Ishlab chiqish jarayonida server repozitoriysida 89 ta, klient repozitoriysida 44 ta commit qilindi. Quyida keltirilgan kod parchalari loyiha haqiqiy manba kodidan olingan. Ayrim joylarda ular qisqartirilgan va bunday joylar “...” belgisi bilan ko‘rsatilgan.

3.1. Server qismini ishlab chiqish

Loyiha tuzilmasi va ilovani ishga tushirish.

Server yechimi (NationalChat.sln) ikkinchi bobda tavsiflangan to‘rtta loyihadan (Domain, Application, Infrastructure, API) va unit testlar loyihasidan (tests/NationalChat.Tests) iborat. Ilovaning kirish nuqtasi — API loyihasidagi Program.cs fayli (3.1-listing). U faqat ilovani yig‘ish va so‘rovlarni qayta ishlash konveyerini (middleware pipeline) sozlash bilan shug‘ullanadi. Barcha servislarni ro‘yxatdan o‘tkazish esa ServiceCollectionExtensions sinfidagi mavzuli metodlarga ajratilgan: API transporti, autentifikatsiya, ma’lumotlar bazasi, repozitoriylar, Application servislari, xavfsizlik, elektron pochta va fon ishlari.

3.1-listing. Program.cs — so‘rovlarni qayta ishlash konveyeri (qisqartirilgan)

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddServices(builder.Configuration, builder.Environment);

var app = builder.Build();
...
app.UseMiddleware<ExceptionHandlerMiddleware>();
app.UseMiddleware<SecurityHeadersMiddleware>();

app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseCors("Client");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.MapHub<ChatHub>("/hubs/chat");

app.Run();
```

Konveyerdagi tartib muhim. Xatolarni ushlovchi middleware eng birinchi turadi, shuning uchun u keyingi bosqichlarda yuz bergan har qanday istisnoni ushlay oladi. Undan keyin har bir javobga xavfsizlik sarlavhalarini qo‘shuvchi SecurityHeadersMiddleware keladi. Autentifikatsiya avtorizatsiyadan oldin bajariladi. REST controllerlar va SignalR habi eng oxirida ulanadi.

REST API.

Server 11 ta controller orqali jami 73 ta HTTP endpointni taqdim etadi (3.1-jadval). Endpointlar resurslarga yo‘naltirilgan (RESTful) tamoyil asosida nomlangan: resurslar ko‘plik shaklida (chats, messages, groups), amal esa HTTP metodi (GET, POST, PUT, DELETE) orqali ifodalanadi. Masalan, “POST /api/chats/{chatId}/messages” chatga xabar qo‘shadi, “PUT /api/groups/{chatId}/members/{userId}/role” esa guruh a’zosining rolini o‘zgartiradi.

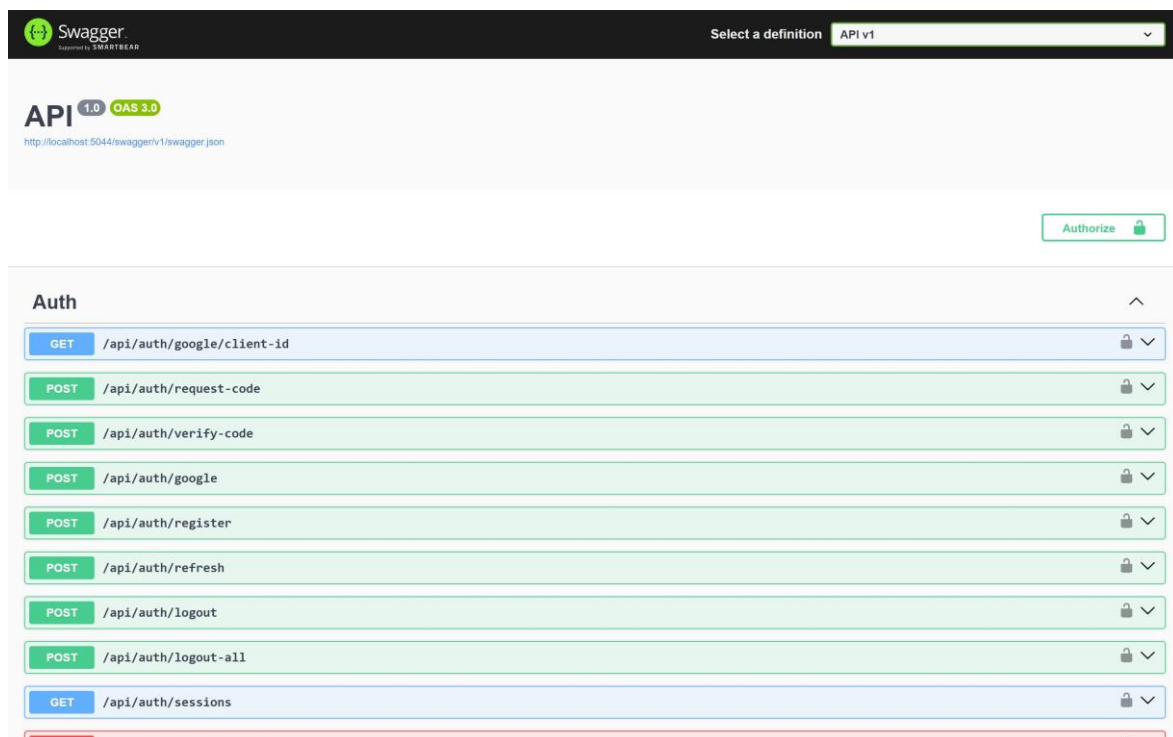
3.1-jadval

Server API controllerlari va endpointlari

Controller	Asosiy yo‘l	Soni	Vazifasi
AuthController	/api/auth	11	Kod so‘rash va tasdiqlash, Google, ro‘yxatdan o‘tish, tokenni yangilash, chiqish, sessiyalar

Controller	Asosiy yo‘l	Soni	Vazifasi
ProfileController	/api/users/me	6	Profilni olish va tahrirlash, profil rasmi, yozuv sozlamasi
UserController	/api/users	1	Foydalanuvchilarni qidirish
ContactController	/api/contacts	3	Kontaktlar ro‘yxati, qo‘shish, o‘chirish
ChatController	/api/chats	3	Chatlar ro‘yxati, shaxsiy chat, chatni o‘chirish
GroupController	/api/groups	12	Guruh yaratish, tahrirlash, rasm, a‘zolar va rollar, chiqish, taklif havolalari
MessageController	/api/chats/{id}/messages	7	Xabarlar tarixi, yuborish, tahrirlash, o‘chirish, qidiruv, kontekst
MessageAttachmentController	/api/chats/..., /api/media/...	10	Rasm, video, fayl yuklash va berish, ulashilgan media
StoryController	/api/stories	8	Hikoyalar lentasi, yaratish, media, ko‘rishlar
OrganizationController	/api/organizations	1	Foydalanuvchining tashkiloti va roli
SecretChatController	/api/secret-chats	11	Maxfiy chat so‘rovi, qabul qilish, yopish, shifrlangan xabar va fayllar, tasdiqlash (ack)
Jami		73	

API avtomatik ravishda OpenAPI (Swagger) spetsifikatsiyasi bilan hujjatlashtiriladi. Swagger UI interfeysida barcha endpointlar, ularning parametrlari va javob sxemalarini ko‘rish hamda JWT token bilan to‘g‘ridan-to‘g‘ri sinab ko‘rish mumkin (3.1-rasm). Bu imkoniyat klient va server qismlarini parallel ishlab chiqishda qo‘l keldi.



3.1-rasm. Swagger UI: server API hujjatlari

Javob formati, validatsiya va xatolar.

Barcha endpointlar yagona formatdagi javob qaytaradi: succeeded (muvaffaqiyat belgisi), message (xabar) va data (natija). Buning uchun API qatlamidagi Result va Result<T> sinflaridan foydalaniladi. Mijoz tomonida javoblarni qayta ishlash bitta ApiResponse<T> interfeysi orqali amalga oshiriladi. Xato holatlarida foydalanuvchiga ko'rsatiladigan xabarlar o'zbek tilida va ayblovchi bo'lmagan uslubda yozilgan, masalan: "Bu amal uchun ruxsat yo'q.", "Rasm hajmi 10 MB dan oshmasligi kerak."

Kiruvchi ma'lumotlar FluentValidation kutubxonasi yordamida tekshiriladi. Application qatlamida har bir so'rov DTOsi uchun alohida validator sinfi yozilgan (masalan, CreateGroupRequestValidator, SendMessageRequestValidator, SendSecretMessageRequestValidator). Validator ikki joyda ishlaydi: ASP.NET Core'ning avtomatik validatsiyasi so'rov controllerga yetib kelishidan oldin, servis esa biznes-qoidalarini qo'llashdan oldin tekshiradi. Kutilmagan istisnalar ExceptionHandlingMiddleware tomonidan ushlanadi. U xatoni so'rov metodi, yo'li va TraceId bilan logga yozadi va mijozga 500 status kodli, ichki tafsilotlarsiz standart javob qaytaradi.

Kursorli sahifalashni amalga oshirish.

Kursorli sahifalash barcha repozitoriylar uchun umumiy bo‘lgan IQueryable kengaytma metodi sifatida yozilgan (3.2-listing). Metod identifikatorni tanlovchi ifodani (Expression) qabul qiladi, “Id < beforeId” shartini ifodalar daraxti orqali dinamik quradi va limitdan bittaga ko‘p yozuv o‘qiydi. Qo‘shimcha yozuv mavjudligi hasMore qiymatini aniqlaydi, alohida COUNT so‘roviga esa hojat qolmaydi.

3.2-listing. Kursorli sahifalash (QueryableExtensions)

```
public static async Task<CursorPagedResponse<TResult>>
ToCursorPagedResponseAsync<TEntity, TResult>(
    this IQueryable<TEntity> query, CursorPaginationRequest pagination,
    Expression<Func<TEntity, int>> idSelector, Expression<Func<TEntity, TResult>>
resultSelector,
    Func<TResult, int> cursorSelector, CancellationToken cancellationToken = default)
{
    if (pagination.BeforeId is not null)
    {
        var beforeId = Expression.Lambda<Func<TEntity, bool>>(
            Expression.LessThan(idSelector.Body,
Expression.Constant(pagination.BeforeId.Value)),
            idSelector.Parameters);
        query = query.Where(beforeId);
    }

    var results = await query.OrderByDescending(idSelector)
        .Take(pagination.Limit + 1)
        .Select(resultSelector)
        .ToListAsync(cancellationToken);

    var hasMore = results.Count > pagination.Limit;
    IReadOnlyList<TResult> items = hasMore ? results.Take(pagination.Limit).ToArray()
: results;
    int? nextCursor = hasMore ? cursorSelector(items[^1]) : null;
    return new(items, nextCursor, hasMore);
}
```

Natija to‘g‘ridan-to‘g‘ri DTOga proyeksiya qilinadi (Select). Shu sababli EF Core ma’lumotlar bazasidan faqat kerakli ustunlarni o‘qiydi va entity obyektlarini kuzatishga resurs sarflamaydi. Proyeksiya ifodalari Application qatlamidagi Mapper sinflarida (ChatListMapper.Projection, MessageMapper.Projection) saqlanadi. Masalan, chatlar ro‘yxati so‘rovi bitta SQL so‘rovida har bir chat uchun oxirgi xabarni, suhbatdosh ma’lumotlarini, guruh nomi va a‘zolar sonini hamda o‘qilmagan xabarlar sonini hisoblaydi.

3.2. Real vaqtda xabar almashishni amalga oshirish

Real vaqt aloqasi ChatHub SignalR habi hamda IChatRealtimeNotifier va ISecretChatRealtimeNotifier interfeyslarining SignalR orqali ishlaydigan amalga oshirilishlari yordamida quriladi. Hab mijoz chaqira oladigan to'rtta metodni taqdim etadi. Server esa mijozlarga oddiy va maxfiy chatlar bo'yicha jami o'n to'rt turdagi hodisa yuboradi (3.2-jadval). Har bir ulanish ulanish paytida ikkita guruhga qo'shiladi: foydalanuvchining barcha qurilmalari uchun umumiy "user:{id}" va aynan shu qurilma (sessiya) uchun "session:{sid}". Ikkinchi guruh maxfiy chat hodisalarini faqat chat bog'langan qurilmaga yetkazish uchun ishlatiladi.

3.2-jadval

SignalR habi metodlari va hodisalari

Nomi	Yo'nalish	Vazifasi
JoinChat(chatId)	Mijoz → server	Ochilgan chatning "chat:{id}" guruhiga qo'shilish (a'zolik tekshiriladi)
LeaveChat(chatId)	Mijoz → server	Chat guruhidan chiqish
SetTyping(chatId, isTyping)	Mijoz → server	"Yozmoqda" holatini chatdagi boshqalarga uzatish
MarkMessagesRead(chatId, ids)	Mijoz → server	Xabarlarini o'qilgan deb belgilash (100 tagacha)
MessageReceived	Server → mijoz	Yangi xabar yoki xizmat xabari
MessageUpdated / MessageDeleted	Server → mijoz	Xabar tahrirlandi yoki o'chirildi
MessagesRead	Server → mijoz	Jo'natuvchiga xabarlar o'qilgani haqida (✓✓)
TypingChanged	Server → mijoz	Suhbatdosh yozmoqda yoki to'xtadi
ChatCleared / ChatDeleted	Server → mijoz	Chat tarixi tozalandi yoki chat o'chirildi (a'zolikdan chiqarilganda ham)
GroupUpdated	Server → mijoz	Guruh nomi, rasmi, a'zolari yoki rollari o'zgardi
UserPresenceChanged	Server → mijoz	Foydalanuvchi onlayn bo'ldi yoki oflayn holatga o'tdi
ProfilePhotoUpdated	Server → mijoz	Foydalanuvchi profil rasmini yangiladi
SecretChatRequested / SecretChatAccepted	Server → mijoz	Maxfiy chat taklifi keldi yoki qabul qilindi (suhbatdosh ochiq kaliti bilan)
SecretMessageReceived / SecretChatClosed	Server → mijoz	Shifrlangan xabar (faqat bog'langan qurilmaga) yoki maxfiy chat yopildi

Hab metodlari ham REST endpointlari kabi himoyalangan: har bir chaqiruvda foydalanuvchining shu chat a'zosi ekanligi tekshiriladi (3.3-listing). Bu tekshiruv bo'lsa, xabarni boshqa foydalanuvchi chat identifikatorini taxmin qilib, begona chatdagi "yozmoqda" hodisalarini tinglashi yoki xabarlarini o'qilgan deb belgilashi mumkin bo'lardi.

3.3-listing. ChatHub: o'qilganlik belgisini qayd etish va a'zolikni tekshirish

```
public async Task MarkMessagesRead(int chatId, IReadOnlyCollection<int> messageIds)
{
    await EnsureChatMembershipAsync(chatId);
    var ids = messageIds.Where(id => id > 0).Distinct().Take(100).ToArray();
    if (ids.Length == 0) return;

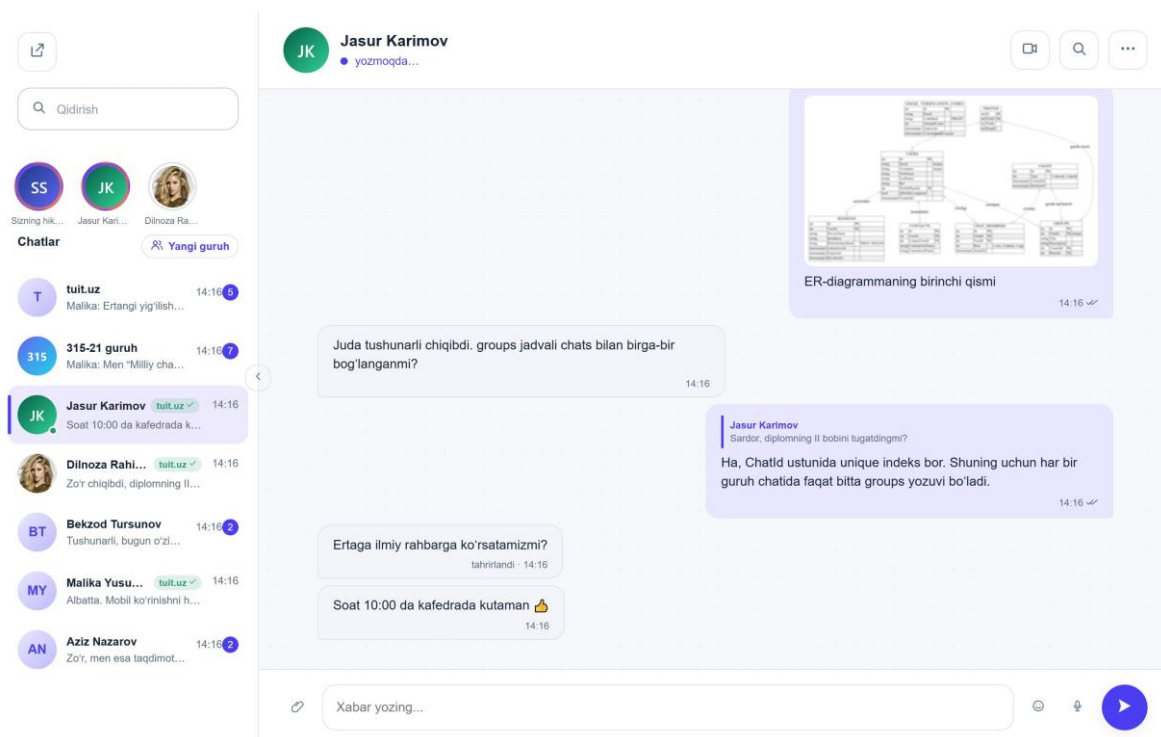
    var readerUserId = GetCurrentUser();
    await messageRepository.MarkAsReadAsync(chatId, readerUserId, ids,
        timeProvider.GetUtcNow().UtcDateTime, Context.ConnectionAborted);
    await realtimeNotifier.MessagesReadAsync(chatId, readerUserId, ids,
        await messageRepository.GetMemberUserIdsAsync(chatId,
Context.ConnectionAborted),
Context.ConnectionAborted);
}

private async Task EnsureChatMembershipAsync(int chatId)
{
    if (!await messageRepository.IsChatMemberAsync(chatId, GetCurrentUser(),
Context.ConnectionAborted))
        throw new HubException("Bu chatga kirish huquqi yo'q.");
}
```

Onlayn holatni kuzatish IPresenceTracker interfeysi orqali abstraksiyalangan. InMemoryPresenceTracker har bir foydalanuvchi uchun faol ulanishlar to'plamini ConcurrentDictionary'da saqlaydi. RedisPresenceTracker esa xuddi shu ma'lumotni Redis to'plamlarida (SET) saqlaydi va bir nechta server nusxasi ishlaganda qo'llanadi. Foydalanuvchining birinchi ulanishi paydo bo'lganda u bilan umumiy chati bor foydalanuvchilarga "onlayn" hodisasi yuboriladi. Oxirgi ulanish uzilganda esa server 8 soniya kutadi (grace period). Sahifani yangilash yoki qisqa tarmoq uzilishida foydalanuvchi qayta ulanib ulguradi, natijada suhbatdoshlarda "oflayn" holati bir zumga ham ko'rinmaydi. Shundan so'ng sessiyaning oxirgi faollik vaqti yangilanadi va "oxirgi marta ... onlayn edi" ma'lumoti hisoblanadi.

Klient tomonida ChatRealtimeService servisi SignalR ulanishini boshqaradi. Ulanish avtomatik qayta ulanish siyosati (0, 2, 5 va 10 soniyadan keyin urinish) bilan yaratiladi. Qayta ulanilgach, ochiq turgan chatga JoinChat avtomatik qayta

chaqiriladi. Kiruvchi hodisalar RxJS Subject'lari orqali ChatApiService'ga uzatiladi va u ilova holatini (chatlar ro'yxati, xabarlar, "yozmoqda" holati) yangilaydi. 3.2-rasmda suhbatdosh javob yozayotgan paytdagi shaxsiy chat ko'rsatilgan: sarlavhada "yozmoqda..." yozuvi, jo'natilgan xabarlar yonida esa o'qilganlik belgisi ko'rinib turibdi. Chatlar ro'yxatida tasdiqlangan tashkilot a'zolarining ismi yonida "tuit.uz ✓" belgisi ham ko'rinadi.



3.2-rasm. Shaxsiy chat: real vaqtda "yozmoqda" holati va o'qilganlik belgilari

3.3. Media fayllar bilan ishlash

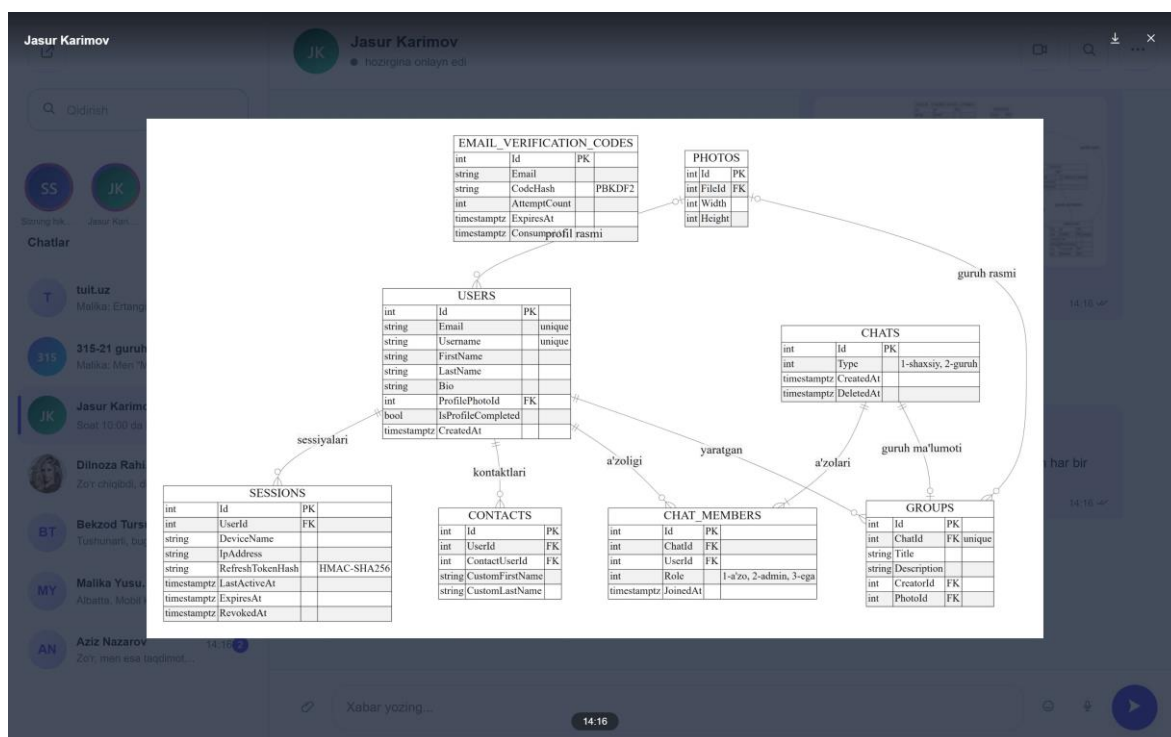
Rasm, video va ixtiyoriy fayllar multipart/form-data formatida yuklanadi. Barcha turdagi fayllar bir xil konveyerdan o'tadi. Umumiy qadamlar Application qatlamidagi FileService servisida, texnik amallar esa Infrastructure qatlamidagi LocalImageStorage sinfida amalga oshirilgan:

1. fayl tanlangani va hajmi tekshiriladi (rasm — 10 MB, fayl — 50 MB, xabardagi video — 200 MB gacha);
2. fayl ClamAV antivirusi bilan tekshiriladi (sozlamalarda yoqilgan bo'lsa);
3. rasm formati kengaytma bo'yicha emas, tarkib bo'yicha aniqlanadi (JPEG, PNG, WebP), rasm 90% sifat bilan qayta kodlanadi va 256 piksellik kichik

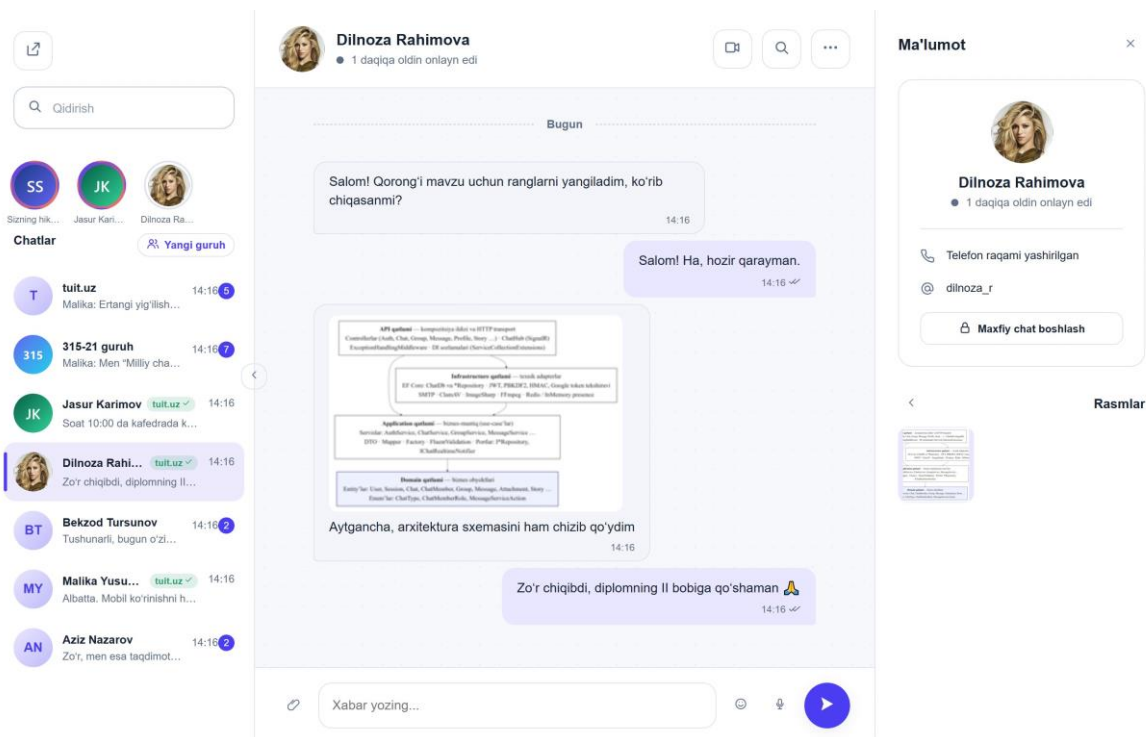
nusxa (thumbnail) yaratiladi. Video uchun esa ffmpeg yordamida o'lchamlar aniqlanadi va ffmpeg yordamida birinchi kadrda muqova rasm olinadi;

4. fayl diskka yil/oy bo'yicha ajratilgan papkaga GUID nom bilan yoziladi;
5. ma'lumotlar bazasiga files va photos yozuvlari, xabar va biriktirma (attachment) qo'shiladi;
6. chat a'zolariga MessageReceived hodisasi yuboriladi.

Trafikni tejash uchun chatda avval faqat rasmning o'lchamlari va hajmi ko'rsatiladi. Rasmning kichik nusxasi foydalanuvchi so'raganda yuklanadi, to'liq o'lchamdagi asl nusxa esa ko'rish oynasida ochiladi (3.3-rasm). Videolar HTTP Range so'rovlari orqali oqim (streaming) ko'rinishida beriladi, shuning uchun videoni to'liq yuklamasdan ham istalgan joyidan ko'rish mumkin. Chatdagi barcha media fayllar ma'lumot panelida turlari bo'yicha (rasmlar, videolar, fayllar) guruhlab ko'rsatiladi (3.4-rasm).



3.3-rasm. Rasmni to'liq o'lchamda ko'rish oynasi



3.4-rasm. Chatdagi ulashilgan media paneli

3.4. Guruh chatlari va hikoyalar

Guruh chatlari GroupService servisinde amalga oshirilgan. Servis har bir amaldan oldin foydalanuvchining guruhdagi rolini aniqlaydi va 2.4-jadvaldagi ruxsatlarni qoʻllaydi. Oʻzgarishlar esa xizmat xabarlarini va real vaqt hodisalari bilan birga saqlanadi. Eng murakkab holatlardan biri — guruh egasining guruhdan chiqishi (3.4-listing). Bu holatda guruh egasiz qolmasligi uchun egalik eng oldin tayinlangan administratorga, administratorlar boʻlmasa esa guruhga eng oldin qoʻshilgan aʼzoga oʻtadi. Guruhda hech kim qolmasa, u yumshoq oʻchiriladi.

3.4-listing. GroupService.LeaveAsync: guruhdan chiqish va egalikni oʻtkazish (qisqartirilgan)

```
var now = timeProvider.GetUtcNow().UtcDateTime;
RemoveMember(group, leaver);

if (group.Chat.Members.Count == 0)
{
    group.Chat.DeletedAt = now;
    await repository.SaveChangesAsync(cancellationToken);
    await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
    cancellationToken);
    return new(true, null);
}

if (leaver.Role == ChatMemberRole.Creator)
{

```

```

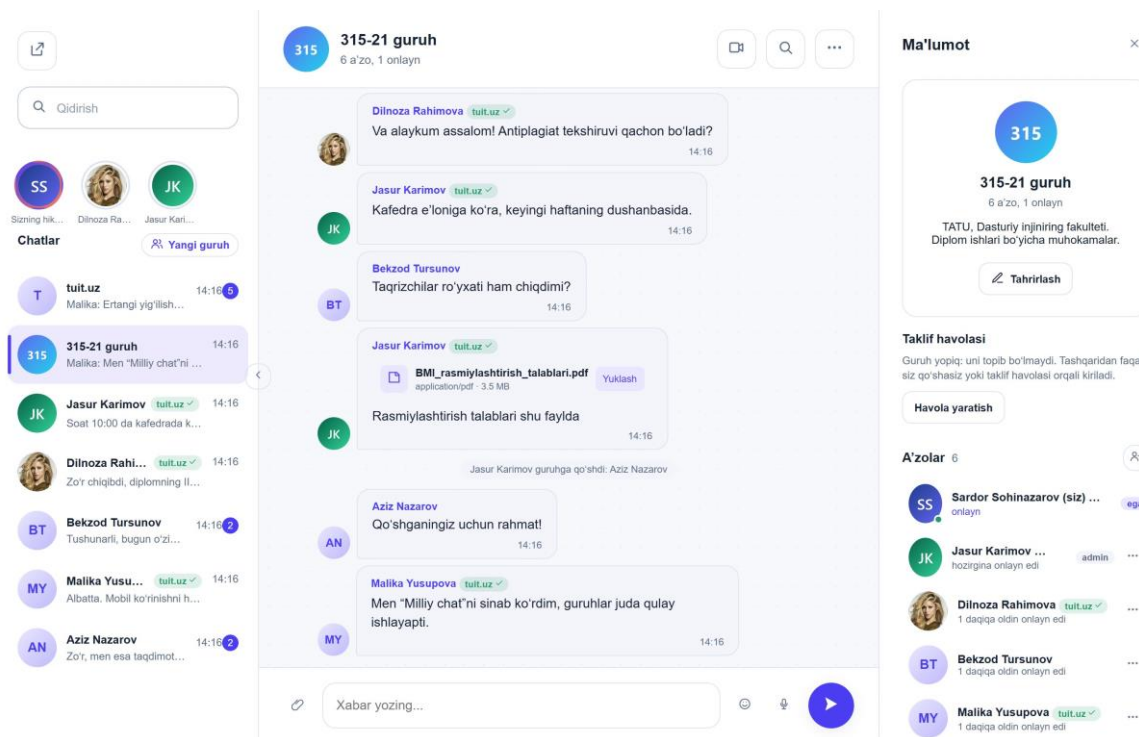
// Ownership passes to the longest-serving admin, or to the longest-serving
member.
var successor = group.Chat.Members
    .OrderByDescending(member => member.Role == ChatMemberRole.Admin)
    .ThenBy(member => member.JoinedAt)
    .ThenBy(member => member.Id)
    .First();
successor.Role = ChatMemberRole.Creator;
group.CreatorId = successor.UserId;
}

var serviceMessage = AddServiceMessage(group, leaver.User,
MessageServiceAction.MemberLeft, ...);
await repository.SaveChangesAsync(cancellationToken);

```

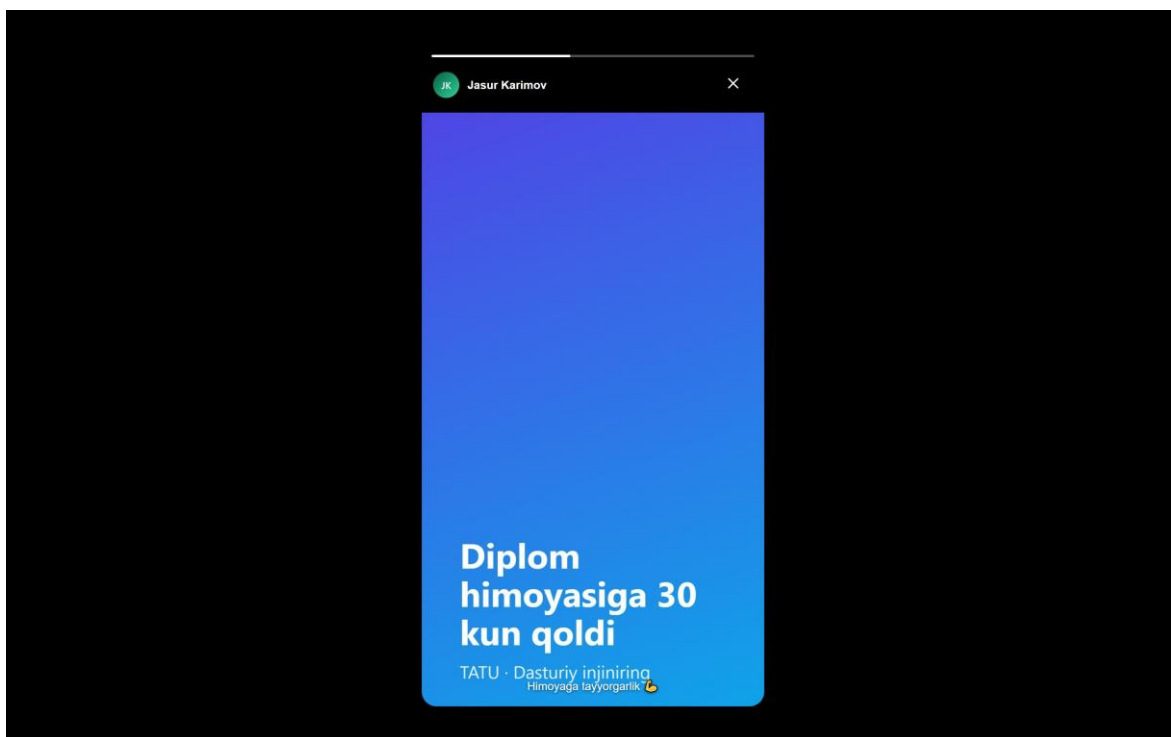
Xizmat xabarlarining matni klient tomonida xabar turi (ServiceAction) va unda saqlangan ism yoki nom asosida yig‘iladi. Masalan, “Jasur Karimov guruhga qo‘shdi: Aziz Nazarov”, “Sardor Sohinazarov guruh rasmini yangiladi”. Bunday xabarlar chat markazida kulrang yozuv ko‘rinishida chiqadi va ularni tahrirlab, o‘chirib yoki ularga javob berib bo‘lmaydi. Guruhda boshqa a‘zolarining xabarlari ustida jo‘natuvchining ismi, tashkilot belgisi va avatari ko‘rsatiladi. Sarlavhada a‘zolar va onlayn a‘zolar soni, kimdir yozayotgan bo‘lsa esa uning ismi chiqadi. Ma’lumot panelida guruh rasmi, tavsifi, taklif havolasi va a‘zolar ro‘yxati rollari bilan ko‘rsatiladi. Guruh egasi va administratorlarga esa a‘zolari boshqarish menyusi ochiladi (3.5-rasm).

Taklif havolasi ixtiyoriy guruh uchun yaratilishi mumkin. Admin “Havola yaratish” tugmasini bosganda server GroupInviteTokens yordamida 16 baytli kriptografik tasodifiy tokenni base64url ko‘rinishida (22 belgi) yaratadi va groups jadvaliga yozadi. Havolani ochgan foydalanuvchi guruh nomi, tavsifi va a‘zolar sonini ko‘radi, lekin a‘zolar ro‘yxatini ko‘rmaydi. Guruhga qo‘shilganda chatda xizmat xabari paydo bo‘ladi. Havola yangilanganda yoki bekor qilinganda eski token darhol ishlamay qoladi.



3.5-rasm. Guruh chati va guruh haqidagi ma'lumot paneli

Hikoyalar (stories) StoryService servisinde amalga oshirilgan. Hikoya rasm yoki video (MP4, WebM, MOV, 50 MB gacha) va qisqa izohdan iborat. U joylangan paytdan boshlab 24 soat davomida ko'rinadi ($\text{ExpiresAt} = \text{CreatedAt} + 24 \text{ soat}$). Hikoyalar lentasi foydalanuvchining o'zi va u bilan chati bor foydalanuvchilarning amaldagi hikoyalarini qaytaradi. Chatlar ro'yxati ustidagi hikoyalar panelida ko'rilmagan hikoyasi bor foydalanuvchilar rangli halqa bilan ajratiladi. Hikoya ochilganda `story_views` jadvaliga yozuv qo'shiladi. Hikoya muallifi esa uni kim va qachon ko'rganini bilishi mumkin. Hikoya ko'ruvchi oynasi 3.6-rasmda keltirilgan: yuqorida progress chiziqlari, muallif va vaqt, pastda esa izoh joylashgan.



3.6-rasm. Hikoyani ko‘rish oynasi

3.5. Lotin–kirill transliteratsiyasi va yozuvdan qat’i nazar qidiruv

O‘zbekistonda ikki yozuv parallel qo‘llanadi: rasmiy lotin alifbosi va hanuzgacha keng ishlatiladigan kirill alifbosi. Katta avlod vakillari va ko‘plab rasmiy matnlar kirillda, yoshlar esa asosan lotinda yozadi. Mavjud messengerlar xabarni qanday yozilgan bo‘lsa, shunday ko‘rsatadi: kirillni o‘qishga qiynaladigan foydalanuvchi uchun lotin yozuvi, yoki aksincha, noqulay bo‘ladi. “Milliy chat”da bu muammo hal etilgan: har bir foydalanuvchi xabarlarni o‘zi tanlagan yozuvda ko‘radi. Ushbu imkoniyat faqat o‘zbek tiliga xos va ko‘rib chiqilgan xorijiy messengerlarning birortasida mavjud emas.

Xabar ma’lumotlar bazasida doimo **asl holida** saqlanadi, o‘girish esa faqat ko‘rsatish paytida klientda bajariladi. Shu sababli bitta xabarni bir foydalanuvchi lotinda, boshqasi kirillda ko‘radi, jo‘natuvchi esa uni qanday yozgan bo‘lsa, shunday ko‘radi. Foydalanuvchi sozlamasi (ScriptPreference) serverda saqlanadi va uning barcha qurilmalarida amal qiladi. Yozuv almashtirilganda sahifani qayta yuklash talab etilmaydi: uzScript pipe’i Angular signaliga bog‘langan, shuning uchun barcha xabarlar darhol qayta chiziladi.

Transliteratsiya algoritmi.

Algoritm 1995-yilda tasdiqlangan o‘zbek lotin alifbosi va imlo qoidalariga asoslanadi. U Domain qatlamidagi UzbekTransliterator statik sinfida amalga oshirilgan, klientda esa uning TypeScript’dagi aynan nusxasi ishlaydi. Ikkala tomondagi algoritm bir-biridan farq qilib qolmasligi uchun ular bitta umumiy test to‘plami (transliteration-cases.json, 80 ta kirill–lotin juftligi) bo‘yicha tekshiriladi. Kirilldan lotinga o‘girish deyarli bir ma’noli, teskari yo‘nalishda esa kontekstga bog‘liq qoidalar talab etiladi (3.3-jadval).

3.3-jadval

Kontekstga bog‘liq transliteratsiya qoidalari

Holat	Qoida	Misol
Kirill “e”	So‘z boshida, unli harfdan yoki ъ, ь dan keyin “ye”, qolgan hollarda “e”	ер → yer; биринчи → birinchi; келди → keldi
Kirill “и”	Unli harfdan keyin “ts”, qolgan hollarda “s”	лицей → litsey; цирк → sirk
ŷ, ƒ va tutuq	ŷ → o‘, ƒ → g‘, ъ → ‘; “cx” tutuq bilan yoziladi, “ш” bilan adashmasligi uchun	Ўзбек → O‘zbek; Исҳоқ → Is’hoq
Lotin digraflari	sh → ш, ch → ч, o‘ → ŷ, g‘ → ƒ, yo → ё, yu → ю, ya → я, ye → е	shahar → шахар; yo‘l → йўл
Lotin “e”	So‘z boshida yoki unlidan keyin “ʻ”, qolgan hollarda “e”	eshik → эшик; кеча → кеча
Lotin “ts”	Istisnolar lug‘atidagi o‘zlashma so‘zlarda “и”, qolgan hollarda “tc”	litsey → лицей; eritsa → эритса
Registr	Bosh harf saqlanadi; butun so‘z katta harfda bo‘lsa, digraf ham katta	ШАҲАР → SHAHAR; Шаҳар → Shahar
Tutuq belgisi variantlari	‘, ’, ` , ‘, ’ belgilarining barchasi bir xil tushuniladi	o'zbek = o‘zbek = o’zbek
Himoyalangan qismlar	URL, e-pochta, @username, #teg va ‘kod’ o‘girilmaydi	@sardor_s, https://tuit.uz o‘zgarmaydi

Kirilldan lotinga o‘girish funksiyasining asosiy qismi 3.5-listingda keltirilgan. Har bir harf uchun avval kontekstga bog‘liq qoidalar tekshiriladi, keyin esa oddiy moslik lug‘atidan foydalaniladi. Registr alohida qayta ishlanadi: “III” so‘z boshida “Sh”, butunlay katta harflardan iborat so‘zda esa “SH” ko‘rinishiga keladi.

3.5-listing. UzbekTransliterator: kirill matnini lotinga o‘girish (C#)

```
private static string ToLatinSegment(string text)
{
```

```

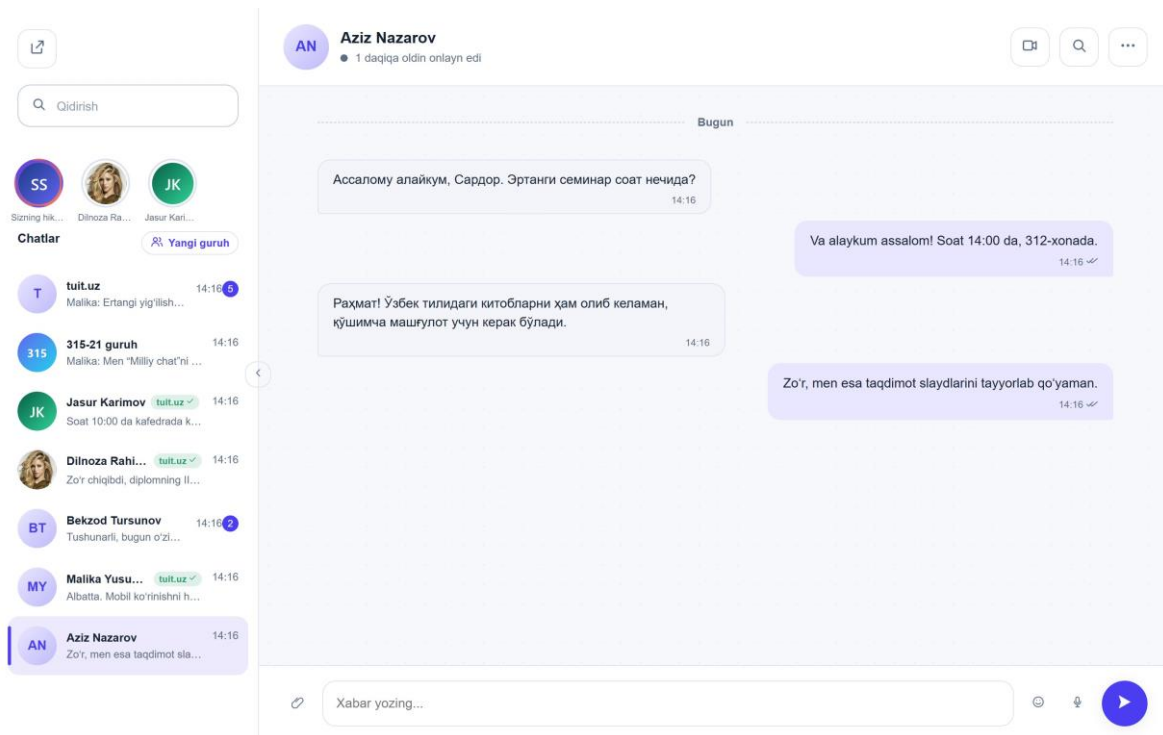
var builder = new StringBuilder(text.Length + 8);
for (var i = 0; i < text.Length; i++)
{
    var ch = text[i];
    var lower = char.ToLowerInvariant(ch);
    var previous = i > 0 ? char.ToLowerInvariant(text[i - 1]) : '\0';

    string latin;
    if (lower == 'e')
    {
        var soundsYe = !IsWordChar(text, i - 1) ||
            CyrillicVowels.Contains(previous) || previous is 'ѣ' or 'ѥ';
        latin = soundsYe ? "ye" : "e";
    }
    else if (lower == 'ц')
    {
        latin = CyrillicVowels.Contains(previous) ? "ts" : "s";
    }
    else if (lower == 'х' && previous == 'с')
    {
        // "cx" (Исхок) is written with a tutuq in Latin so that it is not read as
        "ш".
        latin = TutuqMark + "h";
    }
    else if (!CyrillicToLatinMap.TryGetValue(lower, out latin!))
    {
        builder.Append(ch);
        continue;
    }

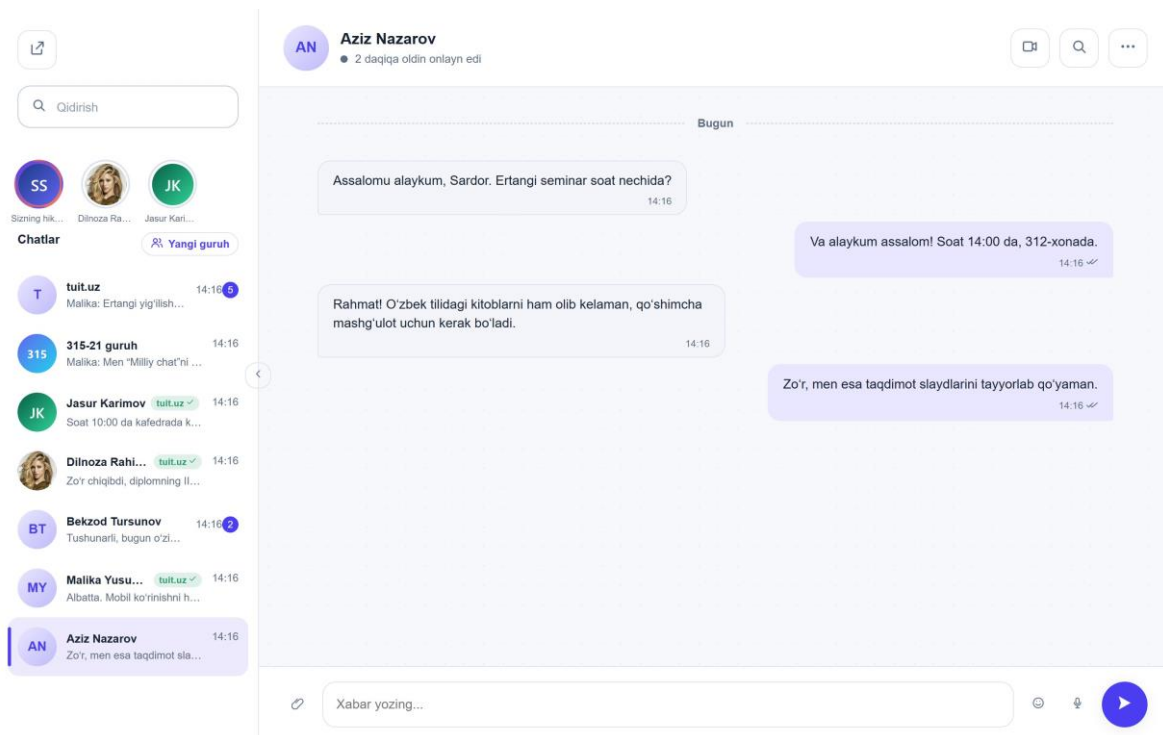
    builder.Append(ch == lower ? latin : ApplyUpperCase(latin,
        IsUpperCaseWord(text, i)));
}
return builder.ToString();
}

```

Sozlama “Asl holida” bo‘lsa, xabarlar yozilganidek ko‘rsatiladi (3.7-rasm). “Lotin” tanlanganda esa xuddi shu suhbatdagi kirillda yozilgan xabarlar lotin yozuvida chiqadi (3.8-rasm). Ma’lum cheklov sifatida shuni ta’kidlash kerakki, kirill sozlamasida inglizcha so‘zlar ham kirill harflariga o‘giriladi. Shuning uchun URL va username kabi qismlar himoyalangan, foydalanuvchi esa istalgan vaqtda asl holiga qaytishi mumkin.



3.7-rasm. Suhbat asl holida: suhbatdosh kirillda, foydalanuvchi lotinda yozgan



3.8-rasm. Xuddi shu suhbat “Lotin” sozlamasida

Yozuvdan qat'i nazar qidiruv.

Chat ichidagi qidiruv ham yozuvga bog‘liq bo‘lmasligi kerak: foydalanuvchi “rahmat” deb qidirganda “Раҳмат” ham topilishi lozim. Buning uchun har bir xabar yaratilganda va tahrirlanganda MessageFactory uning **qidiruv kalitini** hisoblaydi va

SearchText ustuniga yozadi. Kalit NormalizeForSearch funksiyasi yordamida olinadi: matn lotinga o'giriladi, kichik harflarga o'tkaziladi, tutuq belgisining barcha shakllari bitta "''" belgisiga keltiriladi va ortiqcha bo'shliqlar olib tashlanadi. Qidiruv so'rovi ham xuddi shu funksiyadan o'tkaziladi, keyin esa SearchText ustunida ILIKE bilan qidiriladi. Natijada "o'zbek", "o'zbek", "o'zbek" va "Ўзбек" so'rovlari bir xil natija beradi.

Yangi ustun qo'shilishidan oldin yozilgan xabarlar uchun SearchText bo'sh bo'ladi. Ularni to'ldirish uchun SearchTextBackfill fon ishi yozildi: server ishga tushganda u bunday xabarlarni 500 tadan qayta ishlaydi. Qayta ishga tushirilganda u faqat hali to'ldirilmagan yozuvlarni oladi, shuning uchun bir necha marta bajarilishi xavfsiz. Ma'lumotlar bazasidagi natija 3.9-rasmda ko'rsatilgan: TextContent ustunida xabar asl holida (kirill yoki lotin), SearchText ustunida esa uning yagona lotin ko'rinishi saqlanadi.

```
PS> psql -d NationalChat -c "SELECT Id, TextContent, SearchText FROM messaging.messages WHERE ChatId = 2;"
```

Id	TextContent	SearchText
5	Ассалому алайкум, Сардор. Эртанги семи	assalomu alaykum, sardor. ertangi semi
6	Va alaykum assalom! Soat 14:00 da, 312	va alaykum assalom! soat 14:00 da, 312
7	Раҳмат! Ўзбек тилидаги китобларни ҳам	rahmat! o'zbek tilidagi kitoblarni ham
8	Zo'r, men esa taqdimot slaydlarini tay	zo'r, men esa taqdimot slaydlarini tay

(4 rows)

3.9-rasm. messages jadvali: asl matn (TextContent) va qidiruv kaliti (SearchText)

3.6. Tashkilot rejimini amalga oshirish

Tashkilot rejimi Application qatlamidagi OrganizationEmailMatcher yordamchi sinfi va OrganizationMembershipService servisinde amalga oshirilgan. OrganizationEmailMatcher e-pochta manzilidan domenni ajratadi, uni normallashtiradi va umumiy pochta xizmatlari ro'yxati (PublicEmailDomains — gmail.com, mail.ru, yandex.ru, umail.uz, vaqtinchalik pochta xizmatlari va boshqalar) bilan solishtiradi. Bu sinf tashqi bog'liqlikka ega emas va 22 ta unit test bilan tekshirilgan: subdomen alohida tashkilot ekanligi, "evil-tuit.uz" kabi domenlar mos kelmasligi, registr va noto'g'ri formatdagi manzillar.

OrganizationMembershipService.EnsureMembershipAsync metodi foydalanuvchi tasdiqlangan pochta bilan kirgan har safar chaqiriladi (3.6-listing).

A'zolik yozuvi allaqachon bo'lsa, metod darhol qaytadi, chunki e-pochta manzili o'zgarmaydi. Aks holda tashkilot topiladi yoki yaratiladi. Bir vaqtda ikki foydalanuvchi bir xil yangi domen bilan kirgan holat ham hisobga olingan: unique indeks tufayli faqat bittasi tashkilotni yarata oladi va u admin bo'ladi, ikkinchisi esa oddiy a'zo sifatida qo'shiladi.

3.6-listing. OrganizationMembershipService: foydalanuvchini tashkilotga va domen guruhiga qo'shish

```
public async Task EnsureMembershipAsync(User user, CancellationToken cancellationToken
= default)
{
    // An e-mail never changes, so an existing membership is final.
    if (await repository.GetMembershipAsync(user.Id, cancellationToken) is not null)
        return;

    var domain = OrganizationEmailMatcher.OrganizationDomainOf(user.Email);
    if (domain is null) return;

    var now = timeProvider.GetUtcNow().UtcDateTime;
    var organization = await repository.FindByDomainAsync(domain, cancellationToken);
    var isFounder = false;
    if (organization is null)
    {
        var created = OrganizationFactory.Create(domain, now);
        isFounder = await repository.TryAddOrganizationAsync(created,
cancellationToken);
        organization = isFounder ? created : await
repository.FindByDomainAsync(domain, cancellationToken);
        if (organization is null) return;
    }

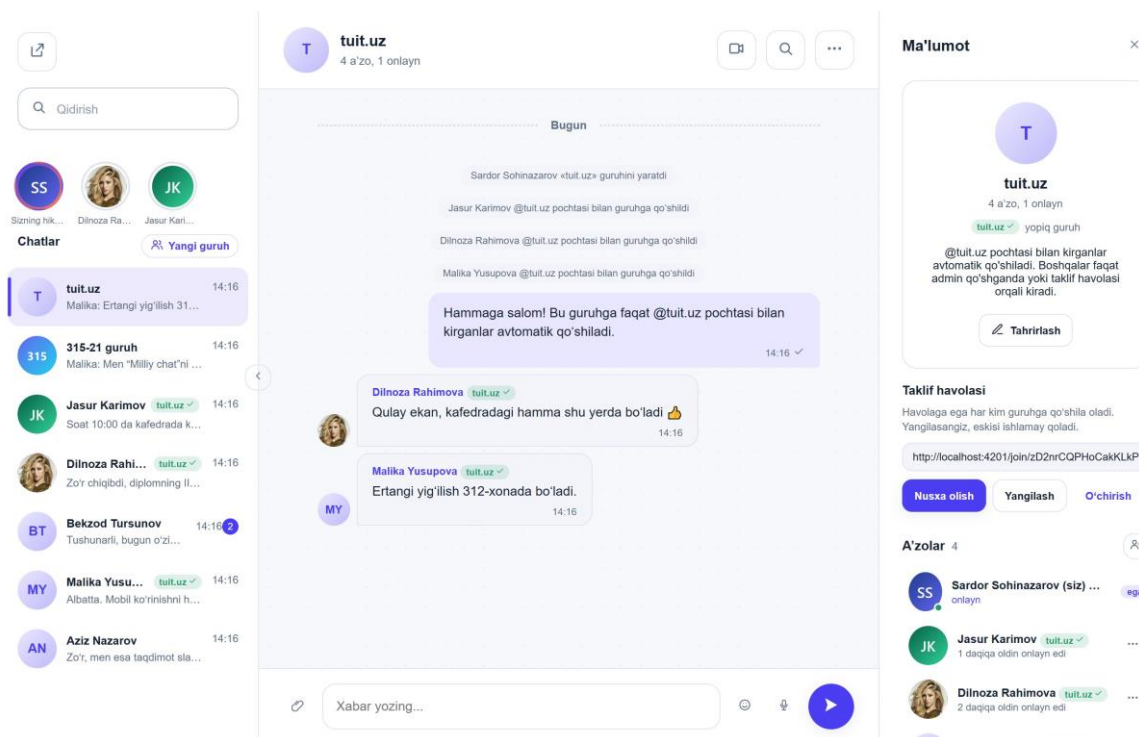
    // The first person from a domain becomes its admin.
    var role = isFounder ? OrganizationRole.Admin : OrganizationRole.Member;
    var member = OrganizationFactory.CreateMember(organization.Id, user.Id, role,
now);
    if (!await repository.TryAddMemberAsync(member, cancellationToken)) return;

    var groupChatIds = await
repository.GetOrganizationGroupChatIdsAsync(organization.Id, cancellationToken);
    if (groupChatIds.Count == 0)
    {
        await groupService.CreateOrganizationGroupAsync(user.Id, cancellationToken);
        return;
    }

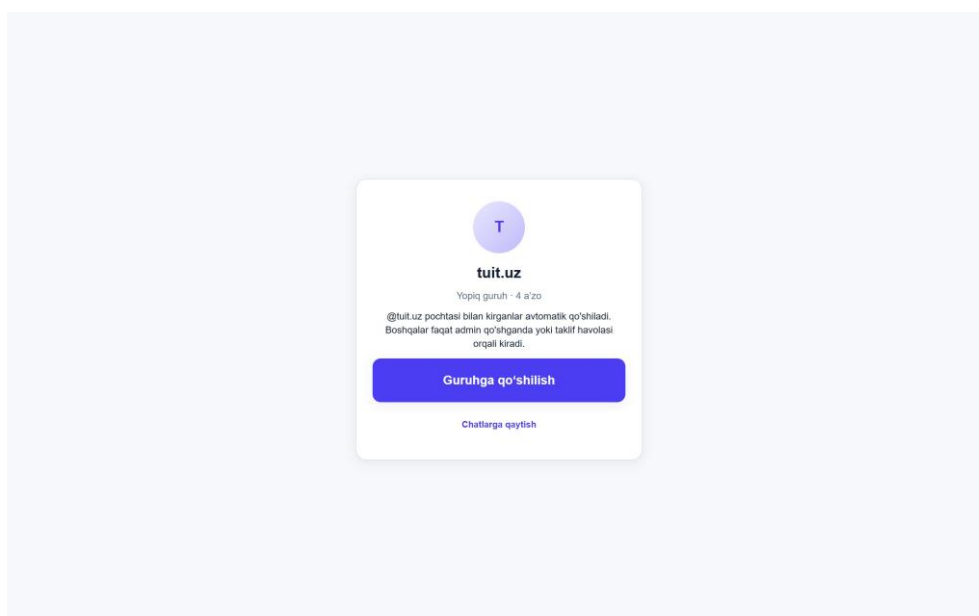
    foreach (var chatId in groupChatIds)
        await groupService.JoinViaOrganizationAsync(chatId, user.Id,
cancellationToken);
}
```

Tashkilot a'zoliqi foydalanuvchi haqidagi barcha DTO'larga (ProfileDto, UserSearchDto, MessageSenderDto, GroupMemberDto) OrganizationBadgeDto sifatida qo'shilgan. Klientdagi org-badge komponenti uni ismlar yonida "tuit.uz ✓"

ko‘rinishida chiqaradi. Domen guruhi 3.10-rasmda ko‘rsatilgan: tuit.uz pochta bilan kirgan har bir foydalanuvchi guruhga o‘zi qo‘shilgan va bu xizmat xabarlar bilan qayd etilgan. Ma’lumot panelida guruhning yopiqligi haqida izoh va faqat adminlarga ko‘rinadigan taklif havolasi bor. Tashkilotdan tashqaridagi foydalanuvchi havolani ochganda 3.11-rasmdagi sahifani ko‘radi.



3.10-rasm. Tashkilotning yopiq domen guruhi va taklif havolasi



3.11-rasm. Taklif havolasi orqali yopiq guruhga qo‘shilish sahifasi

3.7. Maxfiy chatlarni amalga oshirish

Server qismi.

Server tomonida maxfiy chatlar SecretChatService servisi va SecretChatController orqali amalga oshirilgan. Barcha endpointlar joriy qurilma nomidan ishlaydi: qurilma access token ichidagi sid (sessiya identifikatori) bo'yicha aniqlanadi. JWT logoutdan keyin ham bir necha daqiqa yaroqli bo'lib qolishi mumkin, shuning uchun har bir so'rovda sessiyaning faolligi ma'lumotlar bazasidan qayta tekshiriladi. SecretChatAccess yordamchi sinfi so'rov qaysi tomondan (tashabbuskor yoki qabul qiluvchi) va aynan chat bog'langan qurilmadan kelayotganini aniqlaydi. Boshqa qurilmadan kelgan so'rov "topilmadi" javobini oladi.

Shifrlangan xabarni qabul qilish 3.7-listingda ko'rsatilgan. Server blobni ochmaydi va uning tuzilishiga qaramaydi: faqat Base64 formati va hajmi (32 KB gacha) tekshiriladi. Tartib raqami yuboruvchi tomonning oxirgi raqamidan katta bo'lishi shart, aks holda so'rov takroriy deb rad etiladi. Bir vaqtda kelgan ikki bir xil so'rov uchun (SecretChatId, SenderSessionId, Seq) unique indeks ikkinchi himoya chizig'ini beradi. Xabar faqat qabul qiluvchi qurilmaning "session:{sid}" guruhiga yuboriladi.

3.7-listing. SecretChatService.SendAsync: shifrlangan xabarni qabul qilish (qisqartirilgan)

```
var chat = await repository.GetAsync(secretChatId, cancellationToken);
var side = chat is null ? SecretChatAccess.Side.None : SecretChatAccess.SideOf(chat,
userId, sessionId);
if (chat is null || side == SecretChatAccess.Side.None) return new(null,
NotFoundError);
if (chat.Status != SecretChatStatus.Active) return new(null, "Maxfiy chat faol
emas.");

var lastSeq = side == SecretChatAccess.Side.Initiator ? chat.InitiatorLastSeq :
chat.ParticipantLastSeq;
if (request.Seq <= lastSeq) return new(null, DuplicateSeqError, Duplicate: true);

if (side == SecretChatAccess.Side.Initiator) chat.InitiatorLastSeq = request.Seq;
else chat.ParticipantLastSeq = request.Seq;

var ciphertext = SecretChatBase64.TryDecode(request.Ciphertext)!;
var message = SecretChatFactory.CreateMessage(chat.Id, sessionId, request.Seq,
ciphertext, Now());
if (!await repository.TryAddMessageAsync(message, cancellationToken))
    return new(null, DuplicateSeqError, Duplicate: true);

var recipientSessionId = side == SecretChatAccess.Side.Initiator
    ? chat.ParticipantSessionId!.Value : chat.InitiatorSessionId;
await realtimeNotifier.MessageReceivedAsync(SecretChatMapper.ToDto(message),
recipientSessionId, cancellationToken);
```

Qabul qiluvchi qurilma xabarni olgach, POST /api/secret-chats/{id}/ack so‘rovi bilan tasdiqlaydi va server shu raqamgacha bo‘lgan xabarlarni o‘chiradi. Logout, boshqa qurilmalarni chiqarish yoki bitta qurilmani chiqarish AuthService’da shu qurilmalarga bog‘langan maxfiy chatlarni darhol yopadi. SecretChatCleanup fon ishi esa har soatda muddati tugagan sessiyalarning chatlarini, 7 kundan beri javob kutayotgan so‘rovlarni hamda 7 kundan eski yetkazilmagan xabar va fayllarni tozalaydi. Shifrlangan fayllar LocalSecretFileStorage tomonidan veb-ildizdan tashqaridagi App_Data/secret-files papkasida GUID nom bilan saqlanadi. Bitta chatda yetkazilmagan fayllar soni 20 ta bilan cheklangan, bu esa bir qurilmaning server diskini to‘ldirib yuborishiga yo‘l qo‘ymaydi.

Klient qismi.

Klientdagi kriptografik kod secret-crypto.ts modulida joylashgan va Angular’ga umuman bog‘liq emas. Shu sababli uni alohida sinash va kerak bo‘lsa Web Worker’ga ko‘chirish oson. Xabarni shifrlash funksiyasi va zanjirni bir qadam

oldinga surish 3.8-listingda keltirilgan. Yangi zanjir holati xabar yuborilishidan oldin IndexedDB'ga xabar bilan bitta tranzaksiyada yoziladi. Shu tufayli sahifa to'satdan yopilsa ham bitta xabar kaliti hech qachon ikki marta ishlatilmaydi.

3.8-listing. secret-crypto.ts: xabarni shifrlash va ratchet qadami (TypeScript, qisqartirilgan)

```
export async function encryptMessage(chain: ChainState, secretChatId: number,
  direction: SecretDirection, payload: SecretPayload) {
  const seq = chain.seq + 1;
  const { messageKey, nextChainKey } = await step(chain.chainKey);
  const iv = globalThis.crypto.getRandomValues(new Uint8Array(IV_BYTES));
  const plaintext = encoder.encode(JSON.stringify({ v: PROTOCOL_VERSION, ...payload }));
  const sealed = new Uint8Array(await subtle().encrypt(
    { name: 'AES-GCM', iv, additionalData: associatedData(secretChatId, direction, seq) },
    messageKey, plaintext));

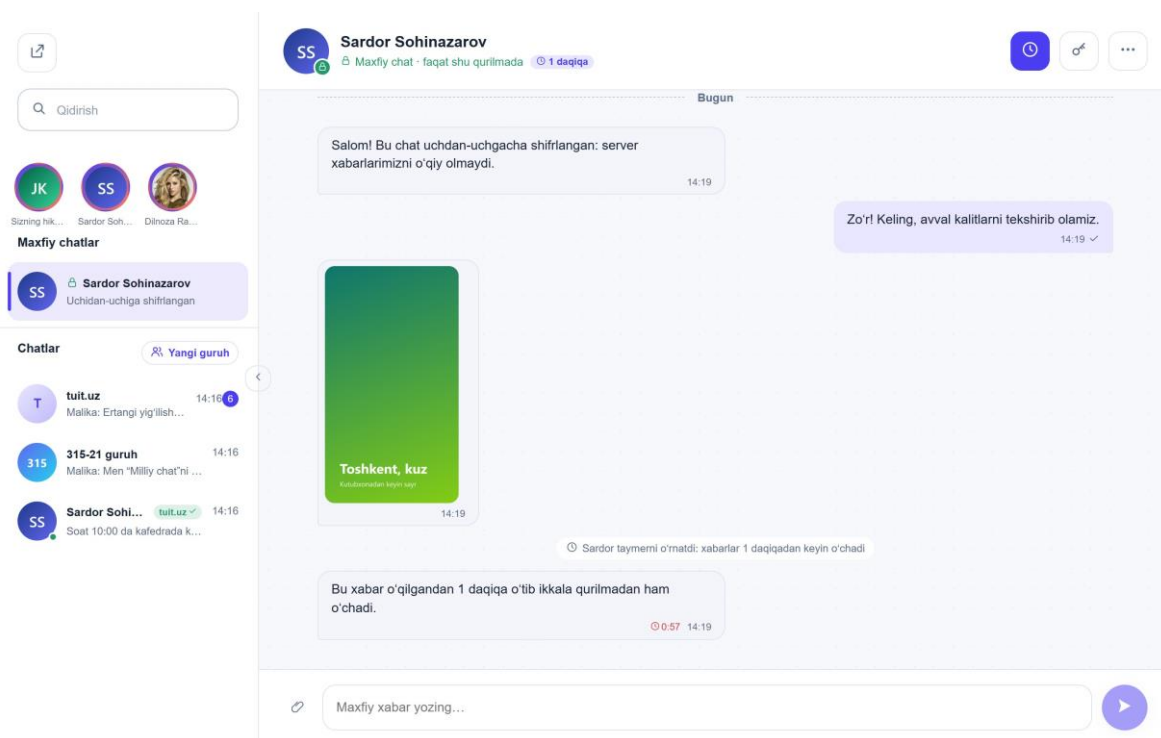
  const envelope = new Uint8Array(1 + IV_BYTES + sealed.length);
  envelope[0] = PROTOCOL_VERSION;
  envelope.set(iv, 1);
  envelope.set(sealed, 1 + IV_BYTES);
  return { chain: { chainKey: nextChainKey, seq }, seq, ciphertext: toBase64(envelope) };
}

async function step(chainKey: CryptoKey) {
  const messageBytes = new Uint8Array(await subtle().sign('HMAC', chainKey, MESSAGE_KEY_INPUT));
  const chainBytes = new Uint8Array(await subtle().sign('HMAC', chainKey, CHAIN_KEY_INPUT));
  try {
    const messageKey = await subtle().importKey('raw', messageBytes, { name: 'AES-GCM' },
    false, ['encrypt', 'decrypt']);
    return { messageKey, nextChainKey: await importChainKey(chainBytes) };
  } finally {
    messageBytes.fill(0); // raw key bytes live only for a moment
    chainBytes.fill(0);
  }
}
```

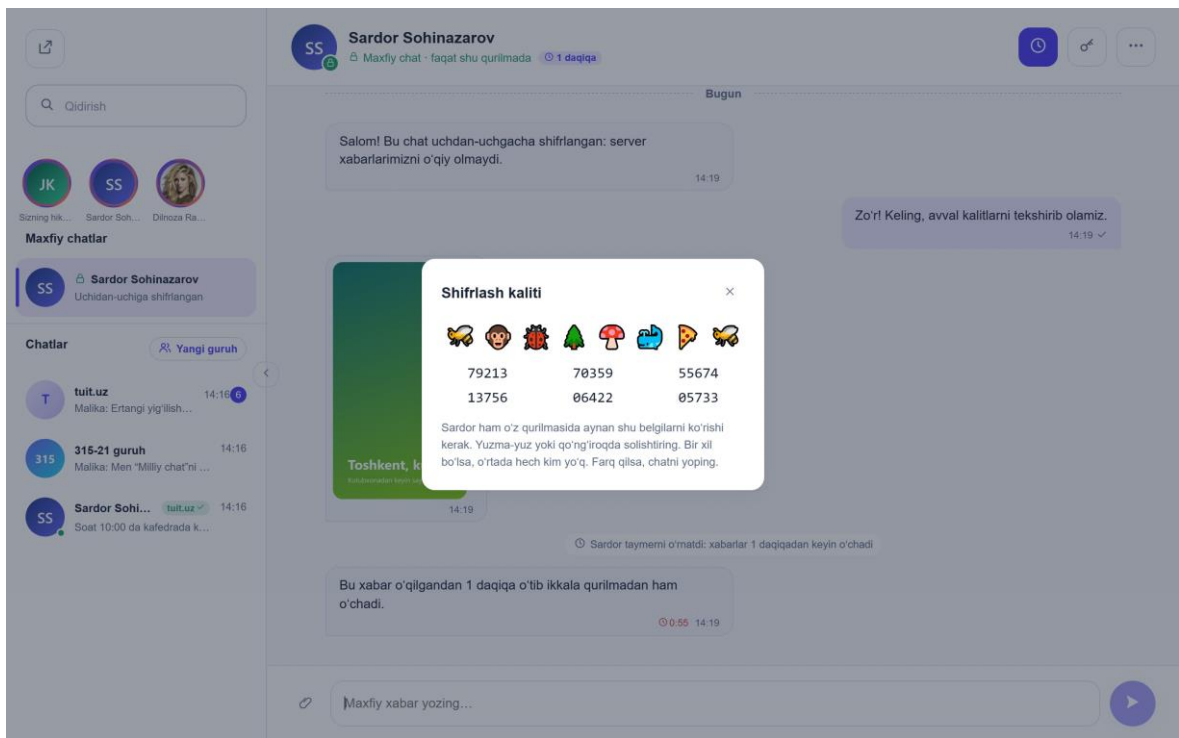
Xabarlar avval qurilmadagi navbatga (outbox) yoziladi va qat'iy tartibda yuboriladi. Tarmoq uzilsa, qayta ulanishda yuborish davom ettiriladi. Qabul qilishda xabar faqat muvaffaqiyatli ochilgandan yoki tashlab yuborilgandan keyin tasdiqlanadi. Maxfiy chat oynasi (SecretConversationComponent) oddiy suhbat oynasidan farq qiladi: sarlavhada qulf belgisi va “faqat shu qurilmada” yozuvi, taymer va kalit tekshirish tugmalari bor, chatlar ro'yxatida esa maxfiy chatlar alohida bo'limda ko'rsatiladi (3.12-rasm). Rasmda suhbatdoshga yuborilgan shifrlangan rasm va taymer o'rnatilgandan keyingi xabar ko'rinib turibdi, xabar

yonida qolgan vaqt hisoblanmoqda. Kalit izini tekshirish oynasi 3.13-rasmda keltirilgan.

Protokolning asosiy xususiyati — server xabar mazmunini bilmasligi — amalda ham tekshirildi. Suhbatdosh oflayn boʻlgan paytda yuborilgan xabar serverda yetkazilishini kutib turadi. Uning maʼlumotlar bazasidagi koʻrinishi 3.14-rasmda keltirilgan: jadvalda faqat 183 baytli maʼnosiz shifrlangan blob bor. Ilgari yuborilgan va yetkazilgan xabarlar esa tasdiqdan keyin serverdan butunlay oʻchirilgan.



3.12-rasm. Maxfiy chat: shifrlangan rasm va oʻz-oʻzini oʻchirish taymeri



3.13-rasm. Shifrlash kaliti izini tekshirish oynasi

```
PS> psql -d NationalChat -c "SELECT Id, SecretChatId, Seq, length(Ciphertext), encode(Ciphertext, base64) FROM messaging.secret_messages;"
 Id | chat | Seq | bytes |                ciphertext                | created
-----+-----+-----+-----+-----+-----
  6 |    1 |   5 |   183 | AdBp3POJtrxqSydreRLgRuQr0qIrOMLWPeLcchcGuJmc... | 02:19:24
(1 row)
```

3.14-rasm. secret_messages jadvali: serverda faqat yetkazilmagan shifrlangan blob saqlanadi

3.8. Klient qismini ishlab chiqish

Klient ilovasi Angular 22 ning mustaqil (standalone) komponentlaridan tashkil topgan. Ilova konfiguratsiyasida (app.config.ts) marshrutizator, SSR sahifasini brauzerda “jonlantirish” (hydration) va HTTP klient ulanadi. HTTP klientga authInterceptor ham qo‘shiladi. Uchta marshrut mavjud: /auth (kirish va ro‘yxatdan o‘tish), /chat (asosiy messenjer sahifasi) va /join/:token (taklif havolasi). /chat sahifasi authGuard bilan, /auth sahifasi esa guestGuard bilan himoyalangan: tizimga kirgan foydalanuvchi kirish sahifasini ko‘rmaydi. Taklif havolasini tizimga kirmagan odam ochsa, havola eslab qolinadi va kirishdan keyin sahifa avtomatik ochiladi. Barcha sahifalar kerak bo‘lganda yuklanadi (lazy loading), bu esa boshlang‘ich yuklanish hajmini kamaytiradi.

Ilova holati Angular signals mexanizmi orqali boshqariladi. ChatApiService'da chatlar ro'yxati, ochiq chat xabarlar, joriy guruh ma'lumotlari va "yozmoqda" holati signal ko'rinishida saqlanadi. Komponentlar ulardan computed() yordamida hosila qiymatlar oladi, masalan sarlavhadagi "6 a'zo, 2 onlayn" yoki "Jasur yozmoqda..." matni. Signal o'zgarganda Angular faqat unga bog'liq qismlarni qayta chizadi. Holat bitta servisda to'plangani uchun REST javoblari ham, SignalR hodisalari ham bir xil yo'l bilan qo'llanadi. Maxfiy chatlar holati esa alohida SecretChatService'da saqlanadi, chunki uning manbai server emas, balki qurilmadagi IndexedDB.

authInterceptor har bir API so'roviga access tokenni qo'shadi. Server 401 status kodini qaytarsa, interceptor tokenni avtomatik yangilab, so'rovni qayta yuboradi (3.9-listing). Bir vaqtda bir nechta so'rov muvaffaqiyatsiz bo'lsa ham, AuthService ular uchun bitta umumiy refresh so'rovini ishlatadi. Yangilash imkonsiz bo'lsa, foydalanuvchi kirish sahifasiga yo'naltiriladi.

3.9-listing. authInterceptor: muddati o'tgan tokenni avtomatik yangilash (TypeScript)

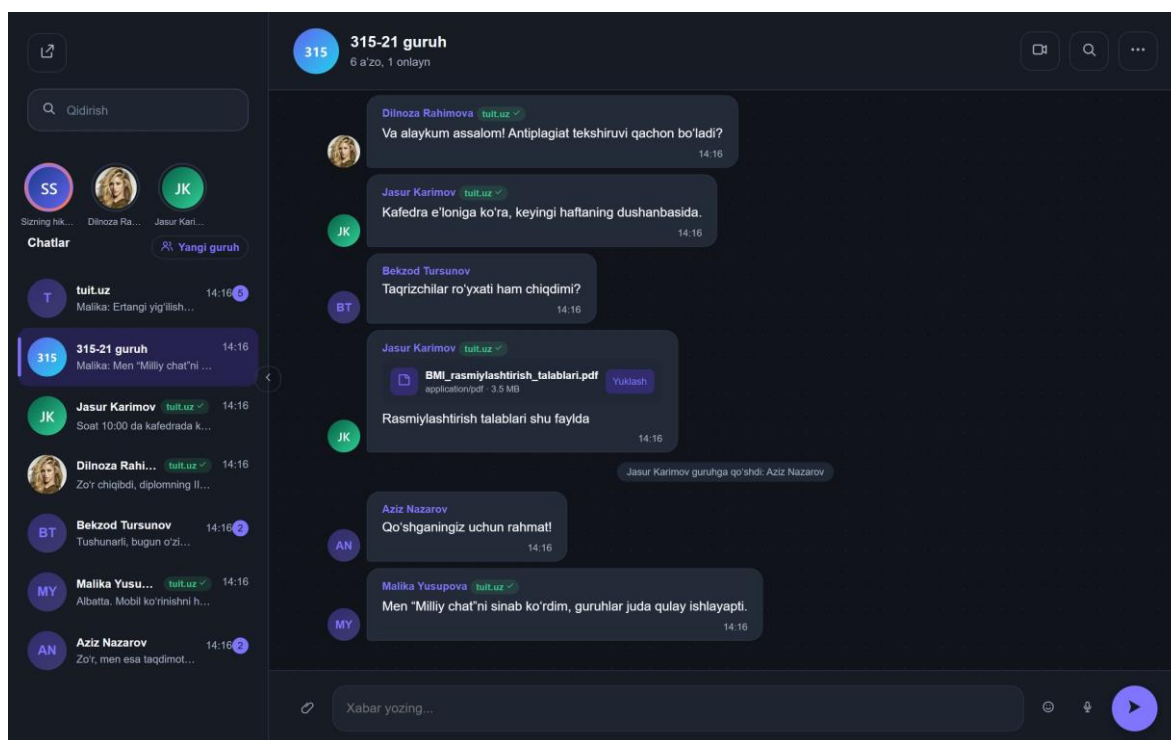
```
return next(authenticatedRequest).pipe(
  catchError((error: HttpErrorResponse) => {
    if (error.status !== 401 || !isApiRequest || isRefreshRequest || isAnonymous ||
    !auth.accessToken())
      return throwError(() => error);
    return auth.refreshAccessToken().pipe(
      switchMap((token) =>
        next(request.clone({ setHeaders: { Authorization: `Bearer ${token}` } })),
      ),
      catchError((refreshError) => {
        auth.clearToken();
        router.navigateByUrl('/auth');
        return throwError(() => refreshError);
      })
    );
  })
);
```

Interfeys loyiha uchun ishlab chiqilgan dizayn tizimiga asoslangan. Umumiy UI komponentlar (tugma, ikonkali tugma, matn maydoni, checkbox, banner, tasdiqlash oynasi, tashkilot belgisi, ikonkalar to'plami) shared/ui papkasida joylashgan. Ranglar, soyalar va radiuslar CSS o'zgaruvchilari (design tokens) orqali berilgan. Qorong'i mavzu shu o'zgaruvchilarning muqobil qiymatlari yordamida

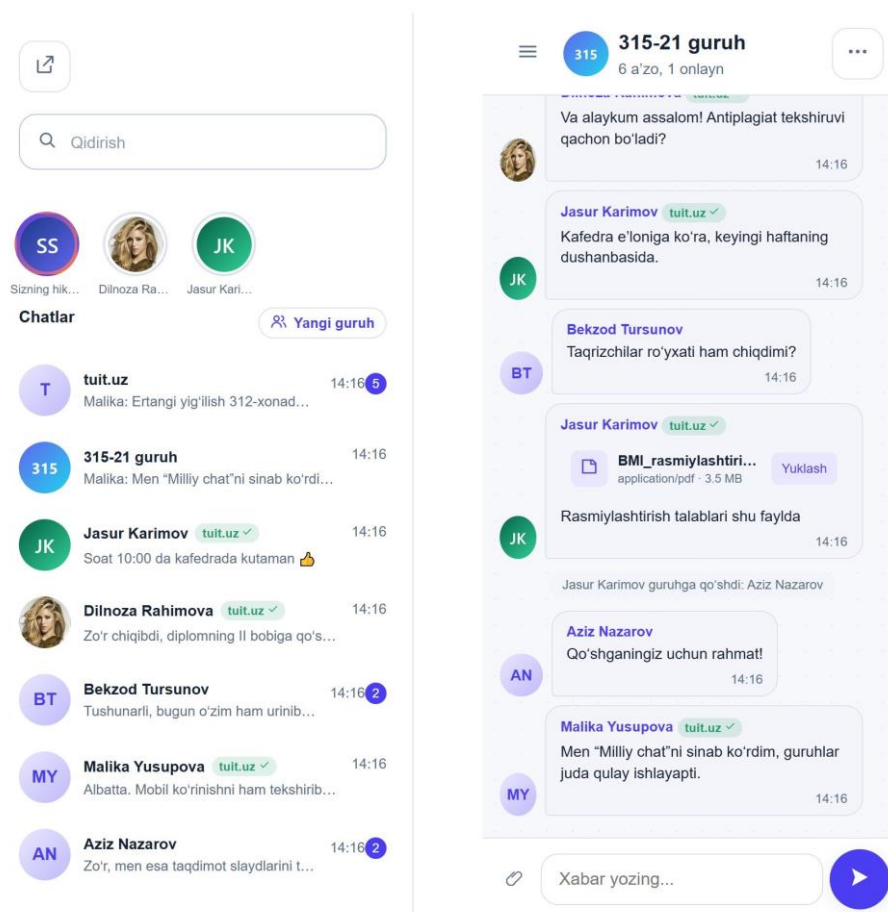
amalga oshirilgan (3.15-rasm). Foydalanuvchi tanlagan mavzu brauzer xotirasida saqlanadi.

Interfeys moslashuvchan (responsive) qilib yaratilgan. Katta ekranda uch ustunli joylashuv (chatlar ro'yxati, suhbat, ma'lumot paneli) ishlatiladi. Planshetda chatlar ro'yxati yig'iladi. Telefonda esa bir vaqtda bitta ekran ko'rsatiladi: chatlar ro'yxati yoki suhbat (3.16-rasm). Profil va guruh muharrirlari ekran markazidagi dialog oynasida ochiladi.

Ilova qat'iy Content Security Policy bilan ishlaydi (2.5-bo'lim). Asosiy siyosat index.html faylida meta-teg sifatida beriladi: skriptlar faqat ilovaning o'z manzilidan va Google Identity xizmatidan, sahifa ichidagi skriptlar esa faqat oldindan hisoblangan SHA-256 xeshlari bo'yicha ruxsat etiladi. Meta-teg orqali berib bo'lmaydigan frame-ancestors sarlavhasi SSR serveri (server.ts) tomonidan qo'shiladi.



3.15-rasm. Qorong'i mavzudagi interfeys



3.16-rasm. Mobil qurilmadagi ko‘rinish: chatlar ro‘yxati va guruh chati

3.9. Joylashtirish va testlash

Joylashtirish.

Server ilovasi Docker konteyneri ko‘rinishida joylashtiriladi (3.10-listing). Birinchi bosqichda .NET SDK obrazida bog‘liqliklar tiklanadi va loyiha Release rejimida nashr qilinadi. Loyiha fayllari manba kodidan oldin nusxalanadi, shuning uchun Docker kodi o‘zgarganda ham bog‘liqliklar qatlamini keshdan qayta ishlatadi. Ikkinchi bosqichda faqat nashr natijasi kichik hajmli ASP.NET runtime obraziga ko‘chiriladi. Shu tufayli yakuniy obrazda kompilyator va manba kodi bo‘lmaydi.

3.10-listing. Server ilovasining Dockerfile fayli

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY src/API/API.csproj src/API/
COPY src/Application/Application.csproj src/Application/
COPY src/Domain/Domain.csproj src/Domain/
COPY src/Infrastructure/Infrastructure.csproj src/Infrastructure/
RUN dotnet restore src/API/API.csproj
COPY src/ src/
RUN dotnet publish src/API/API.csproj -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "API.dll"]
```

Konteyner Render bulut platformasida “Web Service” sifatida ishga tushiriladi. Platforma GitHub repozitoriysidagi o‘zgarishlardan so‘ng obrazni avtomatik qayta yig‘adi. Maxfiy sozlamalar (ma’lumotlar bazasiga ulanish satri, JWT va HMAC kalitlari, SMTP va Google sozlamalari, ruxsat etilgan klient manzillari) kodda saqlanmaydi. Ular muhit o‘zgaruvchilari orqali beriladi, appsettings fayllari esa Git’dan chiqarib tashlangan. Ma’lumotlar bazasi sxemasi yettita EF Core migratsiyasi orqali bosqichma-bosqich yangilanadi. Klient ilovasining ishlab chiqarish konfiguratsiyasi (environment.prod.ts) bulutdagi API manziliga yo‘naltirilgan.

Unit testlar.

Biznes-mantiqning to‘g‘riligini tekshirish uchun xUnit freymvorkida NationalChat.Tests test loyihasi yaratildi. Tashqi bog‘liqliklar (real vaqt xabarnomasi, fayl servisi, repozitoriylar) NSubstitute kutubxonasi yordamida soxtalashtirildi (mock). Guruh va maxfiy chat servislari uchun esa repozitoriy va fayl omborining xotirada ishlaydigan soxta amalga oshirilishlari yozildi. Kursorli sahifalash EF Core InMemory provayderida sinaldi. Vaqtga bog‘liq mantiq (taymerlar, muddatlar, tozalash) qat’iy vaqt qaytaradigan TimeProvider yordamida tekshirildi.

Server testlari soni 307 taga yetdi. Klient qismi uchun esa Vitest freymvorkida 12 ta test faylida 229 ta test yozildi. Ular transliteratsiyaning server bilan bir xil ishlashini (o‘sha JSON to‘plami bo‘yicha), shifrlash va ochish, kalit zanjiri, tartibsiz

kelgan va soxta xabarlar, taymerlar, maxfiy chat oynasi, taklif sahifasi va boshqa komponentlarni tekshiradi. Testlarni bajarish natijasi 3.17-rasmda, server testlari tarkibi esa 3.4-jadvalda keltirilgan.

```
PS> dotnet test tests\NationalChat.Tests --logger "console;verbosity=minimal"
Test run for C:\Users\Sardor\Desktop\diplom ish\NationalChat\tests\NationalChat.Tests\bin\Debug\net8.0\NationalChat.Tests.dll (.NETCoreApp,Version=v8.0)
A total of 1 test files matched the specified pattern.
Passed! - Failed: 0, Passed: 307, Skipped: 0, Total: 307, Duration: 1 s - NationalChat.Tests.dll (net8.0)
PS> cd ../NationalChatClient; npx ng test --watch=false
RUN v4.1.10 C:\Users\Sardor\Desktop\diplom ish\NationalChatClient
Test Files 12 passed (12)
Tests 229 passed (229)
Duration 29.43s (transform 3.35s, setup 7.72s, import 6.08s, tests 6.03s, environment 256.78s)
```

3.17-rasm. Server (dotnet test) va klient (ng test) testlarini bajarish natijasi

3.4-jadval

Server unit testlari tarkibi va natijalari

Test sinfi	Soni	Tekshiriladigan holatlar	Natija
UzbekTransliteratorTests	160	Umumiy to‘plamdagi 80 ta juftlik ikkala yo‘nalishda; kontekst qoidalari, registr, tutuq belgilari, himoyalangan qismlar, qidiruv kaliti	O‘tdi
MessageSearchTextTests	7	Xabar yaratilganda va tahrirlanganda SearchText hisoblanishi, qidiruv so‘rovining normallashtirilishi	O‘tdi
OrganizationEmailMatcherTests	22	Domenni ajratish va normallashtirish, subdomenlar, o‘xshash domenlar, umumiy pochta xizmatlari	O‘tdi
OrganizationMembershipServiceTests	9	Birinchi foydalanuvchi — admin va guruh egasi; keyingilar a’zo; gmail foydalanuvchisi hech narsa olmasligi; qayta kirishda takror qo‘shilmaslik	O‘tdi
OrganizationGroupTests	13	Domen guruhi, taklif havolasi (yaratish, yangilash, bekor qilish, noto‘g‘ri token), tashqi foydalanuvchining kira olmasligi	O‘tdi
SecretChatServiceTests	31	Qurilmaga bog‘lash, qabul qilish, takroriy seq’ni rad etish, boshqa qurilmadan yuborish, ack, fayllar, yopish va tozalash	O‘tdi
AuthServiceSecretChatTests, AuthServiceSessionReplacementTests	8	Logout va sessiyani yakunlashda maxfiy chatlarning yopilishi; shu brauzerdagi eski sessiyaning almashtirilishi	O‘tdi
ClientOriginOptionsTests	13	Ruxsat etilgan klient manzillari va boshqa manzillarni rad etish	O‘tdi
GroupServiceTests, ChatPermissionTests	19	Guruh yaratish va xizmat xabari, rollar va ruxsatlar, egalikni o‘tkazish, a’zolar limiti, chat o‘chirish huquqlari	O‘tdi

Test sinfi	Soni	Tekshiriladigan holatlar	Natija
Pbkdf2 va HmacRefreshTokenHasherTests	7	To'g'ri va noto'g'ri kod, har safar yangi tuz, kalitga bog'liqlik	O'tdi
CursorPaginationTests, ValidatorTests	18	Sahifalash, validatorlar (nom uzunligi, a'zolar ro'yxati, rollar, sahifalash chegaralari)	O'tdi
Jami	307		307 / 307

Barcha testlar muvaffaqiyatli o'tdi. Eng uzoq davom etgan testlar PBKDF2 xeshlash testlari bo'ldi (har biri ~230 ms). Bu 210 000 iteratsiyali xeshlash kodni tanlab topishni qanchalik qimmatlashtirishini amalda ko'rsatadi.

Integratsion va qo'lda sinovlar.

Unit testlardan tashqari, tizim lokal muhitda PostgreSQL ma'lumotlar bazasi bilan birgalikda sinovdan o'tkazildi. Guruh chatlari funksiyasi uchun API darajasida 43 ta tekshiruvdan iborat avtomatik ssenariy yozildi. U bir nechta foydalanuvchini ro'yxatdan o'tkazib, guruh yaratish, ruxsatlar, xizmat xabarlar, tarixni tozalash, egalikni o'tkazish va guruhni o'chirish holatlarini HTTP so'rovlari orqali tekshiradi. Barcha 43 ta tekshiruv muvaffaqiyatli o'tdi. Maxfiy chatlar ikki alohida brauzerda, haqiqiy server bilan sinaldi. Diplom ishidagi barcha skrinshotlar ham avtomatik ssenariy yordamida haqiqiy tizimdan olindi. Foydalanuvchi interfeysi brauzerda bir nechta akkaunt bilan qo'lda sinaldi. Asosiy ssenariylar 3.5-jadvalda keltirilgan.

3.5-jadval

Qo'lda o'tkazilgan asosiy sinov ssenariylari

№	Ssenariy	Kutilgan natija	Natija
1	Yangi e-pochta bilan kirish va ro'yxatdan o'tish	Kod keladi, profil yaratiladi, chat sahifasi ochiladi	Mos
2	Noto'g'ri kodni 5 marta kiritish	Kod bekor qilinadi, shundan keyin to'g'ri kod ham qabul qilinmaydi	Mos
3	Ikki brauzerda xabar almashish	Xabar sahifani yangilamasdan keladi, ✓✓ belgisi paydo bo'ladi	Mos
4	Suhbatdosh matn yozmoqda	Sarlavhada "yozmoqda..." ko'rinadi, 3 soniyada yo'qoladi	Mos
5	Rasm va PDF fayl yuborish	Fayllar yuklanadi, ulashilgan media panelida ko'rinadi	Mos

№	Ssenariy	Kutilgan natija	Natija
6	Guruh yaratish va a'zo qo'shish	Guruh barcha a'zolarida darhol paydo bo'ladi, xizmat xabari chiqadi	Mos
7	Oddiy a'zo guruhni boshqarishga urinadi	Boshqaruv tugmalari ko'rinmaydi, API 400 qaytaradi	Mos
8	Guruhdan chiqarilgan a'zo	Chat uning ro'yxatidan real vaqtda yo'qoladi, kirish huquqi yo'qoladi	Mos
9	Boshqa qurilma sessiyasini yakunlash	Qurilma ro'yxatdan o'chadi, uning tokeni yangilanmaydi, undagi maxfiy chatlar yopiladi	Mos
10	Telefon o'lchamida (375 px) foydalanish	Gorizontaal aylantirish yo'q, barcha amallar mavjud	Mos
11	Kirilda yozilgan xabarni "Lotin" sozlamasida ko'rish va lotincha qidirish	Xabar lotinda ko'rinadi, lotincha so'rov kirillcha xabarni topadi	Mos
12	@tuit.uz va @gmail.com pochtalari bilan ro'yxatdan o'tish	Birinchisi belgi oladi va domen guruhiga qo'shiladi, ikkinchisi yo'q	Mos
13	Taklif havolasini yangilash	Eski havola "yaroqsiz", yangisi ishlaydi	Mos
14	Ikki brauzerda maxfiy chat, rasm va taymer	Kalit izlari bir xil, xabar va rasm ochiladi, muddati o'tgan xabar ikkala tomonda o'chadi	Mos

Sinovlar davomida ikkita kamchilik aniqlandi va tuzatildi. Uzun guruh nomi mobil ekranda sarlavhani ekran chegarasidan chiqarib yuborayotgan edi. Guruh tarixi tozalanganda esa klient xizmat xabarlarini ham ekrandan olib tashlab, server bilan nomuvofiqlik yuzaga kelayotgan edi.

3.10. Foydalanish yo'riqnomasi

Tizimga kirish va ro'yxatdan o'tish.

Ilova ochilganda kirish sahifasi ko'rsatiladi (3.18-rasm). Foydalanuvchi elektron pochta manzilini kiritib, "Davom etish" tugmasini bosadi yoki Google hisobi orqali kiradi. Shundan so'ng pochtaga 6 xonali tasdiqlash kodi yuboriladi va kodni kiritish oynasi ochiladi (3.19-rasm). Manzil tizimda birinchi marta ishlatilayotgan bo'lsa, kod tasdiqlangach profil yaratish oynasi chiqadi. Unda ism, familiya va noyob username kiritiladi (3.20-rasm). Tashkilot pochta (masalan, @tuit.uz) bilan kirilganda foydalanuvchi tashkilotning domen guruhiga avtomatik qo'shiladi.

 **MILLIY CHAT**

Xush kelibsiz

Akkauntingiz yo'qmi? [Ro'yxatdan o'ting](#)

 Tasdiqlash kodi emailingizga yuboriladi.

Email manzilingiz

 siz@example.com

☐ Ushbu qurilmada eslab qolish

[Yordam kerakmi?](#)

Davom etish

yoki davom eting

 Passkey bilan

 QR kodni skanerlash

Davom etish orqali Foydalanish shartlari va Maxfiylik siyosatiga rozilik bildirasiz.

Xavfsiz muloqot, o'zingizga xos.

Mahalliy platforma orqali yaqinlaringiz bilan erkin va ishonchli bog'laning. Xabarlaringiz sizniki — himoyalangan va maxfiy.



Mahalliy yordam va nazorat siz bilan.

3.18-rasm. Tizimga kirish sahifasi

 **MILLIY CHAT**

Kodni tasdiqlang

yangi.talaba@example.com manziliga yuborilgan 6 xonali kodni kiriting.

 Kod 10 daqiqa davomida amal qiladi.

Tasdiqlash kodi

0 0 0 0 0

Kodni tekshirish

[← Email manzilini o'zgartirish](#)

Davom etish orqali Foydalanish shartlari va Maxfiylik siyosatiga rozilik bildirasiz.

Xavfsiz muloqot, o'zingizga xos.

Mahalliy platforma orqali yaqinlaringiz bilan erkin va ishonchli bog'laning. Xabarlaringiz sizniki — himoyalangan va maxfiy.



Mahalliy yordam va nazorat siz bilan.

3.19-rasm. Tasdiqlash kodini kiritish oynasi

Profil yarating

Siz uchun Milliy chatdagi profilningizni sozlaymiz.

Ma'lumotlaringiz faqat profilningizda ko'rsatiladi.

Ism Familiya

Username

5-50 belgi: harf, raqam va pastki chiziq ishlatiladi.

Profilni yaratish

[Email manzilini o'zgartirish](#)

Davom etish orqali Foydalanish shartlari va Maxfiylik siyosatiga rozilik bildirasiz.

Xavfsiz muloqot, o'zingizga xos.

Mahalliy platforma orqali yaqinlaringiz bilan erkin va ishonchli bog'laning. Xabarlaringiz sizniki — himoyalangan va maxfiy.



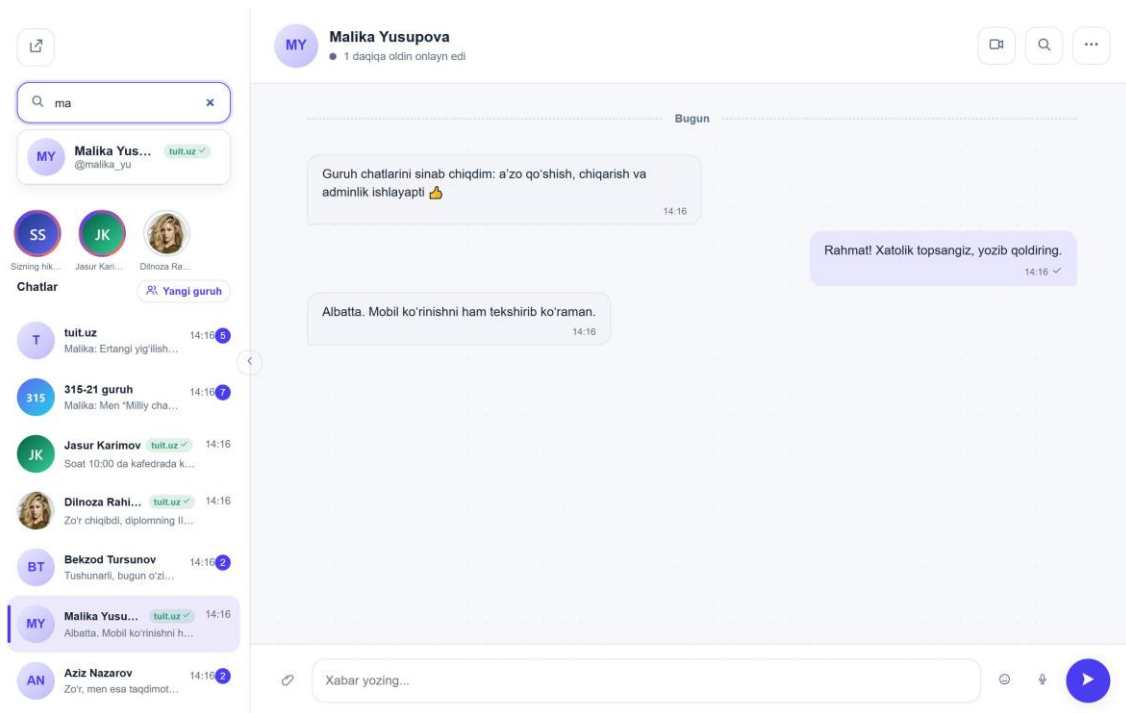
Mahalliy yordam va nazorat siz bilan.

3.20-rasm. Profil yaratish (ro'yxatdan o'tish) oynasi

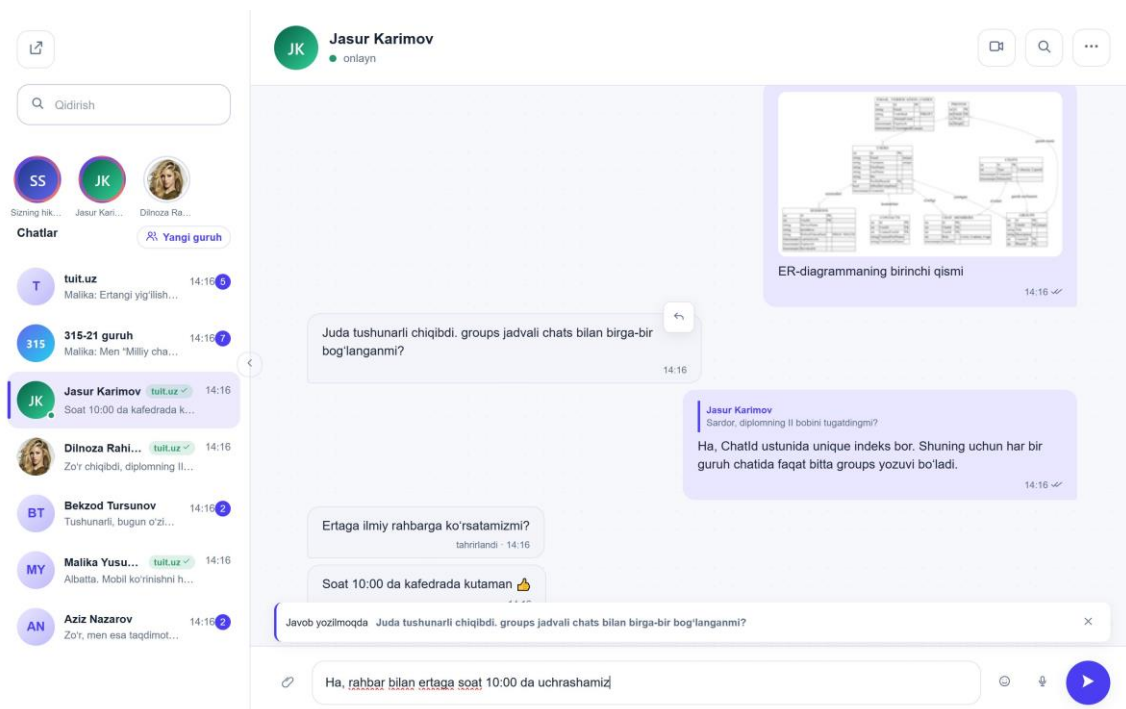
Suhbat boshlash va xabar almashish.

Asosiy sahifaning chap qismida qidiruv maydoni, hikoyalar paneli va chatlar ro'yxati joylashgan. Ro'yxatda har bir chatning oxirgi xabari, vaqti va o'qilmagan xabarlar soni ko'rsatiladi. Yangi suhbat boshlash uchun qidiruv maydoniga foydalanuvchining ismi yoki username'i yoziladi va natijalardan kerakli shaxs tanlanadi (3.21-rasm).

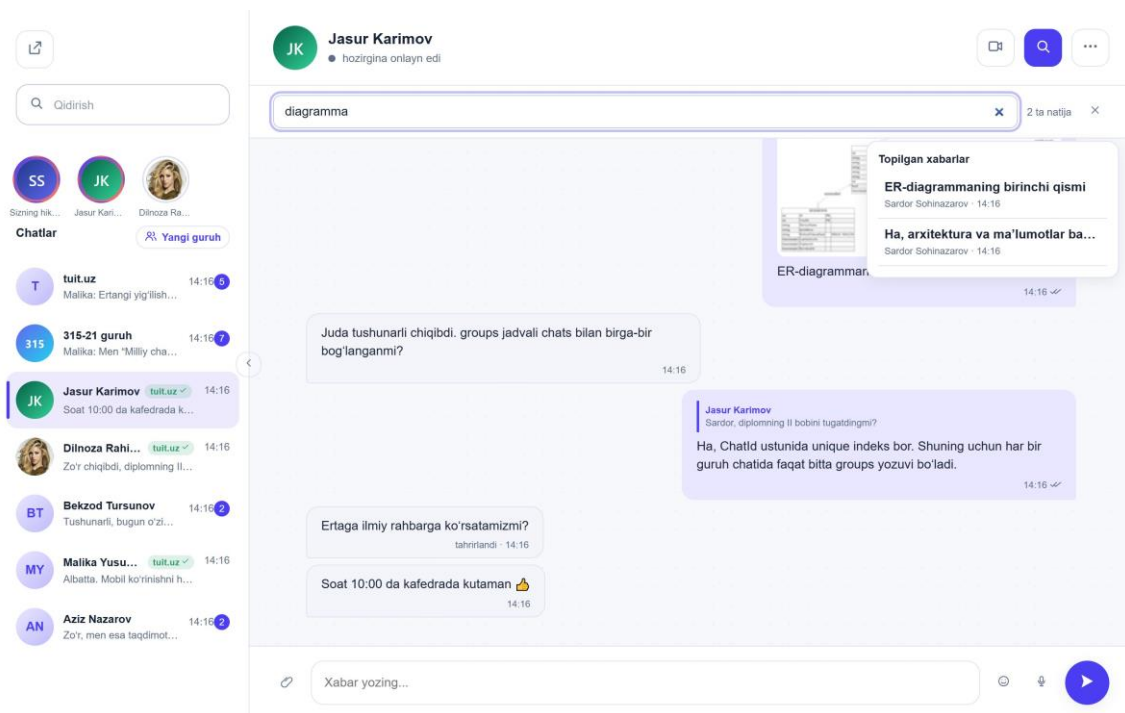
Xabar pastdagi maydonga yoziladi va Enter tugmasi yoki yuborish tugmasi bilan jo'natiladi. Qisqich () tugmasi orqali rasm, video yoki fayl biriktiriladi. Sichqoncha xabar ustiga olib borilganda amallar paneli chiqadi: javob berish, o'z xabarini tahrirlash va o'chirish. Javob berish tanlanganda yozish maydoni ustida asl xabar ko'rsatiladi (3.22-rasm). Suhbat sarlavhasidagi qidiruv tugmasi chat ichida matn bo'yicha qidirish imkonini beradi. Qidiruv yozuvga bog'liq emas: lotincha so'rov kirillda yozilgan xabarni ham topadi. Topilgan natija bosilganda chat o'sha xabarga aylantiriladi va xabar ajratib ko'rsatiladi (3.23-rasm).



3.21-rasm. Foydalanuvchilarni qidirish



3.22-rasm. Xabarga javob yozish

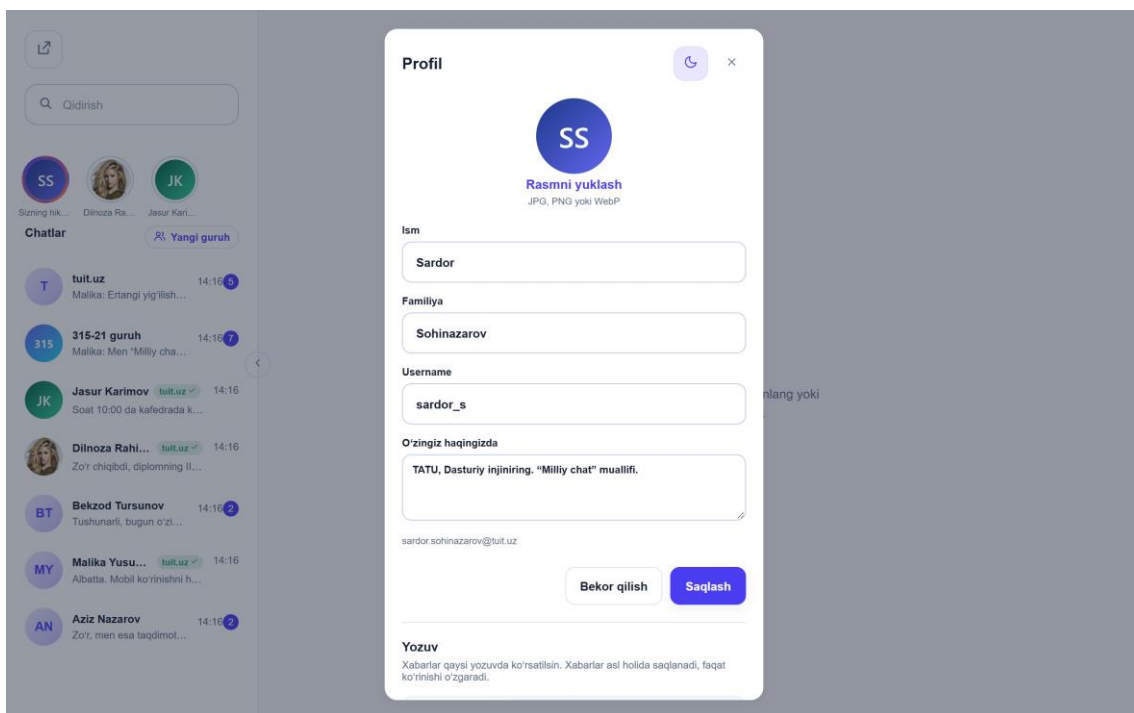


3.23-rasm. Chat ichida xabarlarni qidirish

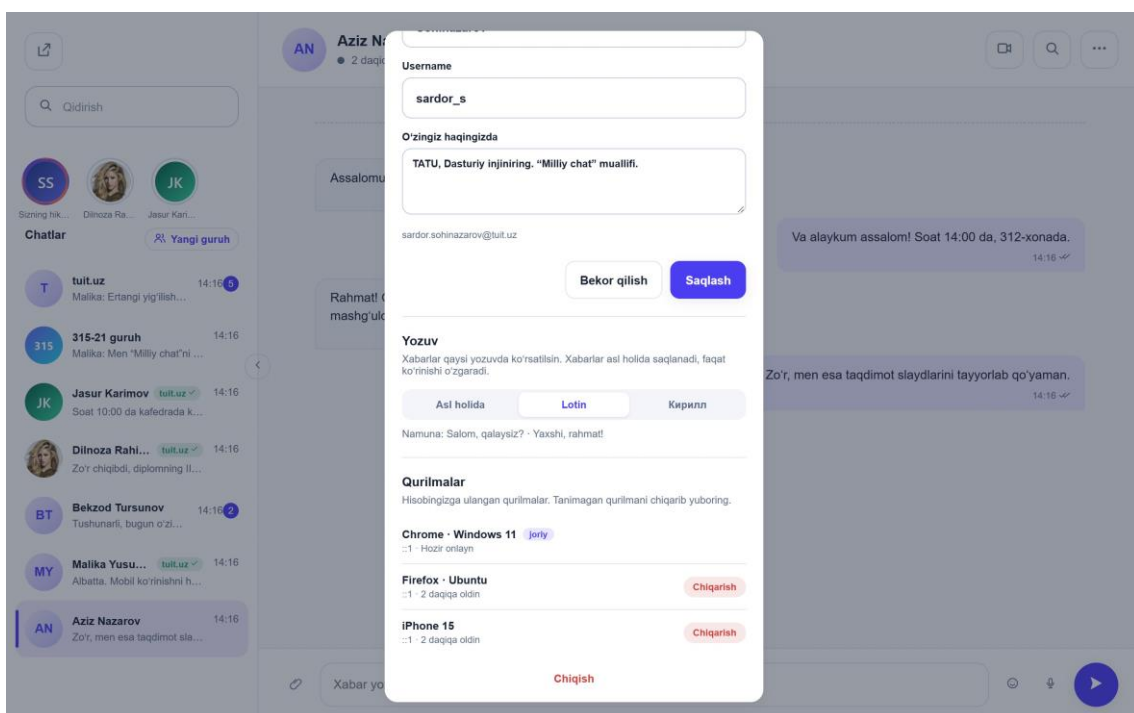
Profil, yozuv va qurilmalar.

Sidebar'ning yuqori qismidagi profil tugmasi orqali profil oynasi ochiladi. U yerda ism, familiya, username va o'zi haqidagi ma'lumotni tahrirlash, profil rasmini yuklash, yorug' va qorong'i mavzu o'rtasida almashish mumkin (3.24-rasm). "Qurilmalar" bo'limida hisobga ulangan barcha qurilmalar ro'yxati turadi. Joriy qurilma alohida belgilanadi, boshqa qurilmalarni esa "Chiqarish" tugmasi bilan hisobdan chiqarish mumkin.

Profil oynasidagi "Yozuv" bo'limida xabarlar qaysi yozuvda ko'rsatilishi tanlanadi: "Asl holida", "Lotin" yoki "Кирилл" (3.25-rasm). Tanlov darhol barcha suhbatlarga qo'llanadi, serverda saqlanadi va foydalanuvchining boshqa qurilmalarida ham amal qiladi. Bo'lim ostida tanlangan yozuvdagi namuna matn ko'rsatiladi.



3.24-rasm. Profilni tahrirlash oynasi



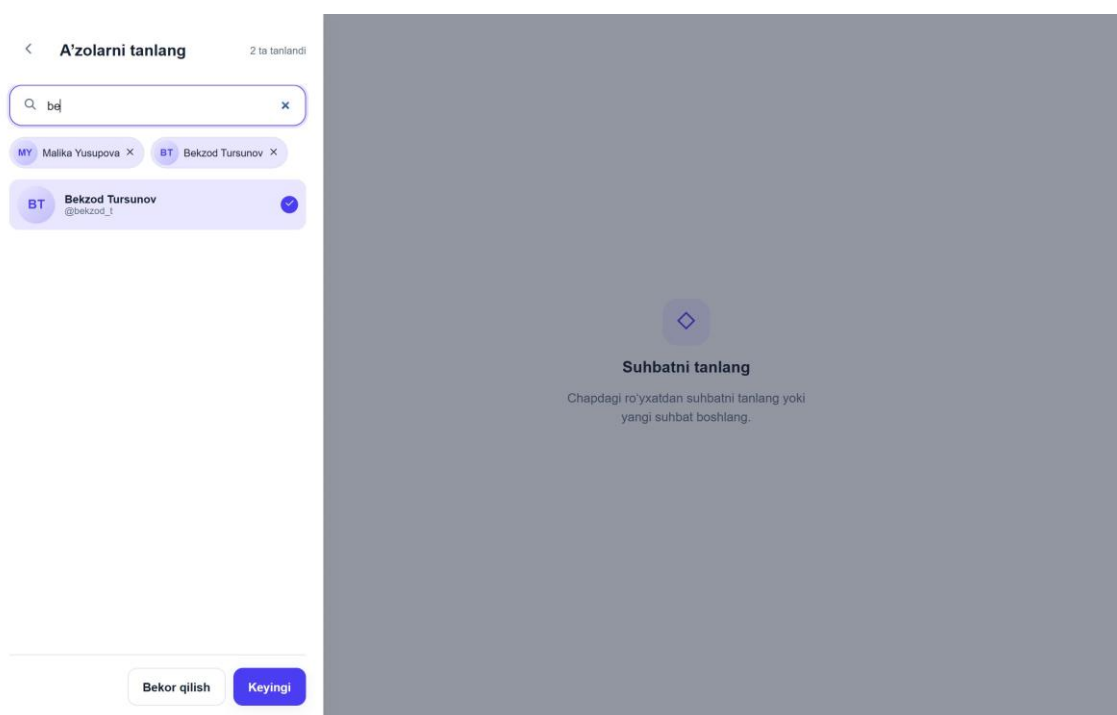
3.25-rasm. Profil oynasidagi “Yozuv” sozlamasi

Guruh yaratish va boshqarish.

Guruh yaratish uchun chatlar ro‘yxati ustidagi “Yangi guruh” tugmasi bosiladi. Birinchi qadamda qidiruv orqali a‘zolar tanlanadi. Tanlangan foydalanuvchilar yuqorida belgilar (chip) ko‘rinishida ko‘rsatiladi (3.26-rasm). “Keyingi” tugmasi bosilgach, guruh nomi, tavsifi va ixtiyoriy rasmi kiritiladi va

“Yaratish” tugmasi bosiladi (3.27-rasm). Yangi guruh barcha a’zolarning chatlar ro’yxatida darhol paydo bo’ladi.

Guruh sarlavhasi bosilganda ma’lumot paneli ochiladi (3.5-rasm). Guruh egasi va administratorlar bu yerdan guruhni tahrirlashi, yangi a’zo qo’shishi va a’zo yonidagi menyu orqali uni guruhdan chiqarishi mumkin. “Taklif havolasi” bo’limida havolani yaratish, nusxalash, yangilash va o’chirish mumkin (3.10-rasm). Havolani olgan odam uni brauzerda ochib, “Guruhga qo’shilish” tugmasini bosadi (3.11-rasm). Guruh egasi qo’shimcha ravishda administrator tayinlashi va guruhni butunlay o’chirishi mumkin. Istalgan a’zo “Guruhdan chiqish” tugmasi orqali guruhni tark eta oladi.



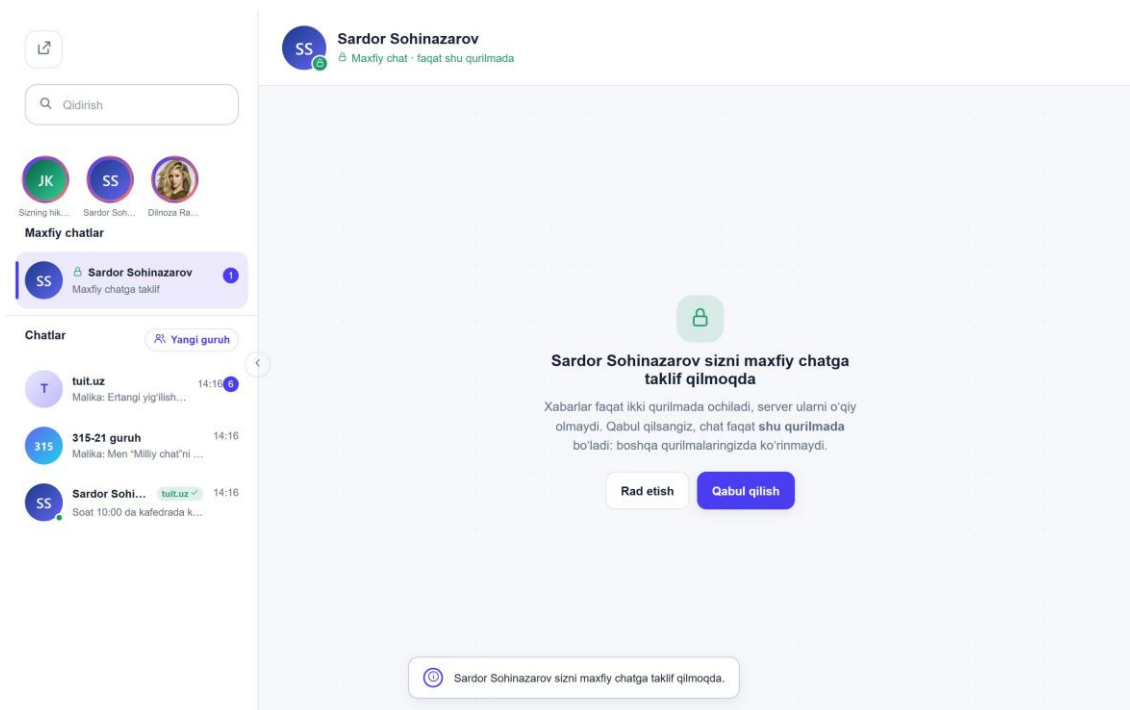
3.26-rasm. Guruh yaratish: a’zolarni tanlash

3.27-rasm. Guruh yaratish: nom va tavsifni kiritish

Maxfiy chat.

Maxfiy chat boshlash uchun shaxsiy chat sarlavhasidagi “...” menyusidan yoki suhbatdosh ma’lumot panelidan “Maxfiy chat boshlash” tanlanadi. Suhbatdoshga taklif boradi va u chatlar ro‘yxatining “Maxfiy chatlar” bo‘limida qulf belgisi bilan paydo bo‘ladi (3.28-rasm). Suhbatdosh “Qabul qilish” tugmasini bosgach, ikki qurilma kalitlarni almashadi va chat ochiladi. Xavfsizlikni tekshirish uchun sarlavhadagi kalit tugmasi bosiladi va ikkala tomonda ko‘rsatilgan emoji hamda raqamlar yuzma-yuz yoki qo‘ng‘iroq orqali solishtiriladi (3.13-rasm).

Soat tugmasi orqali o‘z-o‘zini o‘chirish taymeri tanlanadi (o‘chiq, 10 soniya, 1 daqiqa, 1 soat, 1 kun, 1 hafta). Qisqich tugmasi orqali shifrlangan fayl yuboriladi. “...” menyusidan tarixni tozalash yoki chatni yopish mumkin. Maxfiy chat faqat u ochilgan qurilmada ishlaydi. Foydalanuvchi shu qurilmada tizimdan chiqsa, chat va uning tarixi o‘chiriladi.



3.28-rasm. Maxfiy chat taklifi chatlar ro'yxatida

3-bob bo'yicha xulosa

Uchinchi bobda “Milliy chat” tizimining dasturiy amalga oshirilishi ko‘rib chiqildi. Server qismida 11 ta controller va 73 ta endpointdan iborat REST API, yagona javob formati, FluentValidation va markazlashgan xatolarni qayta ishlash, kursorli sahifalash, to‘rt metodli va o‘n to‘rt hodisali SignalR habi, onlayn holatni kuzatish, media fayllarni xavfsiz qayta ishlash konveyeri, guruh chatlari, taklif havolalari va hikoyalar amalga oshirildi. Klient qismida Angular signals asosidagi holat boshqaruvi, tokenni avtomatik yangilovchi interceptor, dizayn tizimi, qorong‘i mavzu, moslashuvchan interfeys va qat’iy Content Security Policy yaratildi.

Tizimning o‘ziga xos uchta imkoniyati amalga oshirildi. Lotin–kirill transliteratsiyasi server va klientda bitta test to‘plami bo‘yicha tekshiriladigan yagona algoritm asosida ishlaydi va yozuvdan qat’i nazar qidiruvni ta’minlaydi. Tashkilot rejimi e-pochta domeni orqali foydalanuvchini tasdiqlaydi va yopiq domen guruhlarini avtomatik shakllantiradi. Maxfiy chatlar Web Crypto API asosida uchdan-uchgacha shifrlangan bo‘lib, server xabar mazmunini bilmasligi ma’lumotlar bazasi darajasida ko‘rsatib berildi.

Tizim Docker konteyneri ko‘rinishida bulutli serverga joylashtirildi. Uning to‘g‘ri ishlashi serverdagi 307 ta va klientdagi 229 ta unit test, API darajasidagi 43 ta integratsion tekshiruv va qo‘lda o‘tkazilgan sinov ssenariylari bilan tasdiqlandi. Bob yakunida skrinshotlar bilan foydalanish yo‘riqnomasi keltirildi.

IV-BOB. HAYOT FAOLIYATI XAVFSIZLIGI

Hayot faoliyati xavfsizligi inson faoliyati jarayonida yuzaga keladigan xavfli va zararli omillarni aniqlash, ularning ta'sirini kamaytirish hamda favqulodda vaziyatlarda odamlar hayoti va sog'lig'ini saqlashga qaratilgan bilimlar va chora-tadbirlar tizimidir. "Milliy chat" kabi dasturiy mahsulotni ishlab chiqish va unga xizmat ko'rsatish jamoasi ish vaqtining asosiy qismini kompyuter qarshisida o'tkazadi. Shu sababli ushbu bobda ikki masala ko'rib chiqiladi: dasturchining ish joyini ergonomik talablar asosida tashkil etish hamda IT-kompaniya ofisida evakuatsiya tadbirlarini rejalashtirish va avariya yoki yong'in sodir bo'lganda xodimlarning harakatlari.

4.1. Dasturchining ergonomik ish joyini tashkil etish

Ergonomika (yunoncha *ergon* — ish va *nomos* — qonun) inson, mehnat vositalari va ish muhitining o'zaro moslashuvini o'rganadigan fan. Uning maqsadi mehnatni xavfsiz, qulay va samarali qilishdir. Ergonomik ish joyida jihozlar insonning antropometrik va fiziologik xususiyatlariga moslashtiriladi, aksincha emas.

Dasturchi mehnati aqliy mehnat turiga kiradi. Unda axborotni qabul qilish va qayta ishlash, diqqatni uzoq vaqt jamlash va ko'rish organlariga yuqori yuklama ustunlik qiladi. Jismoniy yuklama esa kam va asosan statik bo'ladi: xodim soatlab bir xil holatda o'tiradi, faqat barmoqlar va ko'zlar faol ishlaydi. Ish joyi noto'g'ri tashkil etilganda bu omillar surunkali charchoq va kasb kasalliklariga olib keladi.

Kompyuter bilan ishlashdagi xavfli va zararli omillar.

Dasturchining ish joyida quyidagi asosiy xavfli va zararli ishlab chiqarish omillari mavjud:

- **ko'rish analizatoriga yuklama** — ekranga uzoq tikilish, kam ko'z qirpish, yorqinlik kontrasti va aks etishlar ko'zning quruqlashishi, charchashi va ko'rishning pasayishiga ("kompyuter ko'rish sindromi") olib keladi;

- **statik yuklama va gipodinamiya** — noto‘g‘ri o‘tirish bo‘yin, yelka va bel og‘riqlarini, qon aylanishining sustlashishini keltirib chiqaradi;
- **qo‘l va bilakdagi takroriy harakatlar** — klaviatura va sichqoncha bilan uzoq ishlash bilak (tunnel) sindromi va paylarning yallig‘lanishiga sabab bo‘lishi mumkin;
- **noqulay mikroiklim** — kompyuterlar va monitorlar issiqlik chiqaradi, konditsionerlar esa havoni quritadi va sovuq havo oqimlarini hosil qiladi;
- **yoritilganlikning yetarli emasligi yoki ortiqchaligi**, ekrandagi yaltirash va aks etishlar;
- **shovqin** — sovutish ventilyatorlari, printerlar, ochiq ofisdagi suhbatlar diqqatni chalg‘itadi;
- **elektr toki urish xavfi** — shikastlangan kabellar, yerga ulanmagan jihozlar va ortiqcha yuklangan uzaytirgichlar;
- **psixosotsional zo‘riqish** — muddatlar bosimi, uzoq vaqt diqqatni jamlash, tungi navbatchiliklar va tizim avariylarini bartaraf etish.

Ish joyini tashkil etishga qo‘yiladigan talablar O‘zbekiston Respublikasining Mehnat kodeksi va “Mehnatni muhofaza qilish to‘g‘risida”gi Qonuni, sanitariya qoidalari va normalari hamda ISO 9241-5 (ish joyini joylashtirish va gavda holatiga talablar), GOST 12.2.032-78 (o‘tirib bajariladigan ishlarda ish joyi), GOST 12.1.005-88 (ish zonasi havosi) va KMK 2.01.05-98 (tabiiy va sun‘iy yoritish) standartlari bilan belgilanadi.

Ish xonasiga qo‘yiladigan talablar.

Kompyuterli ish joylari joylashgan xonada bitta ish joyiga to‘g‘ri keladigan maydon kamida 6 m², hajm esa kamida 20 m³ bo‘lishi tavsiya etiladi. Xonada tabiiy va sun‘iy yoritish bo‘lishi lozim. Derazalar shimol yoki shimoli-sharq tomonga qaragani ma’qul. Ular jalyuzi yoki pardalar bilan jihozlanadi.

Monitorlar derazaga nisbatan yon tomondan joylashtiriladi: yorug‘lik chap tomondan tushishi kerak. Deraza ekran orqasida bo‘lsa, ko‘z yorqin fon va nisbatan qorong‘i ekran o‘rtasida doimo moslashishga majbur bo‘ladi. Deraza xodimning orqasida bo‘lsa, ekranda yaltirash hosil bo‘ladi. Bir monitoring ekrani bilan

ikkinchisining orqa tomoni orasidagi masofa kamida 2 m, yon tomonlari orasidagi masofa esa kamida 1,2 m bo'lishi kerak.

Xona devorlari va shifti yaltiramaydigan (matli) materiallar bilan pardoqlanadi. Yorug'likni qaytarish ko'effitsiyentlari shift uchun 0,7–0,8, devorlar uchun 0,5–0,6, pol uchun 0,3–0,5 bo'lishi tavsiya etiladi. Xona har kuni nam usulda tozalanadi va har bir ish soatidan so'ng shamollatiladi.

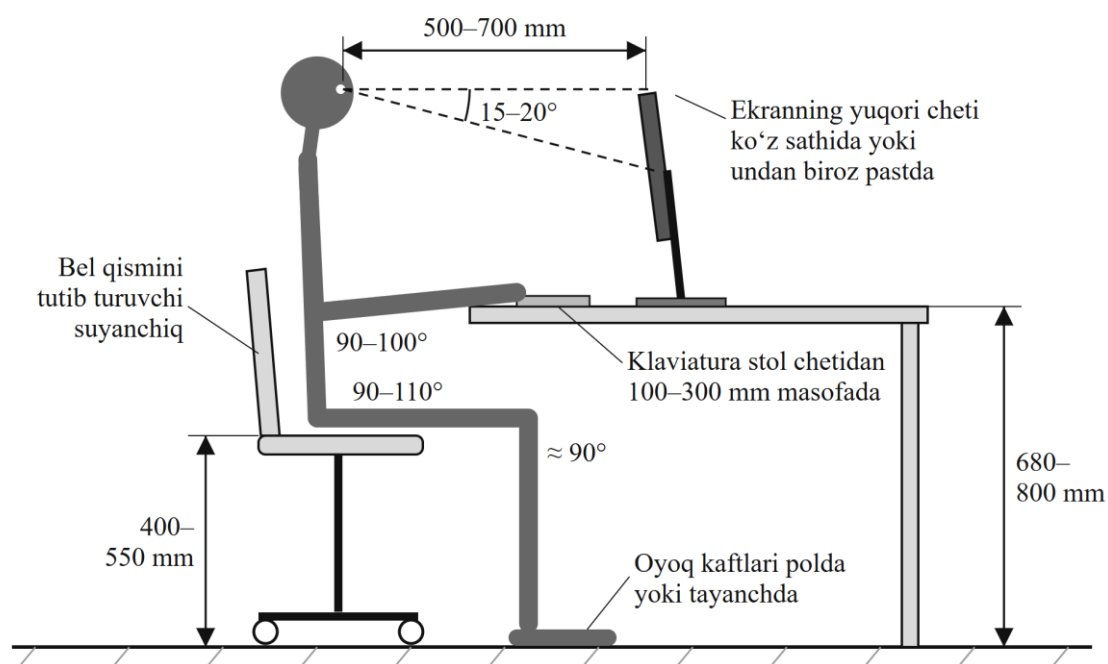
Ish joyi jihozlarining ergonomik parametrlari.

Dasturchining ish joyi ish stoli, sozlanadigan stul, monitor, klaviatura, sichqoncha va zarur bo'lsa oyoq tayanchidan iborat. Ularning tavsiya etiladigan o'lchamlari 4.1-jadvalda, xodimning to'g'ri gavda holati esa 4.1-rasmda keltirilgan.

4.1-jadval

Ish joyi elementlarining ergonomik parametrlari

Element	Parametr	Tavsiya etiladigan qiymat
Ish stoli	Ish yuzasining balandligi	680–800 mm (sozlanmaydigan stol uchun 725 mm)
	Ish yuzasining o'lchamlari	eni 800–1400 mm, chuqurligi 800–1000 mm
	Oyoqlar uchun bo'shliq	balandligi ≥ 600 mm, eni ≥ 500 mm, chuqurligi tizza sathida ≥ 450 mm, oyoq sathida ≥ 650 mm
Stul	O'rindiqlik balandligi	400–550 mm, sozlanadigan
	O'rindiqlik eni va chuqurligi	kamida 400 mm, old qirrasi yumaloqlangan
	Suyanchiq	balandligi 300 ± 20 mm, eni ≥ 380 mm, qiyalik burchagi $0 \dots 30^\circ$ oralig'ida sozlanadi
	Qo'l tayanchlari	uzunligi ≥ 250 mm, eni 50–70 mm, balandligi sozlanadi
Monitor	Ko'zdan ekrangacha masofa	500–700 mm
	Joylashuv balandligi	ekranning yuqori cheti ko'z sathida yoki undan pastroq, nigoh ekran markaziga $15\text{--}20^\circ$ pastga yo'naladi
Klaviatura	Stol chetidan masofa	100–300 mm, bilak uchun tayanch joy qoldiriladi
Oyoq tayanchi	O'lchamlari	eni ≥ 300 mm, chuqurligi ≥ 400 mm, qiyaligi $0 \dots 20^\circ$



4.1-rasm. Kompyuterda ishlovchi xodimning to‘g‘ri gavda holati va ish joyining asosiy o‘lchamlari

To‘g‘ri gavda holatining asosiy qoidasi “to‘g‘ri burchaklar” qoidasi deb ataladi: tirsaklar 90–100°, son-chanoq bo‘g‘imlari 90–110° va tizzalar taxminan 90° burchak ostida bukiladi. Oyoq kaftlari polga yoki oyoq tayanchiga to‘liq tiraladi. Bel suyanchiqqa tayanadi, yelkalar bo‘shashgan holatda bo‘ladi. Bilaklar klaviatura ustida to‘g‘ri tutiladi, yuqoriga yoki yon tomonga bukilmaydi. Sichqoncha klaviatura bilan bir sathda va tanaga yaqin joylashtiriladi.

Dasturchilar ko‘pincha ikki monitor bilan ishlaydi. Bunda asosiy monitor to‘g‘ridan qarshida, ikkinchisi esa yonida, ko‘zdan bir xil masofada va xodimga qaratib joylashtiriladi. Ikkala monitor teng ishlatilsa, ularning tutashgan joyi xodim ro‘parasida bo‘ladi. Noutbuk bilan uzoq ishlaganda uning ekrani ko‘z sathiga maxsus tagliklar bilan ko‘tariladi, alohida klaviatura va sichqoncha ulanadi. Aks holda boshni doimo pastga egib o‘tirishga to‘g‘ri keladi.

Ish joyi faqat jismoniy jihozlardan iborat emas. Dasturiy interfeys ham ergonomik talablarga javob berishi kerak (ISO 9241-110). Ishlab chiqilgan “Milliy chat” interfeysida matn va fon o‘rtasida yetarli kontrast, qorong‘i mavzu, tizim shrift o‘lchamini hisobga oladigan maketlar va mobil qurilmalarga moslashuv ko‘zda

tutilgani foydalanuvchilarning ko‘z charchog‘ini kamaytirishga xizmat qiladi. Dasturchining o‘z ish vositalarida (kod muharriri, terminal) ham qorong‘i mavzu, katta shrift va yaxshi o‘qiladigan monoshirinli shriftlardan foydalanish tavsiya etiladi.

Mikroiqlim, yoritish va shovqin.

Dasturchi mehnati energiya sarfi bo‘yicha Ia toifadagi yengil ishlarga kiradi (energiya sarfi 139 W gacha). Bunday ishlar uchun ish zonasi mikroiqlimining maqbul ko‘rsatkichlari, yoritilganlik va shovqin me‘yorlari 4.2-jadvalda keltirilgan.

4.2-jadval

Dasturchi ish joyi uchun mikroiqlim, yoritilganlik va shovqin me‘yorlari

Ko‘rsatkich	Yilning sovuq davri	Yilning iliq davri
Havo harorati, °C	22–24	23–25
Nisbiy namlik, %	40–60	40–60
Havo harakatining tezligi, m/s	≤ 0,1	≤ 0,1
Stol yuzasining yoritilganligi, lk	300–500	300–500
Ekran yuzasining yoritilganligi, lk	≤ 300	≤ 300
Yoritilganlik pulsatsiyasi ko‘effitsiyenti, %	≤ 5	≤ 5
Shovqin darajasi, dBA	≤ 50	≤ 50

Umumiy yoritish uchun diffuzorli (sochuvchi) LED yoritgichlardan foydalaniladi. Ular yorug‘likni bir tekis taqsimlaydi va miltillamaydi. Rang harorati 4000–5000 K bo‘lgan neytral oq yorug‘lik ko‘zni kamroq charchatadi. Yoritgichlar monitorlar bilan parallel qatorlarda joylashtiriladi, ularning yorug‘ligi ekranda aks etmasligi kerak. Zarur bo‘lsa, hujjatlar bilan ishlash uchun yaltiramaydigan mahalliy yoritgich qo‘shiladi.

Shovqinni kamaytirish uchun printerlar va boshqa shovqinli jihozlar alohida joyga chiqariladi, ochiq ofislarda tovush yutuvchi panellar va to‘siqlar o‘rnatiladi, qo‘ng‘iroqlar va muhokamalar uchun alohida xonalar ajratiladi. Server uskunalari ish xonalarida emas, alohida server xonasida joylashtiriladi.

Ish xonasining sun'iy yoritilishini hisoblash.

Misol sifatida dasturchilar bo'limi uchun o'lchamlari $A \times B \times H = 6 \times 6 \times 3$ m bo'lgan xonani ko'rib chiqamiz. Xonaning maydoni $S = 36 \text{ m}^2$, hajmi $V = 108 \text{ m}^3$. Bitta ish joyiga kamida 6 m^2 maydon va 20 m^3 hajm to'g'ri kelishi kerakligidan xonaga joylashtirish mumkin bo'lgan ish joylari soni:

$$n_{\max} = \min(S / 6; V / 20) = \min(36 / 6; 108 / 20) = \min(6; 5,4) = 5 \quad (4.1)$$

Demak, xonada 5 ta ish joyi tashkil etiladi. Umumiy sun'iy yoritish yorug'lik oqimidan foydalanish koeffitsiyenti usuli bilan hisoblanadi. Avval xona indeksi aniqlanadi:

$$i = A \cdot B / (h \cdot (A + B)), \quad (4.2)$$

bu yerda h — yoritgichlarning ish yuzasidan balandligi. Yoritgichlar shiftga o'rnatiladi, ish yuzasi poldan 0,8 m balandlikda joylashgan: $h = 3,0 - 0,8 = 2,2$ m. U holda $i = 36 / (2,2 \cdot 12) \approx 1,36$. Shift, devor va polning yorug'lik qaytarish koeffitsiyentlari mos ravishda 70, 50 va 30 % bo'lganda LED panellar uchun bu indeksga yorug'lik oqimidan foydalanish koeffitsiyenti $\eta \approx 0,54$ to'g'ri keladi. Kerakli yoritgichlar soni quyidagi formula bilan topiladi:

$$N = E_n \cdot S \cdot K_z \cdot z / (\Phi \cdot \eta), \quad (4.3)$$

bu yerda $E_n = 400 \text{ lk}$ — me'yoriy yoritilganlik; $K_z = 1,4$ — zaxira koeffitsiyenti (yoritgichlarning ifloslanishi va eskirishini hisobga oladi); $z = 1,1$ — yoritilganlikning notekislik koeffitsiyenti; $\Phi = 3600 \text{ lm}$ — bitta yoritgichning yorug'lik oqimi ($600 \times 600 \text{ mm}$ o'lchamli, 36 W quvvatli LED panel). Hisoblash natijasi:

$$N = 400 \cdot 36 \cdot 1,4 \cdot 1,1 / (3600 \cdot 0,54) = 22\,176 / 1944 \approx 11,4 \quad (4.4)$$

Yoritgichlar soni butun songa yaxlitlanadi va simmetrik joylashuv uchun $N = 12$ qabul qilinadi (4 ta ustun $\times$ 3 ta qator). Haqiqiy o'rtacha yoritilganlik quyidagiga teng:

$$E_h = N \cdot \Phi \cdot \eta / (S \cdot K_z \cdot z) = 12 \cdot 3600 \cdot 0,54 / (36 \cdot 1,4 \cdot 1,1) \approx 421 \text{ lk} \quad (4.5)$$

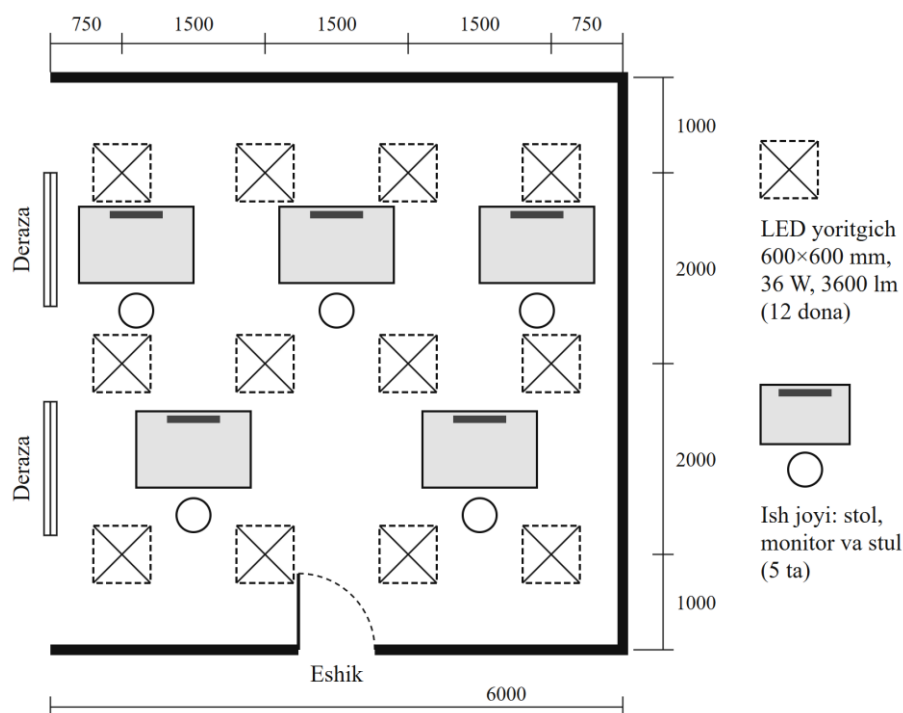
Haqiqiy yoritilganlik me'yordan 5 % ga yuqori. Bu ruxsat etilgan $-10 \dots +20$ % oralig'ida. Yoritish tizimining umumiy quvvati $12 \cdot 36 = 432 \text{ W}$, solishtirma

quvvati 12 W/m². Hisoblash natijalari 4.3-jadvalda umumlashtirilgan, yoritgichlar va ish joylarining joylashuvi 4.2-rasmda ko‘rsatilgan.

4.3-jadval

Ish xonasining sun‘iy yoritilishini hisoblash natijalari

Parametr	Belgilanishi	Qiymati
Xona o‘lchamlari	$A \times B \times H$	6 × 6 × 3 m
Yoritgichlarning ish yuzasidan balandligi	h	2,2 m
Xona indeksi	i	1,36
Yorug‘lik oqimidan foydalanish koeffitsiyenti	η	0,54
Me‘yoriy yoritilganlik	E_n	400 lk
Zaxira va notekislik koeffitsiyentlari	K_z, z	1,4; 1,1
Bitta yoritgichning yorug‘lik oqimi	Φ	3600 lm
Yoritgichlar soni	N	12 ta
Haqiqiy yoritilganlik	E_h	421 lk
Umumiy / solishtirma quvvat	P, p	432 W; 12 W/m ²



4.2-rasm. Dasturchilar xonasida yoritgichlar va ish joylarining joylashuv sxemasi

Xonani shamollatish uchun zarur havo miqdori bitta xodimga to‘g‘ri keladigan hajmga bog‘liq. Bu hajm 20–40 m³ bo‘lganda (bizda $108 / 5 = 21,6$ m³) har bir xodimga soatiga kamida 30 m³ toza havo berilishi kerak:

$$L = n \cdot L_1 = 5 \cdot 30 = 150 \text{ m}^3/\text{soat}; \quad K = L / V = 150 / 108 \approx 1,4 \text{ soat}^{-1} \quad (4.6)$$

Demak, xonada havo soatiga kamida 1,4 marta almashishi kerak. Buni tabiiy shamollatish yetarli bo‘lmaganda kichik quvvatli so‘ruvchi-beruvchi ventilyatsiya yoki rekuperatorli konditsioner ta‘minlaydi.

Mehnat va dam olish rejimi.

Kompyuter bilan ishlashda uzluksiz ish vaqti 2 soatdan oshmasligi kerak. Har 45–60 daqiqada 5–10 daqiqalik yoki har 2 soatda 15 daqiqalik tanaffus qilish tavsiya etiladi. Tanaffus vaqtida ekrandan uzoqlashish, yurish va yengil jismoniy mashqlar bajarish lozim. Dasturchilar orasida keng tarqalgan “Pomodoro” usuli (25 daqiqa ish va 5 daqiqa dam olish) ham bu talabga mos keladi.

Ko‘z charchog‘ini kamaytirish uchun “20-20-20” qoidasi qo‘llaniladi: har 20 daqiqada 20 soniya davomida kamida 20 fut (6 m) uzoqlikdagi narsaga qarash. Ko‘z uchun mashqlar (ko‘zni yumib-ochish, nigohni uzoq va yaqin nuqtalarga ko‘chirish) ham foydali. Bo‘yin, yelka, bel va bilaklar uchun qisqa mashqlar kun davomida bir necha marta bajariladi.

Ish beruvchi kompyuterda ishlovchi xodimlarni dastlabki va davriy tibbiy ko‘riklardan o‘tkazishi, ularga mehnatni muhofaza qilish bo‘yicha yo‘l-yo‘riq berishi kerak. Masofadan (uydan) ishlaganda ham xuddi shu talablar amal qiladi. Shu sababli masofaviy ishlaydigan dasturchilarga ergonomik stul va monitor bilan ta‘minlash yoki ish joyini o‘zi tashkil etishi uchun tavsiyalar berish maqsadga muvofiq.

Elektr xavfsizligi.

Ish joyidagi barcha jihozlar 220 V kuchlanishli tarmoqqa ulanadi. Shu sababli rozetkalar yerga ulash kontakti bilan jihozlanadi, kabellar maxsus kanallarda yoki stol ostidagi tokchalarda yotqiziladi va oyoq ostida qolmaydi. Uzaytirgichlarni bir-biriga ketma-ket ulash, shikastlangan kabel va vilkalardan foydalanish, jihozlar korpusini ochish taqiqlanadi. Server va tarmoq uskunalari uzluksiz quvvat manbalari (UPS) orqali ulanadi. Elektr jihozlarini ta‘mirlash faqat tegishli ruxsatga ega mutaxassis tomonidan bajariladi.

4.2. Evakuatsiya tadbirlarini rejalashtirish hamda avariya va yong'inlar sodir bo'lganda xodimlarning harakatlari

Favqulodda vaziyat deganda odamlar qurbon bo'lishiga, ularning sog'lig'iga yoki atrof-muhitga zarar yetishiga, katta moddiy yo'qotishlarga va hayot faoliyati sharoitlarining buzilishiga olib kelgan yoki olib kelishi mumkin bo'lgan avariya, halokat, tabiiy ofat yoki boshqa hodisa natijasida muayyan hududda yuzaga kelgan holat tushuniladi. Kelib chiqishiga ko'ra favqulodda vaziyatlar texnogen, tabiiy va ekologik turlarga bo'linadi.

IT-kompaniya ofisi uchun eng ehtimoliy favqulodda vaziyatlar yong'in, zilzila (Toshkent shahri yuqori seysmik faollik zonasida joylashgan), elektr ta'minotidagi avariya, isitish yoki suv quvurlarining yorilishi va gaz sizishidir. Bunday vaziyatlarda xodimlar hayotini saqlashning asosiy usuli ularni o'z vaqtida va tartibli evakuatsiya qilishdir.

Bu sohadagi asosiy talablar "Favqulodda vaziyatlarda aholini va hududlarni muhofaza qilish to'g'risida"gi, "Fuqaro muhofazasi to'g'risida"gi va "Yong'in xavfsizligi to'g'risida"gi qonunlar hamda GOST 12.1.004-91 "Yong'in xavfsizligi. Umumiy talablar" standarti bilan belgilanadi.

IT-ofisda yong'in chiqish sabablari va o't o'chirish vositalari.

Ofis binolarida yong'inlarning aksariyati elektr tarmog'i va jihozlar bilan bog'liq. Asosiy sabablar:

- elektr tarmog'ining ortiqcha yuklanishi: bitta rozetkaga ko'p jihoz ulash, uzaytirgichlarni ketma-ket ulash;
- eskirgan yoki shikastlangan izolyatsiya tufayli qisqa tutashuv, rozetka va vilkalardagi bo'sh kontaktlar;
- noutbuk va powerbanklarning litiy-ion akkumulyatorlari shikastlanishi yoki qizib ketishi, sifatsiz zaryadlovchi qurilmalar;
- UPS akkumulyatorlarining nosozligi, server xonasida sovutish tizimining ishdan chiqishi;
- qarovsiz qoldirilgan isitish asboblari, elektr choynaklar va boshqa maishiy jihozlar;

- chekish qoidalarini buzish va ochiq olovdan ehtiyotsiz foydalanish.

Ofisdagi yonuvchi materiallar (qog'oz, mebel, plastik korpuslar, kabel izolyatsiyasi) yonganda ko'p miqdorda zaharli tutun ajraladi. Yong'inlarda halok bo'lganlarning aksariyati olovdan emas, aynan tutun va is gazidan zaharlanishdan jabrlanadi. Shu sababli tutunni erta aniqlash va odamlarni tezkor evakuatsiya qilish hal qiluvchi ahamiyatga ega.

Bino avtomatik yong'in signalizatsiyasi (tutun va issiqlik datchiklari), odamlarni xabardor qilish va evakuatsiyani boshqarish tizimi hamda birlamchi o't o'chirish vositalari bilan jihozlanadi. Server xonasida suv bilan o'chirish uskunalarga zarar yetkazgani uchun avtomatik gazli o't o'chirish tizimi o'rnatiladi. Yong'in sinflari va ularni o'chirish vositalari 4.4-jadvalda keltirilgan.

4.4-jadval

Yong'in sinflari va o't o'chirish vositalari

Sinf	Yonuvchi material	Qo'llaniladigan vositalar
A	Qattiq materiallar: qog'oz, yog'och, mato, plastmassa	suv, ko'pikli va kukunli (ABCE) o't o'chirgichlar, ichki yong'in krani
B	Yonuvchi suyuqliklar: erituvchilar, bo'yoqlar, yoqilg'i	ko'pikli, kukunli va karbonat angidridli o't o'chirgichlar
C	Yonuvchi gazlar: tabiiy gaz, propan	kukunli o't o'chirgichlar; avvalo gaz berilishi to'xtatiladi
D	Metallar: magniy, natriy, alyuminiy kukuni	maxsus kukunli o't o'chirgichlar
E	Kuchlanish ostidagi elektr jihozlari	karbonat angidridli (OU) va kukunli o't o'chirgichlar; suv va ko'pik ishlatilmaydi

Kompyuterlar o'rnatilgan xonalarda asosan karbonat angidridli o't o'chirgichlar (OU-3, OU-5) ishlatiladi. Ular elektr o'tkazmaydi va uskunalarda qoldiq qoldirmaydi. Bunday o't o'chirgich bilan ishlaganda uning karnayini qo'l bilan ushlab bo'lmaydi: gaz kengayganda karnay -70°C gacha soviydi va terini sovuq urishi mumkin. O't o'chirgichlar ko'rinadigan joylarda, poldan 1,5 m dan baland bo'lmagan joyda osiladi. Yong'in chiqishi mumkin bo'lgan joydan eng yaqin o't o'chirgichgacha masofa 20 m dan oshmasligi kerak.

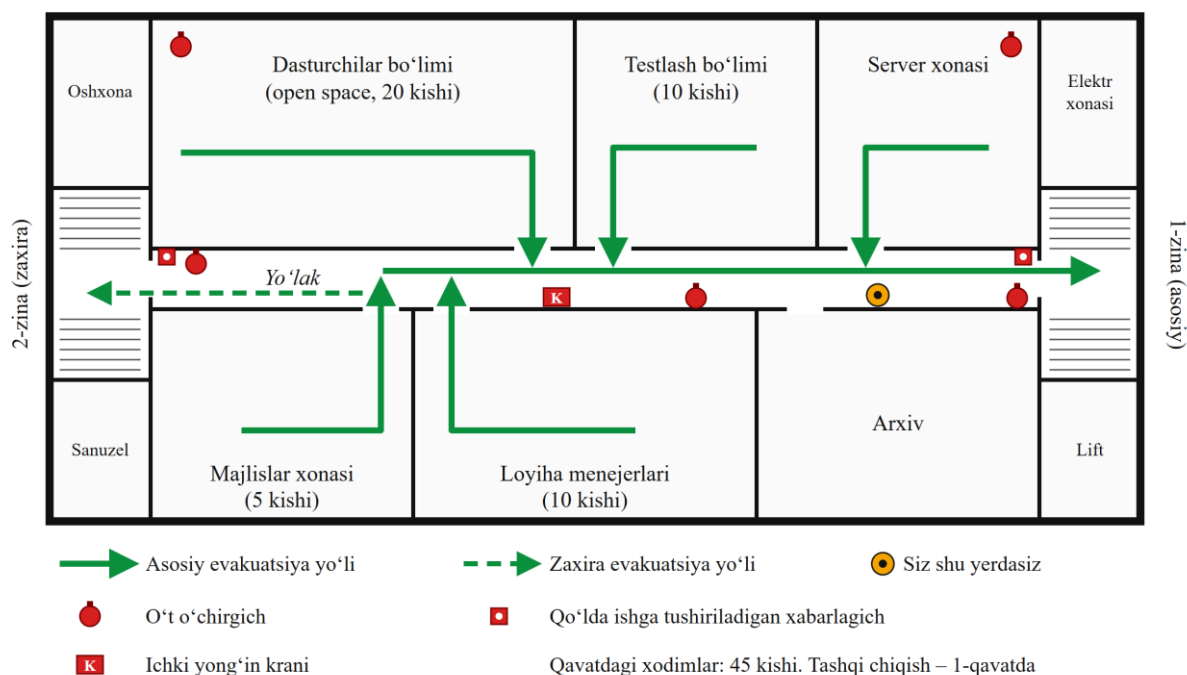
Evakuatsiya tadbirlarini rejalashtirish.

Evakuatsiya — odamlarni xavfli omillar ta'sir qila boshlashidan oldin binodan yoki uning qismidan xavfsiz hududga uyushgan holda chiqarish jarayoni. Evakuatsiya yo'li — evakuatsiya chiqishiga olib boradigan, yong'in xavfli omillaridan himoyalangan yo'l. Evakuatsiya chiqishi esa to'g'ridan-to'g'ri tashqariga, xavfsiz zinapoyaga yoki qo'shni yong'inga qarshi bo'linmaga olib boradigan chiqishdir.

Evakuatsiyani rejalashtirish tashkiliy va texnik tadbirlardan iborat. Texnik tadbirlarga quyidagilar kiradi:

- har bir qavatda kamida ikkita, bir-biridan uzoqda joylashgan evakuatsiya chiqishi;
- evakuatsiya yo'llari va eshiklarning me'yoriy kengligi (yo'laklar kamida 1–1,2 m, eshiklar kamida 0,8 m); eshiklar chiqish yo'nalishida ochiladi va ichkaridan kalitsiz ochiladi;
- yo'laklar va zinapoyalarni mebel, jihozlar va qutilar bilan to'sib qo'yishni taqiqlash;
- avariya (evakuatsiya) yoritishi va ISO 7010 standartiga mos yashil rangli “Favqulodda chiqish” belgilari;
- yong'in signalizatsiyasi, odamlarni ovozli xabardor qilish tizimi va qo'lda ishga tushiriladigan xabarlagichlar;
- har bir qavatda evakuatsiya reja-sxemalari.

Evakuatsiya reja-sxemasi grafik va matnli qismlardan iborat. Grafik qismda qavatning rejasi, “Siz shu yerdasiz” belgisi, asosiy (yashil yaxlit strelkalar) va zaxira (yashil punktir strelkalar) evakuatsiya yo'llari, evakuatsiya chiqishlari, o't o'chirgichlar, ichki yong'in kranlari va xabarlagichlarning joylashuvi ko'rsatiladi. Matnli qismda xodimlarning harakatlar tartibi va mas'ul shaxslar ro'yxati beriladi. Reja-sxemalar ko'zga yaxshi ko'rinadigan joylarda, zinapoyalar va chiqishlar yonida, qorong'ida ko'rinadigan (fotolyuminessent) materialdan tayyorlanib osiladi. Namunaviy IT-kompaniya ofisining uchinchi qavati uchun evakuatsiya reja-sxemasi 4.3-rasmda keltirilgan.



4.3-rasm. IT-kompaniya ofisi 3-qavatining evakuatsiya reja-sxemasi (namuna)

Tashkiliy tadbirlar quyidagilarni o'z ichiga oladi:

- tashkilot rahbarining buyrug'i bilan yong'in xavfsizligi uchun mas'ul shaxs, har bir qavat bo'yicha evakuatsiya mas'ullari va birinchi yordam ko'rsatuvchi xodimlar tayinlanadi;
- binodan tashqarida xavfsiz masofada xodimlar yig'iladigan joy belgilanadi va xodimlar ro'yxatlari tayyorlanadi;
- barcha xodimlar ishga qabul qilinganda kirish yo'riqnomasidan, keyinchalik esa davriy yo'riqnomadan o'tkaziladi, yong'in-texnik minimum bo'yicha o'qitiladi;
- evakuatsiya bo'yicha amaliy mashg'ulotlar muntazam, odatda yarim yilda kamida bir marta o'tkaziladi;
- o't o'chirish vositalari, signalizatsiya va avariya yoritishining sozligi belgilangan muddatlarda tekshiriladi;
- yangi xodimlar va mehmonlar evakuatsiya yo'llari va yig'ilish joyi bilan tanishtiriladi.

Evakuatsiya vaqtini hisoblash.

Evakuatsiya yo'llarining yetarliligini baholash uchun hisobiy evakuatsiya vaqti aniqlanadi. Hisoblash GOST 12.1.004-91 standartida keltirilgan odamlar oqimining harakat parametrlari asosida soddalashtirilgan usulda bajariladi. Evakuatsiya yo'li o'lchamlari va odamlar soni bir xil bo'lgan uchastkalarga ajratiladi. Har bir uchastkadagi odamlar oqimining zichligi quyidagicha aniqlanadi:

$$D_i = N_i \cdot f / (l_i \cdot \delta_i), \quad (4.7)$$

bu yerda N_i — uchastkadagi odamlar soni; $f = 0,1 \text{ m}^2$ — yozgi kiyimdagi katta yoshli odamning gorizontal proyeksiya maydoni; l_i va δ_i — uchastkaning uzunligi va kengligi, m. Zichlikka qarab 4.5-jadvaldan harakat tezligi v_i va oqim intensivligi q_i olinadi. Uchastkadan o'tish vaqti va umumiy hisobiy evakuatsiya vaqti:

$$t_i = l_i / v_i; \quad t_h = t_1 + t_2 + \dots + t_n \quad (4.8)$$

Eshiklar va yo'l torayadigan joylarda tirbandlik bor-yo'qligi oqim intensivligi orqali tekshiriladi:

$$q_e = q_i \cdot \delta_i / \delta_e \leq q_{\max}, \quad (4.9)$$

bu yerda δ_e — eshik kengligi; $q_{\max} = 19,6 \text{ m/min}$ — eshikdan o'tishdagi oqim intensivligining eng katta qiymati. Bu shart bajarilsa, eshik oldida odamlar to'planib qolmaydi.

4.5-jadval

Odamlar oqimining harakat parametrlari (GOST 12.1.004-91 bo'yicha, parcha)

Oqim zichligi $D, \text{m}^2/\text{m}^2$	Gorizontal yo'l: $v, \text{m/min}$	Gorizontal yo'l: $q, \text{m/min}$	Eshik: $q, \text{m/min}$	Zinadan pastga: $v, \text{m/min}$	Zinadan pastga: $q, \text{m/min}$
0,01	100	1,0	1,0	100	1,0
0,05	100	5,0	5,0	100	5,0
0,10	80	8,0	8,7	95	9,5
0,20	60	12,0	13,4	68	13,6
0,30	47	14,1	16,5	52	15,6
0,40	40	16,0	18,4	40	16,0
0,50	33	16,5	19,6	31	15,5

4.3-rasmdagi qavat uchun hisoblashni bajaramiz. Qavatda 45 nafar xodim ishlaydi. Eng og‘ir holat ko‘rib chiqiladi: 2-zina yong‘in tufayli to‘sib qo‘yilgan va barcha xodimlar asosiy (1-) zina orqali chiqadi. Hisobiy yo‘l dasturchilar bo‘limining eng uzoq ish joyidan boshlanadi va to‘rtta uchastkadan iborat:

1. dasturchilar bo‘limi ichidagi o‘tish joyi: $l_1 = 12$ m, $\delta_1 = 1,5$ m, $N_1 = 20$ kishi;
2. yo‘lak: bo‘lim eshigidan zinagacha $l_2 = 17$ m, $\delta_2 = 2,0$ m, $N_2 = 45$ kishi;
3. 3-qavatdan 1-qavatgacha zina marshlari va maydonchalari: $l_3 = 2 \cdot 10 = 20$ m, $\delta_3 = 1,35$ m, $N_3 = 45$ kishi;
4. 1-qavat vestibulidan tashqi chiqishgacha: $l_4 = 15$ m, $\delta_4 = 3,0$ m, $N_4 = 45$ kishi.

Masalan, birinchi uchastka uchun $D_1 = 20 \cdot 0,1 / (12 \cdot 1,5) \approx 0,11$. 4.5-jadvaldagi qiymatlar orasida chiziqli interpolyatsiya qilib $v_1 \approx 78$ m/min va $q_1 \approx 8,4$ m/min topiladi, $t_1 = 12 / 78 \approx 0,15$ min. Qolgan uchastkalar ham xuddi shunday hisoblanadi. Natijalar 4.6-jadvalda keltirilgan.

4.6-jadval

Evakuatsiya vaqtini hisoblash natijalari

Uchastka	l , m	δ , m	N , kishi	D , m ² /m ²	v , m/min	t , min
1. Bo‘lim ichidagi o‘tish joyi	12	1,5	20	0,11	78	0,15
2. Yo‘lak	17	2,0	45	0,13	74	0,23
3. Zina (pastga)	20	1,35	45	0,17	77	0,26
4. Vestibul	15	3,0	45	0,10	80	0,19
Jami hisobiy evakuatsiya vaqti t_h						0,83

Eshiklarda tirbandlik yo‘qligini (4.9) formula bilan tekshiramiz. Bo‘lim eshigi ($\delta = 1,2$ m): $q = 8,4 \cdot 1,5 / 1,2 \approx 10,5$ m/min. Zinaga kirish eshigi ($\delta = 1,2$ m): $q = 9,3 \cdot 2,0 / 1,2 \approx 15,5$ m/min. Tashqi eshik ($\delta = 1,5$ m): $q = 8,0 \cdot 3,0 / 1,5 = 16,0$ m/min. Barcha qiymatlar $q_{\max} = 19,6$ m/min dan kichik, ya‘ni odamlar oqimi to‘xtovsiz harakatlanadi.

Demak, hisobiy evakuatsiya vaqti $t_h \approx 0,83$ min (taxminan 50 soniya). Xabardor qilish tizimi ishga tushgandan so‘ng evakuatsiya boshlanishidagi

kechikishni taxminan 1 daqiqa deb olsak, qavatdan to'liq chiqish uchun taxminan 2 daqiqa kerak bo'ladi. Bu vaqt yong'in xavfli omillari (tutun, harorat, kislorod kamayishi) evakuatsiya yo'llarida kritik qiymatga yetish vaqtidan kichik bo'lishi shart. Bu vaqt binoning yong'in xavfini baholash hujjatlarida aniqlanadi. Hisob ko'rsatadiki, eng tor joy zina hisoblanadi. Shu sababli zinaga olib boradigan eshiklar doimo yopiq holatda (o'zi yopiladigan qurilma bilan), lekin qulflanmagan bo'lishi, zina maydonchalarida esa hech qanday buyum saqlanmasligi kerak.

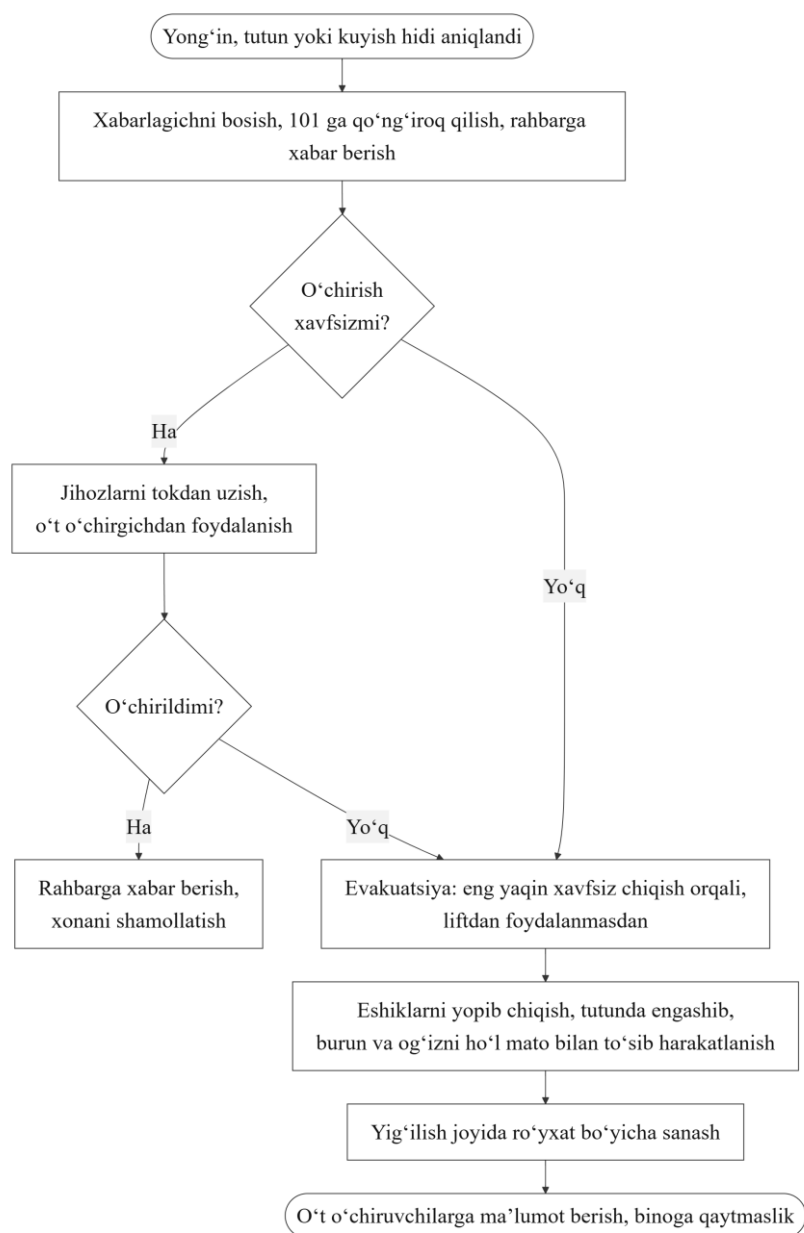
Yong'in sodir bo'lganda xodimlarning harakatlari.

Yong'in chiqqanda vahimaga tushmaslik va oldindan o'rgatilgan tartibda harakat qilish muhim. Xodimlarning harakatlar algoritmi 4.4-rasmda ko'rsatilgan va quyidagi qoidalarga asoslanadi:

1. Yong'in, tutun yoki kuyish hidini sezgan xodim darhol qo'lda ishga tushiriladigan xabarlagichni bosadi va 101 raqamiga (yoki yagona 112 raqamiga) qo'ng'iroq qiladi. Bunda binoning manzili, qavati, nima yonayotgani, odamlar bor-yo'qligi va o'z ismi aytiladi. So'ngra bo'lim rahbari va yong'in xavfsizligi uchun mas'ul shaxsga xabar beriladi.
2. Yong'in endi boshlangan bo'lsa va bu xavfsiz bo'lsa, elektr jihozlari tarmoqdan uziladi (xona yoki qavat avtomat o'chirgichi o'chiriladi) va birlamchi vositalar bilan o'chirishga harakat qilinadi. Kuchlanish ostidagi jihozlarni suv bilan o'chirish mumkin emas.
3. Yong'inni 1–2 daqiqada o'chirib bo'lmasa yoki tutun ko'paysa, darhol evakuatsiya boshlanadi. Evakuatsiya reja-sxemasida ko'rsatilgan eng yaqin xavfsiz chiqishdan foydalaniladi. Lift dan foydalanish taqiqlanadi, chunki u to'xtab qolishi va tutunli qavatda ochilishi mumkin.
4. Xonadan chiqqanda eshik yopiladi, lekin qulflanmaydi. Bu tutun va olovning tarqalishini sekinlashtiradi. Derazalarni ochish mumkin emas, chunki havo oqimi yonishni kuchaytiradi.
5. Tutunli yo'lakda engashib yoki kerak bo'lsa emaklab harakatlaniladi, chunki pastda havo tozaroq bo'ladi. Burun va og'iz ho'l mato bilan

to'siladi, yo'nalishni yo'qotmaslik uchun bir qo'l bilan devorni ushlab boriladi.

6. Qo'ldan kelsa, mehmonlarga va harakatlanishi cheklangan hamkasblarga yordam beriladi. Shaxsiy buyumlarni olish uchun ortga qaytilmaydi.
7. Yig'ilish joyida qavat mas'uli xodimlarni ro'yxat bo'yicha sanaydi. Yetib kelmagan xodimlar va ular bo'lishi mumkin bo'lgan joy haqida yetib kelgan o't o'chirish bo'linmasiga darhol xabar beriladi. O't o'chiruvchilar ruxsat bermaguncha binoga qaytilmaydi.



4.4-rasm. Yong'in sodir bo'lganda xodimlarning harakatlar algoritmi

Yong'in xavfsizligi uchun mas'ul shaxs o't o'chirish xizmati chaqirilganini tekshiradi, evakuatsiyani boshqaradi, ventilyatsiya va konditsionerlarni o'chirishni tashkil etadi (ular tutunni tarqatadi), o't o'chirish bo'linmasini kutib oladi va unga yong'in joyi, odamlar va server xonasidagi gazli o't o'chirish tizimi haqida ma'lumot beradi.

Agar kiyimga olov tushsa, yugurish mumkin emas, chunki havo oqimi olovni kuchaytiradi. Yerga yotib, dumalab yoki qalin mato bilan o'rab olovni o'chirish kerak. Kuyish jarohatida kuygan joy 10–20 daqiqa davomida salqin oqar suv ostida sovutiladi va toza bog'lam qo'yiladi. Tutundan zaharlangan odam toza havoga olib chiqiladi va 103 raqami orqali tez tibbiy yordam chaqiriladi.

Boshqa avariyalarda xodimlarning harakatlari.

- **Zilzila.** Silkinish paytida zinapoya va liftga yugurish xavfli. Mustahkam stol ostiga yashirinish yoki ichki kapital devor burchagida, eshik o'rnida turish, derazalar, shkaflar va osma jihozlardan uzoqlashish kerak. Silkinish tugagach, bino evakuatsiya yo'llari bo'ylab tark etiladi, yig'ilish joyi elektr uzatish liniyalari va binolardan uzoqda tanlanadi.
- **Server xonasida gazli o't o'chirish tizimining ishga tushishi.** Ogohlantiruvchi ovozli va yorug'lik signali berilganda xona darhol tark etiladi va eshik yopiladi. O't o'chiruvchi gaz xonadagi kislorod miqdorini kamaytiradi, shuning uchun xona shamollatilmaguncha unga kirish taqiqlanadi.
- **Elektr toki urishi.** Jabrlanuvchiga yalang qo'l bilan tegish mumkin emas. Avval tok manbai o'chiriladi yoki jabrlanuvchi quruq yog'och kabi dielektrik buyum yordamida tokdan ajratiladi. So'ngra 103 raqami chaqiriladi va zarur bo'lsa yurak-o'pka reanimatsiyasi boshlanadi.
- **Gaz sizishi.** Olov yoqish, elektr kalitlari va qurilmalarni yoqish yoki o'chirish mumkin emas, chunki uchqun portlashga sabab bo'lishi mumkin. Derazalar ochiladi, xonadagilar chiqariladi va 104 raqamiga xabar beriladi.

- **Suv quvurlarining yorilishi.** Suv bosishi mumkin bo'lgan xonalardagi elektr ta'minoti o'chiriladi, uskunalar balandroq joyga ko'chiriladi va xo'jalik xizmatiga xabar beriladi.

Favqulodda vaziyatlarda xodimlarni tezkor xabardor qilish uchun asosiy ovozli xabar berish tizimidan tashqari korporativ aloqa vositalaridan ham foydalanish mumkin. Masalan, ishlab chiqilgan "Milliy chat" tizimida tashkilotning barcha xodimlari a'zo bo'lgan guruh (200 nafargacha a'zo) yaratilib, unda faqat administratorlar xabar e'lon qiladi. Bunday guruh orqali evakuatsiya haqida xabar, yig'ilish joyi va keyingi ko'rsatmalar bir necha soniyada barcha xodimlarning telefon va kompyuterlariga yetkaziladi. Xodimlar esa xabarga javob yozib, xavfsiz joyga yetib kelganini tasdiqlaydi. Bu vosita yong'in signalizatsiyasi va ovozli xabar berish tizimining o'rnini bosmaydi, balki ularni to'ldiradi. U ayniqsa masofadan ishlayotgan va binoda bo'lmagan xodimlarni xabardor qilishda foydali. Tizim serverlari mamlakat ichida joylashgani favqulodda vaziyatlarda aloqaning mustaqilligini oshiradi.

4-bob bo'yicha xulosa

To'rtinchi bobda hayot faoliyati xavfsizligining dasturiy ta'minot ishlab chiquvchilar uchun muhim bo'lgan ikki masalasi ko'rib chiqildi. Birinchi bo'limda kompyuter bilan ishlashdagi xavfli va zararli omillar aniqlandi. Ish xonasi, ish stoli, stul va monitorga qo'yiladigan ergonomik talablar, mikroiklim, yoritish va shovqin me'yorlari hamda mehnat va dam olish rejimi keltirildi. 6×6 m o'lchamli dasturchilar xonasi uchun joylashtirish mumkin bo'lgan ish joylari soni (5 ta), sun'iy yoritish (12 ta 36 W li LED yoritgich, haqiqiy yoritilganlik 421 lk) va zarur havo almashinuvi ($150 \text{ m}^3/\text{soat}$) hisoblandi.

Ikkinchi bo'limda IT-ofisda yong'in chiqish sabablari, yong'in sinflari va o't o'chirish vositalari, evakuatsiyani rejalashtirishning texnik va tashkiliy tadbirlari ko'rib chiqildi hamda namunaviy qavat uchun evakuatsiya reja-sxemasi ishlab chiqildi. 45 kishilik qavat uchun hisobiy evakuatsiya vaqti taxminan 0,83 daqiqani tashkil etishi va eshiklarda tirbandlik yuzaga kelmasligi hisoblab ko'rsatildi.

Yong'in, zilzila va boshqa avariyalarda xodimlarning harakatlar tartibi berildi. Bundan tashqari, "Milliy chat" tizimidan favqulodda vaziyatlarda xodimlarni qo'shimcha xabardor qilish vositasi sifatida foydalanish imkoniyati ko'rsatildi.

XULOSA

Bitiruv malakaviy ishida foydalanuvchi ma'lumotlarini o'z serverida saqlaydigan, Telegramga o'xshash "Milliy chat" veb-messenjeri loyihalandi va ishlab chiqildi. Ishning kirish qismida qo'yilgan vazifalar quyidagicha hal etildi.

- Telegram, WhatsApp, Signal, Viber va WeChat messenjerlari funksional imkoniyatlari, xavfsizlik modeli va ma'lumotlarni saqlash joyi bo'yicha taqqoslandi. Ularning barchasi xorijiy yurisdiksiyada ekani va bu "Shaxsga doir ma'lumotlar to'g'risida"gi Qonunning lokalizatsiya talablari nuqtai nazaridan milliy yechim zarurligini asoslashi ko'rsatildi.
- Real vaqtda aloqa texnologiyalari (short va long polling, Server-Sent Events, WebSocket, SignalR) tahlil qilindi. WebSocket samaradorligini saqlagan holda zaxira transportlarga avtomatik o'tadigan SignalR tanlandi.
- Tizimning aktorlari, 15 ta funksional va 7 guruh nofunksional talablari aniqlandi.
- Server qismi Clean Architecture tamoyillari asosida Domain, Application, Infrastructure va API qatlamlariga ajratildi. PostgreSQL ma'lumotlar bazasi 10 ta sxemadagi 36 ta jadvaldan iborat qilib loyihalandi. Tizim ER, use-case va ketma-ketlik diagrammalari bilan modellashirildi.
- Parolsiz autentifikatsiya tizimi ishlab chiqildi. U elektron pochtaga yuboriladigan va PBKDF2 bilan xeshlanadigan bir martalik kod, Google hisobi, 15 daqiqalik JWT access token, HMAC xeshi saqlanadigan va har safar almashtiriladigan refresh token hamda qurilmalar bo'yicha sessiyalarni boshqarishni o'z ichiga oladi.
- .NET 8 platformasida 73 ta endpointli REST API va SignalR habi amalga oshirildi. Ular shaxsiy va guruh chatlari, xabarlarni yuborish, tahrirlash, o'chirish, javob berish va qidirish, rasm, video va fayllar almashish, "yozmoqda" va o'qilganlik belgilari, onlayn holat hamda 24 soatlik hikoyalarni ta'minlaydi. Guruhlar uchun egasi, administrator va a'zo rollariga asoslangan ruxsatlar tizimi, xizmat xabarlari va taklif havolalari yaratildi.

- Angular 22 freymvorkida signals asosidagi holat boshqaruvi, o‘z dizayn tizimi, yorug‘ va qorong‘i mavzular va mobil qurilmalarga moslashuvchan o‘zbek tilidagi interfeys ishlab chiqildi. Ilova qat’iy Content Security Policy bilan himoyalangan.
- O‘zbek tilining lotin va kirill yozuvlari o‘rtasida kontekstga bog‘liq qoidalarni hisobga oluvchi transliteratsiya algoritmi ishlab chiqildi. Har bir foydalanuvchi xabarlarini asl holida, lotin yoki kirill yozuvida ko‘radi, chat ichidagi qidiruv esa yozuvga bog‘liq emas. Algoritmning server (C#) va klient (TypeScript) nusxalari 80 ta juftlikdan iborat yagona test to‘plami bilan tekshiriladi.
- Tashkilot rejimi amalga oshirildi: e-pochta domeni tashkilot sifatida qabul qilinadi, uning tasdiqlangan a‘zolari “tuit.uz ✓” kabi belgi oladi va tashkilotning yopiq guruhiga avtomatik qo‘shiladi. Umumiy pochta xizmatlari foydalanuvchilari bu guruhlariga faqat admin qo‘shganda yoki 128 bitli taklif havolasi orqali kira oladi.
- Web Crypto API asosida uchdan-uchgacha shifrlangan maxfiy chatlar yaratildi: X25519 kalit kelishuvi, HKDF, HMAC asosidagi kalitlar zanjiri (oldinga maxfiylik) va AES-256-GCM shifrlash, kalit izini tekshirish, o‘z-o‘zini o‘chirish taymeri va shifrlangan fayllar. Serverda faqat ochiq kalitlar va yetkazilguncha saqlanadigan shifrlangan bloblar turishi ma’lumotlar bazasi darajasida ko‘rsatib berildi.
- Server ilovasi Docker konteyneri ko‘rinishida bulutli platformaga joylashtirildi. Tizimning to‘g‘riligi serverdagi 307 ta va klientdagi 229 ta unit test, API darajasidagi 43 ta integratsion tekshiruv va qo‘lda o‘tkazilgan sinov ssenariylari bilan tasdiqlandi.
- Hayot faoliyati xavfsizligi bo‘limida dasturchining ish joyiga qo‘yiladigan ergonomik talablar tizimlashtirildi, dasturchilar xonasi uchun ish joylari soni, sun‘iy yoritish va havo almashinuvi hisoblandi. IT-ofis uchun evakuatsiya reja-sxemasi ishlab chiqildi, hisobiy evakuatsiya vaqti

aniqlandi, yong'in va boshqa avariyalarda xodimlarning harakatlar tartibi belgilandi.

Ishning amaliy ahamiyati shundaki, "Milliy chat" to'liq ishlaydigan dasturiy mahsulot bo'lib, uni ta'lim muassasalari, tashkilotlar yoki davlat idoralari o'z serverlarida, jumladan respublika hududidagi data-markazlarda joylashtirib, ichki muloqot vositasi sifatida qo'llashi mumkin. Bunda foydalanuvchi ma'lumotlari ustidan to'liq nazorat mamlakat ichida qoladi. Tashkilot rejimi korporativ pochta orqali a'zolari avtomatik birlashtiradi, lotin–kirill transliteratsiyasi turli avlod foydalanuvchilarining bir-birini tushunishini osonlashtiradi, maxfiy chatlar esa eng nozik yozishmalarni serverning o'zidan ham himoya qiladi. Qatlamli arxitektura va oldindan loyihalangan ma'lumotlar bazasi sxemasi tizimni bosqichma-bosqich kengaytirish imkonini beradi.

Kelajakda tizimni quyidagi yo'nalishlarda rivojlantirish rejalashtirilgan: kanallar va botlar platformasi; WebRTC asosidagi ovozli va video qo'ng'iroqlar; OneID (id.egov.uz) tizimi orqali shaxsni davlat darajasida tasdiqlash; maxfiy chatlarni ko'p qurilmali modelga va to'liq Double Ratchet protokoliga o'tkazish; ikki bosqichli autentifikatsiya va foydalanuvchilarni bloklash; xabarlarga reaksiyalar, so'rovnomalar va stikerlar; push-bildirishnomalar; mobil ilovalar (Android, iOS); Redis va bir nechta server nusxasi yordamida gorizontol kengaytirish.

FOYDALANILGAN ADABIYOTLAR

Normativ-huquqiy hujjatlar

1. O‘zbekiston Respublikasining “Shaxsga doir ma’lumotlar to‘g‘risida”gi Qonuni. 2019-yil 2-iyul, O‘RQ-547-son.
2. O‘zbekiston Respublikasining “Axborotlashtirish to‘g‘risida”gi Qonuni. 2003-yil 11-dekabr, 560-II-son.
3. O‘zbekiston Respublikasining Mehnat kodeksi. 2022-yil 28-oktabr, O‘RQ-798-son.
4. O‘zbekiston Respublikasining “Mehnatni muhofaza qilish to‘g‘risida”gi Qonuni. 2016-yil 22-sentabr, O‘RQ-410-son.
5. O‘zbekiston Respublikasining “Yong‘in xavfsizligi to‘g‘risida”gi Qonuni. 2009-yil 30-sentabr, O‘RQ-226-son.
6. O‘zbekiston Respublikasining “Favqulodda vaziyatlarda aholini va hududlarni muhofaza qilish to‘g‘risida”gi Qonuni. 1999-yil 20-avgust, 824-I-son.
7. O‘zbekiston Respublikasi Prezidentining “«Raqamli O‘zbekiston – 2030» strategiyasini tasdiqlash va uni samarali amalga oshirish chora-tadbirlari to‘g‘risida”gi Farmoni. 2020-yil 5-oktabr, PF-6079-son.
8. O‘zbekiston Respublikasining “Lotin yozuviga asoslangan o‘zbek alifbosini joriy etish to‘g‘risida”gi Qonuni. 1993-yil 2-sentabr, 931-XII-son (1995-yil 6-maydagi 71-I-son Qonun tahririda).
9. O‘zbekiston Respublikasi Vazirlar Mahkamasining “O‘zbek tilining asosiy imlo qoidalarini tasdiqlash to‘g‘risida”gi qarori. 1995-yil 24-avgust, 339-son.

Asosiy adabiyotlar

10. Martin R. C. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 p.
11. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. — Prentice Hall, 2008. — 464 p.

- 12.Kleppmann M. Designing Data-Intensive Applications. — O'Reilly Media, 2017. — 616 p.
- 13.Fowler M. Patterns of Enterprise Application Architecture. — Addison-Wesley, 2002. — 560 p.
- 14.Evans E. Domain-Driven Design: Tackling Complexity in the Heart of Software. — Addison-Wesley, 2003. — 560 p.
- 15.Lock A. ASP.NET Core in Action. 3rd ed. — Manning Publications, 2023.
- 16.Smith J. P. Entity Framework Core in Action. 2nd ed. — Manning Publications, 2021.
- 17.Price M. J. C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals. 8th ed. — Packt Publishing, 2023.
- 18.Obe R. O., Hsu L. S. PostgreSQL: Up and Running. 3rd ed. — O'Reilly Media, 2017.
- 19.Stallings W. Cryptography and Network Security: Principles and Practice. 8th ed. — Pearson, 2022.
- 20.Fowler M. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3rd ed. — Addison-Wesley, 2003.
- 21.Yormatov G'.Yo., Isamuxamedov Yo.U. Hayot faoliyati xavfsizligi. — T.: Aloqachi, 2009.

Standartlar va texnik spetsifikatsiyalar

- 22.Fette I., Melnikov A. RFC 6455: The WebSocket Protocol. — IETF, 2011. — <https://www.rfc-editor.org/rfc/rfc6455>
- 23.Jones M., Bradley J., Sakimura N. RFC 7519: JSON Web Token (JWT). — IETF, 2015. — <https://www.rfc-editor.org/rfc/rfc7519>
- 24.Moriarty K., Kaliski B., Rusch A. RFC 8018: PKCS #5: Password-Based Cryptography Specification Version 2.1. — IETF, 2017.
- 25.Krawczyk H., Bellare M., Canetti R. RFC 2104: HMAC: Keyed-Hashing for Message Authentication. — IETF, 1997.

26. Langley A., Hamburg M., Turner S. RFC 7748: Elliptic Curves for Security (X25519, X448). — IRTF, 2016. — <https://www.rfc-editor.org/rfc/rfc7748>
27. Krawczyk H., Eronen P. RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF). — IETF, 2010.
28. Dworkin M. NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. — NIST, 2007.
29. NIST FIPS 180-4: Secure Hash Standard (SHS). — NIST, 2015.
30. W3C. Web Cryptography API. W3C Recommendation. — <https://www.w3.org/TR/WebCryptoAPI/>
31. W3C. Content Security Policy Level 3. — <https://www.w3.org/TR/CSP3/>
32. Perrin T., Marlinspike M. The Double Ratchet Algorithm. — Signal, 2016. — <https://signal.org/docs/specifications/doubleratchet/>
33. OWASP Foundation. OWASP Top 10 — 2021. — <https://owasp.org/Top10/>
34. ISO 9241-5:1998. Ergonomic requirements for office work with visual display terminals (VDTs) — Part 5: Workstation layout and postural requirements.
35. ISO 7010:2019. Graphical symbols — Safety colours and safety signs — Registered safety signs.
36. GOST 12.2.032-78. SSBT. Rabocheye mesto pri vypolnenii rabot sidya. Obshchiye ergonomicheskiye trebovaniya.
37. GOST 12.1.005-88. SSBT. Obshchiye sanitarno-gigiyenicheskiye trebovaniya k vozdukhу rabochey zony.
38. GOST 12.1.004-91. SSBT. Pozharnaya bezopasnost. Obshchiye trebovaniya.
39. KMK 2.01.05-98. Tabiiy va sun'iy yoritish. — T.: O'zbekiston Respublikasi Davarxitektqurilish qo'mitasi, 1998.

Internet manbalari

40. Microsoft. ASP.NET Core SignalR documentation. —
<https://learn.microsoft.com/aspnet/core/signalr/>
41. Microsoft. Entity Framework Core documentation. —
<https://learn.microsoft.com/ef/core/>
42. Microsoft. ASP.NET Core fundamentals. —
<https://learn.microsoft.com/aspnet/core/>
43. Angular. Official documentation. — <https://angular.dev>
44. The PostgreSQL Global Development Group. PostgreSQL Documentation. — <https://www.postgresql.org/docs/>
45. Docker Inc. Docker documentation: Multi-stage builds. —
<https://docs.docker.com/build/building/multi-stage/>
46. Telegram. MTProto Mobile Protocol. — <https://core.telegram.org/mtproto>
47. Signal Messenger. Technical documentation. — <https://signal.org/docs/>
48. FluentValidation documentation. — <https://docs.fluentvalidation.net>
49. xUnit.net documentation. — <https://xunit.net>
50. MDN Web Docs. SubtleCrypto (Web Crypto API). —
<https://developer.mozilla.org/docs/Web/API/SubtleCrypto>
51. Vitest documentation. — <https://vitest.dev>

ILOVALAR

1-ilova. SignalR habi (ChatHub)

Fayl: NationalChat/src/API/Hubs/ChatHub.cs

```
using System.Security.Claims;
using Application.Features.Authentication;
using Application.Features.Messages;
using Application.Features.Presence;
using Application.Features.Profiles;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.SignalR;

namespace API.Hubs;

[Authorize]
public sealed class ChatHub(
    IMessageRepository messageRepository,
    IChatRealtimeNotifier realtimeNotifier,
    IPresenceTracker presenceTracker,
    IAuthRepository authRepository,
    IProfileRepository profileRepository,
    TimeProvider timeProvider) : Hub
{
    private static readonly TimeSpan OfflineGracePeriod = TimeSpan.FromSeconds(8);

    public override async Task OnConnectedAsync()
    {
        var userId = GetCurrentUserId();
        await Groups.AddToGroupAsync(Context.ConnectionId,
            ChatHubGroups.User(userId));
        await Groups.AddToGroupAsync(Context.ConnectionId,
            ChatHubGroups.Session(GetCurrentSessionId()));

        if (presenceTracker.AddConnection(userId, Context.ConnectionId))
        {
            var relatedUserIds = await
                profileRepository.GetRelatedUserIdsAsync(userId, Context.ConnectionAborted);
            await realtimeNotifier.UserPresenceChangedAsync(userId, true, null,
                relatedUserIds, Context.ConnectionAborted);
        }

        await base.OnConnectedAsync();
    }

    public override async Task OnDisconnectedAsync(Exception? exception)
    {
        var userId = GetCurrentUserId();
        var sessionId = GetCurrentSessionId();
        await base.OnDisconnectedAsync(exception);

        if (!presenceTracker.RemoveConnection(userId, Context.ConnectionId))
        {
            return;
        }

        // Grace period absorbs quick reconnects (page refresh, brief network drop)
        // without flashing "offline".
        await Task.Delay(OfflineGracePeriod, CancellationToken.None);
        if (presenceTracker.IsOnline(userId))
        {
            return;
        }
    }
}
```

```

    }

    var lastSeenAt = timeProvider.GetUtcNow().UtcDateTime;
    var session = await authRepository.FindSessionAsync(userId, sessionId,
CancellationToken.None);
    if (session is not null)
    {
        session.LastActiveAt = lastSeenAt;
        await authRepository.SaveChangesAsync(CancellationToken.None);
    }

    var recipientUserIds = await profileRepository.GetRelatedUserIdsAsync(userId,
CancellationToken.None);
    await realtimeNotifier.UserPresenceChangedAsync(userId, false, lastSeenAt,
recipientUserIds, CancellationToken.None);
}

public async Task JoinChat(int chatId)
{
    await EnsureChatMembershipAsync(chatId);
    await Groups.AddToGroupAsync(Context.ConnectionId,
ChatHubGroups.Chat(chatId));
}

public Task LeaveChat(int chatId) =>
    Groups.RemoveFromGroupAsync(Context.ConnectionId, ChatHubGroups.Chat(chatId));

public async Task SetTyping(int chatId, bool isTyping)
{
    await EnsureChatMembershipAsync(chatId);
    await Clients.OthersInGroup(ChatHubGroups.Chat(chatId))
        .SendAsync("TypingChanged", new { chatId, userId = GetCurrentUserId(),
isTyping });
}

public async Task MarkMessagesRead(int chatId, IReadOnlyCollection<int>
messageIds)
{
    await EnsureChatMembershipAsync(chatId);
    var ids = messageIds.Where(id => id > 0).Distinct().Take(100).ToArray();
    if (ids.Length == 0) return;

    var readerUserId = GetCurrentUserId();
    await messageRepository.MarkAsReadAsync(chatId, readerUserId, ids,
timeProvider.GetUtcNow().UtcDateTime, Context.ConnectionAborted);
    await realtimeNotifier.MessagesReadAsync(chatId, readerUserId, ids, await
messageRepository.GetMemberUserIdsAsync(chatId, Context.ConnectionAborted),
Context.ConnectionAborted);
}

private async Task EnsureChatMembershipAsync(int chatId)
{
    if (!await messageRepository.IsChatMemberAsync(chatId, GetCurrentUserId(),
Context.ConnectionAborted))
    {
        throw new HubException("Bu chatga kirish huquqi yo'q.");
    }
}

private int GetCurrentUserId() =>
    int.Parse(Context.User?.FindFirstValue(ClaimTypes.NameIdentifier)
        ?? throw new HubException("Foydalanuvchi aniqlanmadi.));

private int GetCurrentSessionId() =>
    int.Parse(Context.User?.FindFirstValue("sid"))

```

```

        ?? throw new HubException("Sessiya aniqlanmadi.");
    }

```

2-ilova. Guruh chatlari servisi (GroupService)

Fayl: NationalChat/src/Application/Features/Groups/Services/GroupService.cs

```

using Application.Features.Files;
using Application.Features.Files.DataTransferObjects.Requests;
using Application.Features.Groups.DataTransferObjects.Requests;
using Application.Features.Groups.DataTransferObjects.Responses;
using Application.Features.Groups.Factories;
using Application.Features.Groups.Mappers;
using Application.Features.Messages;
using Application.Features.Messages.Factories;
using Application.Features.Messages.Mappers;
using Application.Features.Presence;
using Domain.Entities;
using FluentValidation;

namespace Application.Features.Groups;

public sealed class GroupService(
    IGroupRepository repository,
    IFileService fileService,
    IChatRealtimeNotifier realtimeNotifier,
    IPresenceTracker presenceTracker,
    IValidator<CreateGroupRequest> createValidator,
    IValidator<UpdateGroupRequest> updateValidator,
    IValidator<AddGroupMembersRequest> addMembersValidator,
    IValidator<UpdateGroupMemberRoleRequest> updateRoleValidator,
    TimeProvider timeProvider) : IGroupService
{
    private const string NotFoundError = "Guruh topilmadi.";
    private const string ForbiddenError = "Bu amal uchun ruxsat yo'q.";

    public async Task<GroupDto?> GetAsync(int currentUserId, int chatId,
        CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        return group is null || FindMember(group, currentUserId) is null ? null :
        ToDto(group, currentUserId);
    }

    public async Task<GroupResult> CreateAsync(int currentUserId, CreateGroupRequest
        request, CancellationToken cancellationToken = default)
    {
        var validation = await createValidator.ValidateAsync(request,
            cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var memberIds = request.MemberIds.Where(id => id !=
            currentUserId).Distinct().ToArray();
        if (memberIds.Length == 0) return Fail("Kamida bitta a'zo tanlanishi kerak.");

        var users = await repository.FindUsersAsync([currentUserId, .. memberIds],
            cancellationToken);
        var creator = users.FirstOrDefault(user => user.Id == currentUserId);
        var members = users.Where(user => user.Id != currentUserId).ToArray();
        if (creator is null) return Fail("Foydalanuvchi aniqlanmadi.");
        if (members.Length != memberIds.Length) return Fail("Ba'zi foydalanuvchilar
            topilmadi.");
    }

```

```

        var now = timeProvider.GetUtcNow().UtcDateTime;
        var group = GroupFactory.Create(request.Title.Trim(),
NormalizeDescription(request.Description), creator, members, now);
        await repository.AddGroupAsync(group, cancellationTokens);
        await repository.SaveChangesAsync(cancellationTokens);

        await PublishServiceMessageAsync(group.Chat.Messages.Single(), group,
cancellationTokens);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdateAsync(int currentUserId, int chatId,
UpdateGroupRequest request, CancellationTokens cancellationTokens = default)
    {
        var validation = await updateValidator.ValidateAsync(request,
cancellationTokens);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationTokens);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var title = request.Title.Trim();
        var titleChanged = title != group.Title;
        group.Title = title;
        group.Description = NormalizeDescription(request.Description);
        var serviceMessage = titleChanged
            ? AddServiceMessage(group, actor.User, MessageServiceAction.TitleChanged,
title)
            : null;
        await repository.SaveChangesAsync(cancellationTokens);

        if (serviceMessage is not null) await
PublishServiceMessageAsync(serviceMessage, group, cancellationTokens);
        await PublishGroupUpdatedAsync(group, cancellationTokens);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdatePhotoAsync(int currentUserId, int chatId,
StoreImageRequest request, CancellationTokens cancellationTokens = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationTokens);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var storedResult = await fileService.StoreImageAsync(request,
cancellationTokens);
        if (storedResult.File is null) return Fail(storedResult.Error ?? "Rasm
yuklanmadi.");
        var stored = storedResult.File;

        var oldFile = group.Photo?.File;
        var file = new Domain.Entities.File { Name = stored.FileName, MimeType =
stored.MimeType, SizeBytes = stored.SizeBytes, StoragePath = stored.StoragePath };
        var photo = new Photo { File = file, Width = stored.Width, Height =
stored.Height };
        await repository.AddPhotoAsync(file, photo, cancellationTokens);
        group.Photo = photo;
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.PhotoChanged, null);
        await repository.SaveChangesAsync(cancellationTokens);

        if (oldFile is not null) await fileService.DeleteAsync(oldFile.StoragePath,
cancellationTokens);
    }

```

```

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> AddMembersAsync(int currentUserId, int chatId,
AddGroupMembersRequest request, CancellationToken cancellationToken = default)
    {
        var validation = await addMembersValidator.ValidateAsync(request,
cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (!CanManageGroup(actor)) return Fail(ForbiddenError);

        var existingIds = group.Chat.Members.Select(member =>
member.UserId).ToHashSet();
        var newIds = request.UserIds.Where(id =>
!existingIds.Contains(id)).Distinct().ToArray();
        if (newIds.Length == 0) return Fail("Tanlangan foydalanuvchilar allaqachon
guruhda.");
        var maxMembers = GroupLimits.MaxMembersFor(group.OrganizationId is not null);
        if (existingIds.Count + newIds.Length > maxMembers) return Fail($"Guruhda
{maxMembers} tadan ortiq a'zo bo'lishi mumkin emas.");

        var users = await repository.FindUsersAsync(newIds, cancellationToken);
        if (users.Count != newIds.Length) return Fail("Ba'zi foydalanuvchilar
topilmadi.");

        var now = timeProvider.GetUtcNow().UtcDateTime;
        foreach (var user in users)
        {
            group.Chat.Members.Add(GroupFactory.CreateMember(user,
ChatMemberRole.Member, now));
        }

        var names = string.Join(", ", users.Select(GroupMapper.DisplayName));
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.MembersAdded, names);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> RemoveMemberAsync(int currentUserId, int chatId,
int userId, CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);

        var target = FindMember(group, userId);
        if (target is null) return Fail("Foydalanuvchi guruh a'zosi emas.");
        if (target.UserId == currentUserId) return Fail("Guruhdan chiqish uchun
\"Guruhdan chiqish\" amalidan foydalaning.");
        if (!CanRemove(actor, target)) return Fail(ForbiddenError);

        RemoveMember(group, target);
        var serviceMessage = AddServiceMessage(group, actor.User,
MessageServiceAction.MemberRemoved, GroupMapper.DisplayName(target.User));

```

```

        await repository.SaveChangesAsync(cancellationToken);

        await realtimeNotifier.ChatDeletedAsync(chatId, [target.UserId],
cancellationToken);
        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupResult> UpdateMemberRoleAsync(int currentUserId, int
chatId, int userId, UpdateGroupMemberRoleRequest request, CancellationTok
cancellationToken = default)
    {
        var validation = await updateRoleValidator.ValidateAsync(request,
cancellationToken);
        if (!validation.IsValid) return Fail(validation.Errors[0].ErrorMessage);

        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var actor = group is null ? null : FindMember(group, currentUserId);
        if (group is null || actor is null) return Fail(NotFoundError);
        if (actor.Role != ChatMemberRole.Creator) return Fail("Faqat guruh egasi
adminlarni tayinlay oladi.");

        var target = FindMember(group, userId);
        if (target is null) return Fail("Foydalanuvchi guruh a'zosi emas.");
        if (target.Role == ChatMemberRole.Creator) return Fail("Guruh egasining rolini
o'zgartirib bo'lmaydi.");

        target.Role = request.Role;
        await repository.SaveChangesAsync(cancellationToken);

        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<GroupLeaveResult> LeaveAsync(int currentUserId, int chatId,
CancellationToken cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        var leaver = group is null ? null : FindMember(group, currentUserId);
        if (group is null || leaver is null) return new(false, NotFoundError);

        var now = timeProvider.GetUtcNow().UtcDateTime;
        RemoveMember(group, leaver);

        if (group.Chat.Members.Count == 0)
        {
            group.Chat.DeletedAt = now;
            await repository.SaveChangesAsync(cancellationToken);
            await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
cancellationToken);
            return new(true, null);
        }

        if (leaver.Role == ChatMemberRole.Creator)
        {
            // Ownership passes to the longest-serving admin, or to the longest-
serving member if there are no admins.
            var successor = group.Chat.Members
                .OrderByDescending(member => member.Role == ChatMemberRole.Admin)
                .ThenBy(member => member.JoinedAt)
                .ThenBy(member => member.Id)
                .First();
            successor.Role = ChatMemberRole.Creator;
            group.CreatorId = successor.UserId;
        }
    }

```

```

        group.Creator = successor.User;
    }

    var serviceMessage = AddServiceMessage(group, leaver.User,
MessageServiceAction.MemberLeft, GroupMapper.DisplayName(leaver.User));
    await repository.SaveChangesAsync(cancellationToken);

    await realtimeNotifier.ChatDeletedAsync(chatId, [currentUserId],
cancellationToken);
    await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
    await PublishGroupUpdatedAsync(group, cancellationToken);
    return new(true, null);
}

public async Task<GroupResult> CreateInviteLinkAsync(int currentUserId, int
chatId, CancellationToken cancellationToken = default)
{
    var group = await repository.GetGroupAsync(chatId, cancellationToken);
    var actor = group is null ? null : FindMember(group, currentUserId);
    if (group is null || actor is null) return Fail(NotFoundError);
    if (!CanManageGroup(actor)) return Fail(ForbiddenError);

    // A new token also revokes the previous link.
    group.InviteLink = GroupInviteTokens.Create();
    await repository.SaveChangesAsync(cancellationToken);
    return new(ToDto(group, currentUserId), null);
}

public async Task<GroupResult> RevokeInviteLinkAsync(int currentUserId, int
chatId, CancellationToken cancellationToken = default)
{
    var group = await repository.GetGroupAsync(chatId, cancellationToken);
    var actor = group is null ? null : FindMember(group, currentUserId);
    if (group is null || actor is null) return Fail(NotFoundError);
    if (!CanManageGroup(actor)) return Fail(ForbiddenError);

    group.InviteLink = null;
    await repository.SaveChangesAsync(cancellationToken);
    return new(ToDto(group, currentUserId), null);
}

public async Task<GroupInvitePreviewDto?> GetInvitePreviewAsync(int currentUserId,
string token, CancellationToken cancellationToken = default)
{
    if (!GroupInviteTokens.IsWellFormed(token)) return null;
    var group = await repository.GetGroupByInviteTokenAsync(token,
cancellationToken);
    return group is null ? null : GroupMapper.ToInvitePreview(group,
FindMember(group, currentUserId) is not null);
}

public async Task<GroupResult> JoinByInviteAsync(int currentUserId, string token,
CancellationToken cancellationToken = default)
{
    const string invalidLinkError = "Taklif havolasi yaroqsiz yoki bekor
qilingan.";
    if (!GroupInviteTokens.IsWellFormed(token)) return Fail(invalidLinkError);
    var group = await repository.GetGroupByInviteTokenAsync(token,
cancellationToken);
    if (group is null) return Fail(invalidLinkError);
    if (FindMember(group, currentUserId) is not null) return new(ToDto(group,
currentUserId), null);

    var maxMembers = GroupLimits.MaxMembersFor(group.OrganizationId is not null);
    if (group.Chat.Members.Count >= maxMembers) return Fail($"Guruhda {maxMembers}
tadan ortiq a'zo bo'lishi mumkin emas.");
}

```

```

        var user = (await repository.FindUsersAsync([currentUserId],
cancellationToken)).FirstOrDefault();
        if (user is null) return Fail("Foydalanuvchi aniqlanmadi.");

        group.Chat.Members.Add(GroupFactory.CreateMember(user, ChatMemberRole.Member,
timeProvider.GetUtcNow().UtcDateTime));
        var serviceMessage = AddServiceMessage(group, user,
MessageServiceAction.MemberJoinedViaInvite, GroupMapper.DisplayName(user));
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return new(ToDto(group, currentUserId), null);
    }

    public async Task<bool> CreateOrganizationGroupAsync(int userId, Cancellation token
cancellationToken = default)
    {
        var owner = (await repository.FindUsersAsync([userId],
cancellationToken)).FirstOrDefault();
        var organization = owner?.OrganizationMembership?.Organization;
        if (owner is null || organization is null) return false;

        var members = await repository.FindOrganizationUsersAsync(
            organization.Id, [userId], GroupLimits.MaxOrganizationMembers - 1,
cancellationToken);
        var group = GroupFactory.Create(
            organization.Domain,
            $"@{organization.Domain} pochta bilan kirganlar avtomatik qo'shiladi.
Boshqalar faqat admin qo'shganda yoki taklif havolasi orqali kiradi.",
            owner, members, timeProvider.GetUtcNow().UtcDateTime, organization);
        await repository.AddGroupAsync(group, cancellationToken);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(group.Chat.Messages.Single(), group,
cancellationToken);
        return true;
    }

    public async Task<bool> JoinViaOrganizationAsync(int chatId, int userId,
Cancellation token cancellationToken = default)
    {
        var group = await repository.GetGroupAsync(chatId, cancellationToken);
        if (group?.Organization is null || FindMember(group, userId) is not null)
return false;
        if (group.Chat.Members.Count >= GroupLimits.MaxOrganizationMembers) return
false;

        var user = (await repository.FindUsersAsync([userId],
cancellationToken)).FirstOrDefault();
        if (user is null || !IsOrganizationMember(user, group.Organization.Id)) return
false;

        group.Chat.Members.Add(GroupFactory.CreateMember(user, ChatMemberRole.Member,
timeProvider.GetUtcNow().UtcDateTime));
        var serviceMessage = AddServiceMessage(group, user,
MessageServiceAction.MemberJoinedViaOrganization, group.Organization.Domain);
        await repository.SaveChangesAsync(cancellationToken);

        await PublishServiceMessageAsync(serviceMessage, group, cancellationToken);
        await PublishGroupUpdatedAsync(group, cancellationToken);
        return true;
    }

```

```

private static bool IsOrganizationMember(User user, int organizationId) =>
    user.OrganizationMembership?.OrganizationId == organizationId;

private static bool CanManageGroup(ChatMember actor) =>
    actor.Role is ChatMemberRole.Admin or ChatMemberRole.Creator;

private static bool CanRemove(ChatMember actor, ChatMember target) => actor.Role
switch
{
    ChatMemberRole.Creator => true,
    ChatMemberRole.Admin => target.Role == ChatMemberRole.Member,
    _ => false
};

private static ChatMember? FindMember(Group group, int userId) =>
    group.Chat.Members.FirstOrDefault(member => member.UserId == userId);

private void RemoveMember(Group group, ChatMember member)
{
    group.Chat.Members.Remove(member);
    repository.RemoveMember(member);
}

private Message AddServiceMessage(Group group, User actor, MessageServiceAction
action, string? textContent)
{
    var message = MessageFactory.CreateService(group.ChatId, actor.Id, action,
textContent, timeProvider.GetUtcNow().UtcDateTime);
    message.Sender = actor;
    group.Chat.Messages.Add(message);
    return message;
}

private Task PublishServiceMessageAsync(Message message, Group group,
CancellationToken cancellationToken) =>
    realtimeNotifier.MessageCreatedAsync(MessageMapper.ToDto(message),
MemberIds(group), cancellationToken);

private Task PublishGroupUpdatedAsync(Group group, CancellationToken
cancellationToken) =>
    realtimeNotifier.GroupUpdatedAsync(group.ChatId, group.Title,
group.Description, group.PhotoId, group.Chat.Members.Count, MemberIds(group),
cancellationToken);

private static IReadOnlyCollection<int> MemberIds(Group group) =>
    group.Chat.Members.Select(member => member.UserId).ToArray();

private GroupDto ToDto(Group group, int currentUserId) =>
    GroupMapper.ToDto(group, currentUserId, presenceTracker.IsOnline);

private static string? NormalizeDescription(string? description) =>
    string.IsNullOrEmpty(description) ? null : description.Trim();

private static GroupResult Fail(string error) => new(null, error);
}

```

3-ilova. Bir martalik kodlarni xeshlash (PBKDF2)

Fayl:

NationalChat/src/Infrastructure/Security/Hashing/Pbkdf2OneTimeCodeHasher.cs

```
using System.Security.Cryptography;
using Application.Features.Authentication;

namespace Infrastructure.Security.Hashing;

public sealed class Pbkdf2OneTimeCodeHasher : IOneTimeCodeHasher
{
    private const int Iterations = 210_000;
    private const int SaltSize = 16;
    private const int HashSize = 32;

    public string Hash(string code)
    {
        var salt = RandomNumberGenerator.GetBytes(SaltSize);
        var hash = Rfc2898DeriveBytes.Pbkdf2(code, salt, Iterations,
        HashAlgorithmName.SHA256, HashSize);
        return
        $"{Iterations}.{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
    }

    public bool Verify(string code, string storedHash)
    {
        var parts = storedHash.Split('.', StringSplitOptions.RemoveEmptyEntries);
        if (parts.Length != 3 || !int.TryParse(parts[0], out var iterations))
        {
            return false;
        }

        try
        {
            var salt = Convert.FromBase64String(parts[1]);
            var expectedHash = Convert.FromBase64String(parts[2]);
            var actualHash = Rfc2898DeriveBytes.Pbkdf2(code, salt, iterations,
            HashAlgorithmName.SHA256, expectedHash.Length);
            return CryptographicOperations.FixedTimeEquals(actualHash, expectedHash);
        }
        catch (FormatException)
        {
            return false;
        }
    }
}
```

4-ilova. Klientdagi real vaqt ulanishi (ChatRealtimeService)

Fayl: NationalChatClient/src/app/features/chat/data-access/chat-realtime.service.ts

```
async connect(): Promise<void> {
    if (!isPlatformBrowser(this.platformId) || !this.auth.accessToken()) return;
    if (!this.connection) this.createConnection();
    if (this.connection!.state === HubConnectionState.Connected) return;
    if (this.connectPromise) return this.connectPromise;

    this.connectionState.set('connecting');
    this.connectPromise = this.connection!.start()
        .then(() => this.connectionState.set('connected'))
}
```

```

        .catch((error: unknown) => {
            this.connectionState.set('disconnected');
            throw error;
        })
        .finally(() => (this.connectPromise = undefined));
    return this.connectPromise;
}

async joinChat(chatId: number): Promise<void> {
    const previousChatId = this.joinedChatId;
    this.joinedChatId = chatId;
    await this.connect();
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
return;
    if (previousChatId && previousChatId !== chatId)
        await this.connection.invoke('LeaveChat', previousChatId);
    await this.connection.invoke('JoinChat', chatId);
}

async leaveChat(chatId: number): Promise<void> {
    if (this.joinedChatId === chatId) this.joinedChatId = null;
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
return;
    await this.connection.invoke('LeaveChat', chatId);
}

async setTyping(chatId: number, isTyping: boolean): Promise<void> {
    if (!this.connection || this.connection.state !== HubConnectionState.Connected)
return;
    await this.connection.send('SetTyping', chatId, isTyping);
}

async markMessagesRead(chatId: number, messageIds: number[]): Promise<void> {
    if (
        !messageIds.length ||
        !this.connection ||
        this.connection.state !== HubConnectionState.Connected
    )
        return;
    await this.connection.send('MarkMessagesRead', chatId, messageIds);
}

async disconnect(): Promise<void> {
    this.joinedChatId = null;
    if (this.connection && this.connection.state !== HubConnectionState.Disconnected)
        await this.connection.stop();
    this.connectionState.set('disconnected');
}

private createConnection(): void {
    this.connection = new HubConnectionBuilder()
        .withUrl(apiUrl('/hubs/chat'), { accessTokenFactory: () =>
this.auth.accessToken() ?? '' })
        .withAutomaticReconnect([0, 2_000, 5_000, 10_000])
        .configureLogging(LogLevel.Warning)
        .build();

    this.connection.on('MessageReceived', (message: Message) =>
        this.messageReceivedSubject.next(message),
    );
    this.connection.on('TypingChanged', (event: TypingChangedEvent) =>
        this.typingChangedSubject.next(event),
    );
    this.connection.on('MessagesRead', (event: MessagesReadEvent) =>
        this.messagesReadSubject.next(event),
    );
}

```

```

        this.connection.on('MessageUpdated', (message: Message) =>
this.messageUpdatedSubject.next(message));
        this.connection.on('MessageDeleted', (event: { chatId: number; messageId: number
}) => this.messageDeletedSubject.next(event));
        this.connection.on('ChatCleared', (event: { chatId: number }) =>
this.chatClearedSubject.next(event));
        this.connection.on('ChatDeleted', (event: { chatId: number }) =>
this.chatDeletedSubject.next(event));
        this.connection.on('ProfilePhotoUpdated', (event: ProfilePhotoUpdatedEvent) =>
this.profilePhotoUpdatedSubject.next(event));
        this.connection.on('UserPresenceChanged', (event: UserPresenceChangedEvent) =>
this.userPresenceChangedSubject.next(event));
        this.connection.on('GroupUpdated', (event: GroupUpdatedEvent) =>
this.groupUpdatedSubject.next(event));
        this.connection.on('SecretChatRequested', (chat: SecretChatDto) =>
this.secretChatRequestedSubject.next(chat));
        this.connection.on('SecretChatAccepted', (chat: SecretChatDto) =>
this.secretChatAcceptedSubject.next(chat));
        this.connection.on('SecretMessageReceived', (message: SecretMessageDto) =>
this.secretMessageReceivedSubject.next(message));
        this.connection.on('SecretChatClosed', (event: { secretChatId: number }) =>
this.secretChatClosedSubject.next(event));
        this.connection.onreconnecting(() => this.connectionState.set('reconnecting'));
        this.connection.onreconnected(async () => {
            this.connectionState.set('connected');
            if (this.joinedChatId) await this.connection?.invoke('JoinChat',
this.joinedChatId);
            this.reconnectedSubject.next();
        });
        this.connection.onclose(() => this.connectionState.set('disconnected'));

```

5-ilova. Guruh chatlari uchun unit testlar (parcha)

Fayl: NationalChat/tests/NationalChat.Tests/Groups/GroupServiceTests.cs

```

[Fact]
public async Task
Create_ValidRequest_CreatesGroupWithCreatorRoleAndServiceMessage()
{
    var result = await _service.CreateAsync(_sardor.Id, new
CreateGroupRequest("315-21 guruh", "TATU", [_jasur.Id, _dilnoza.Id]));

    Assert.Null(result.Error);
    Assert.NotNull(result.Group);
    Assert.Equal(ChatMemberRole.Creator, result.Group!.MyRole);
    Assert.Equal(3, result.Group.Members.Count);

    var chat = Assert.Single(_repository.Groups).Chat;
    Assert.Equal(ChatType.Group, chat.Type);
    var message = Assert.Single(chat.Messages);
    Assert.Equal(MessageServiceAction.GroupCreated, message.ServiceAction);

    await _notifier.Received(1).MessageCreatedAsync(
        Arg.Is<MessageDto>(m => m.ServiceAction ==
MessageServiceAction.GroupCreated),
        Arg.Is<IReadOnlyCollection<int>>(ids => ids.Count == 3),
        Arg.Any<CancellationToken>());
}

[Fact]
public async Task Create_UnknownMember_IsRejected()
{
    var result = await _service.CreateAsync(_sardor.Id, new
CreateGroupRequest("Guruh", null, [_jasur.Id, 999]));

```

```

        Assert.Null(result.Group);
        Assert.NotNull(result.Error);
        Assert.Empty(_repository.Groups);
    }

    [Fact]
    public async Task Create_EmptyTitle_FailsValidation()
    {
        var result = await _service.CreateAsync(_sardor.Id, new CreateGroupRequest("
", null, [_jasur.Id]));

        Assert.Null(result.Group);
        Assert.Empty(_repository.Groups);
    }

```

6-ilova. O‘zbek yozuvlari o‘rtasida transliteratsiya (UzbekTransliterator)

Fayl: NationalChat/src/Domain/Text/UzbekTransliterator.cs

```

using System.Text;
using System.Text.RegularExpressions;

namespace Domain.Text;

/// <summary>
/// Converts Uzbek text between the Cyrillic and the 1995 Latin alphabet.
/// The TypeScript client keeps an identical copy (uzbek-transliterator.ts); both are
tested
/// against the same transliteration-cases.json, so any rule change must be made in
both places.
/// URLs, e-mail addresses, @usernames, #tags and `code` spans are never converted.
/// </summary>
public static partial class UzbekTransliterator
{
    /// <summary>Letter part of o' and g' in the output of <see
    cref="ToLatin"/>.</summary>
    public const char OkinaMark = 'ʻ';

    /// <summary>Tutuq belgisi (ʼ) in the output of <see cref="ToLatin"/>.</summary>
    public const char TutuqMark = 'ʼ';

    private const string Apostrophes = "ʻʼ`'";
    private const string CyrillicVowels = "aeёиоуўёяы";
    private const string LatinVowels = "aeiou";

    // Loanwords where Latin "ts" stands for Cyrillic "ц"; everywhere else "ts" is
    "тс".
    private static readonly string[] TsExceptionStems =
    [
        "tsiy", "tsion", "litsey", "patsient", "ofitsiant", "ofitser", "spetsial",
        "sotsial", "abzats", "kvarts"
    ];

    private static readonly Dictionary<char, string> CyrillicToLatinMap = new()
    {
        ['a'] = "a", ['б'] = "b", ['в'] = "v", ['г'] = "g", ['ё'] = "g'", ['д'] = "d",
        ['ё'] = "yo",
        ['ж'] = "j", ['з'] = "z", ['и'] = "i", ['й'] = "y", ['к'] = "k", ['қ'] = "q",
        ['л'] = "l",
        ['м'] = "m", ['н'] = "n", ['о'] = "o", ['п'] = "p", ['р'] = "r", ['с'] = "s",
        ['т'] = "t",

```

```

        ['y'] = "u", ['ŷ'] = "o'", ['ф'] = "f", ['x'] = "x", ['x'] = "h", ['ч'] =
"ch", ['ш'] = "sh",
        ['ш'] = "sh", ['ъ'] = "'", ['ы'] = "i", ['ь'] = "", ['э'] = "e", ['ю'] = "yu",
['я'] = "ya"
    };

    private static readonly Dictionary<char, char> LatinToCyrillicMap = new()
    {
        ['a'] = 'a', ['b'] = 'б', ['c'] = 'c', ['d'] = 'д', ['f'] = 'ф', ['g'] = 'г',
['h'] = 'х',
        ['i'] = 'и', ['j'] = 'ж', ['k'] = 'к', ['l'] = 'л', ['m'] = 'м', ['n'] = 'н',
['o'] = 'о',
        ['p'] = 'п', ['q'] = 'к', ['r'] = 'р', ['s'] = 'с', ['t'] = 'т', ['u'] = 'у',
['v'] = 'в',
        ['w'] = 'в', ['x'] = 'x', ['y'] = 'й', ['z'] = 'з'
    };

    // Order matters: e-mail must win over @username, which would otherwise match its
    domain part.
    [GeneratedRegex(
        @"`[^`]*`|(?:(https?://|www\.)\S+|[\p{L}\p{N}._%+-]+@[ \p{L}\p{N}-
]+(?:\.[ \p{L}\p{N}-]+)+|[@#][ \p{L}\p{N}_]+",
        RegexOptions.IgnoreCase)]
    private static partial Regex ProtectedSpanRegex();

    [GeneratedRegex(@"\s+")]
    private static partial Regex WhitespaceRegex();

    public static string ToLatin(string? text) => ConvertOutsideProtectedSpans(text,
    ToLatinSegment);

    public static string ToCyrillic(string? text) =>
    ConvertOutsideProtectedSpans(text, ToCyrillicSegment);

    /// <summary>
    /// Script-independent search key: Latin, lower case, one apostrophe form, single
    spaces.
    /// "Ўзбек", "o'zbek", "o'zbek" and "O'zbek" all become "o'zbek".
    /// </summary>
    public static string NormalizeForSearch(string? text)
    {
        if (string.IsNullOrEmpty(text)) return string.Empty;

        var latin = ToLatin(text).ToLowerInvariant();
        var builder = new StringBuilder(latin.Length);
        foreach (var ch in latin) builder.Append(IsApostrophe(ch) ? '\'' : ch);
        return WhitespaceRegex().Replace(builder.ToString(), " ").Trim();
    }

    public static bool IsApostrophe(char ch) => Apostrophes.Contains(ch);

    private static string ConvertOutsideProtectedSpans(string? text, Func<string,
    string> convert)
    {
        if (string.IsNullOrEmpty(text)) return text ?? string.Empty;

        var builder = new StringBuilder(text.Length + 8);
        var position = 0;
        foreach (Match match in ProtectedSpanRegex().Matches(text))
        {
            builder.Append(convert(text[position..match.Index]));
            builder.Append(match.Value);
            position = match.Index + match.Length;
        }
    }

```

```

        builder.Append(convert(text[position..]));
        return builder.ToString();
    }

    private static string ToLatinSegment(string text)
    {
        var builder = new StringBuilder(text.Length + 8);
        for (var i = 0; i < text.Length; i++)
        {
            var ch = text[i];
            var lower = char.ToLowerInvariant(ch);
            var previous = i > 0 ? char.ToLowerInvariant(text[i - 1]) : '\0';

            string latin;
            if (lower == 'e')
            {
                var soundsYe = !IsWordChar(text, i - 1) ||
CyrillicVowels.Contains(previous) || previous is 'ѣ' or 'ѥ';
                latin = soundsYe ? "ye" : "e";
            }
            else if (lower == 'ц')
            {
                latin = CyrillicVowels.Contains(previous) ? "ts" : "s";
            }
            else if (lower == 'х' && previous == 'с')
            {
                // "cx" (Исхок) is written with a tutuq in Latin so that it is not
read as "ш".
                latin = TutuqMark + "h";
            }
            else if (!CyrillicToLatinMap.TryGetValue(lower, out latin!))
            {
                builder.Append(ch);
                continue;
            }

            builder.Append(ch == lower ? latin : ApplyUpperCase(latin,
IsUpperCaseWord(text, i)));
        }

        return builder.ToString();
    }

    private static string ToCyrillicSegment(string text)
    {
        var lower = text.ToLowerInvariant();
        var tsPositions = FindExceptionTsPositions(lower);
        var builder = new StringBuilder(text.Length);
        var i = 0;
        while (i < text.Length)
        {
            var c = lower[i];
            var next = i + 1 < text.Length ? lower[i + 1] : '\0';
            var afterNext = i + 2 < text.Length ? lower[i + 2] : '\0';
            var isUpper = text[i] != c;

            string? cyrillic = null;
            var length = 1;
            if (c == 's' && next == 'h') (cyrillic, length) = ("ш", 2);
            else if (c == 's' && IsApostrophe(next) && afterNext == 'h') (cyrillic,
length) = ("ш", 2);
            else if (c == 'c' && next == 'h') (cyrillic, length) = ("ц", 2);
            else if (c == 'o' && IsApostrophe(next)) (cyrillic, length) = ("ѣ", 2);
            else if (c == 'g' && IsApostrophe(next)) (cyrillic, length) = ("ѥ", 2);
            else if (c == 'y' && next == 'o' && !IsApostrophe(afterNext)) (cyrillic,
length) = ("ѧ", 2);
            else if (c == 'y' && next == 'u') (cyrillic, length) = ("Ѩ", 2);
        }
    }

```

```

        else if (c == 'y' && next == 'a') (cyrillic, length) = ("я", 2);
        else if (c == 'y' && next == 'e') (cyrillic, length) = ("е", 2);
        else if (c == 't' && next == 's' && tsPositions.Contains(i)) (cyrillic,
length) = ("ц", 2);
        else if (c == 'e')
        {
            // Cyrillic "e" after a vowel or at a word start is written "ye", so a
bare "e" there was "э".
            var previous = i > 0 ? lower[i - 1] : '\0';
            cyrillic = !IsWordChar(text, i - 1) || LatinVowels.Contains(previous)
? "э" : "е";
        }
        else if (IsApostrophe(c) && IsWordChar(text, i - 1) && IsLetter(text, i +
1))
        {
            cyrillic = IsUpperCaseWord(text, i) ? "Ъ" : "ь";
        }
        else if (LatinToCyrillicMap.TryGetValue(c, out var single))
        {
            cyrillic = single.ToString();
        }

        if (cyrillic is null)
        {
            builder.Append(text[i]);
        }
        else
        {
            builder.Append(isUpper ? cyrillic.ToUpperInvariant() : cyrillic);
        }

        i += length;
    }

    return builder.ToString();
}

private static HashSet<int> FindExceptionTsPositions(string lowerText)
{
    var positions = new HashSet<int>();
    foreach (var stem in TsExceptionStems)
    {
        for (var start = lowerText.IndexOf(stem, StringComparison.Ordinal); start
>= 0; start = lowerText.IndexOf(stem, start + 1, StringComparison.Ordinal))
        {
            for (var ts = stem.IndexOf("ts", StringComparison.Ordinal); ts >= 0;
ts = stem.IndexOf("ts", ts + 2, StringComparison.Ordinal))
            {
                positions.Add(start + ts);
            }
        }
    }

    return positions;
}

private static string ApplyUpperCase(string latin, bool wholeWordUpper)
{
    if (latin.Length == 0) return latin;
    return wholeWordUpper ? latin.ToUpperInvariant() :
char.ToUpperInvariant(latin[0]) + latin[1..];
}

private static bool IsLetter(string text, int index) =>
    index >= 0 && index < text.Length && char.IsLetter(text[index]) &&
!IsApostrophe(text[index]);

```

```

    private static bool IsWordChar(string text, int index) =>
        index >= 0 && index < text.Length && (char.IsLetter(text[index]) ||
        IsApostrophe(text[index]));

    /// <summary>True when the word around <paramref name="index"/> has at least two
    cased letters, all upper case.</summary>
    private static bool IsUpperCaseWord(string text, int index)
    {
        var start = index;
        while (IsWordChar(text, start - 1)) start--;
        var end = index;
        while (IsWordChar(text, end + 1)) end++;

        var casedLetters = 0;
        for (var i = start; i <= end; i++)
        {
            var ch = text[i];
            if (char.ToUpperInvariant(ch) == char.ToLowerInvariant(ch)) continue;
            if (!char.IsUpper(ch)) return false;
            casedLetters++;
        }

        return casedLetters >= 2;
    }
}

```

7-ilova. Maxfiy chatlar kriptografiyasi: kalitlarni chiqarish, shifrlash va ratchet (secret-crypto.ts, parcha)

Fayl: NationalChatClient/src/app/features/chat/secret/secret-crypto.ts

```

export async function deriveChains(
    role: SecretRole,
    secretChatId: number,
    myKeyPair: CryptoKeyPair,
    initiatorPublicKey: string,
    participantPublicKey: string,
): Promise<{ send: ChainState; receive: ReceiveState }> {
    const peerPublicKey = role === 'initiator' ? participantPublicKey :
    initiatorPublicKey;
    const peerKey = await subtle().importKey('raw', publicKeyBytes(peerPublicKey), {
    name: 'X25519' }, false, []);
    const shared = new Uint8Array(
        await subtle().deriveBits({ name: 'X25519', public: peerKey },
    myKeyPair.privateKey, KEY_BYTES * 8),
    );
    try {
        if (shared.every((byte) => byte === 0)) throw new SecretCryptoError('Ochiq kalit
    yaroqsiz.');
```

```

        const hkdfKey = await subtle().importKey('raw', shared, 'HKDF', false,
    ['deriveBits']);
        const salt = new Uint8Array(await subtle().digest('SHA-256',
    encoder.encode(HKDF_SALT_LABEL)));
        const info = encoder.encode(`nc-secret-
    v${PROTOCOL_VERSION}|${secretChatId}|${initiatorPublicKey}|${participantPublicKey}`);
        const okm = new Uint8Array(await subtle().deriveBits({ name: 'HKDF', hash: 'SHA-
    256', salt, info }, hkdfKey, KEY_BYTES * 2 * 8));
        try {
            const initiatorChain = await importChainKey(okm.subarray(0, KEY_BYTES));
            const participantChain = await importChainKey(okm.subarray(KEY_BYTES));
            const [send, receive] = role === 'initiator'
                ? [initiatorChain, participantChain]

```

```

        : [participantChain, initiatorChain];
        return { send: { chainKey: send, seq: 0 }, receive: { chainKey: receive, seq: 0,
skipped: new Map() } };
    } finally {
        okm.fill(0);
    }
    } finally {
        shared.fill(0);
    }
}

/**
 * Encrypts the next message. Returns the new chain state, which the caller must
persist before sending
 * so that a key is never used twice.
 */
export async function encryptMessage(
    chain: ChainState,
    secretChatId: number,
    direction: SecretDirection,
    payload: SecretPayload,
): Promise<{ chain: ChainState; seq: number; ciphertext: string }> {
    if (!isValidPayload(payload)) {
        throw new SecretCryptoError(
            payload.kind === 'text' ? `Xabar 1 dan ${MAX_TEXT_LENGTH} belgigacha bo'lishi
kerak.` : 'Xabar formati noto'g'ri.',
        );
    }

    const seq = chain.seq + 1;
    const { messageKey, nextChainKey } = await step(chain.chainKey);
    const iv = globalThis.crypto.getRandomValues(new Uint8Array(IV_BYTES));
    const plaintext = encoder.encode(JSON.stringify({ v: PROTOCOL_VERSION, ...payload
}));
    const sealed = new Uint8Array(
        await subtle().encrypt({ name: 'AES-GCM', iv, additionalData:
associatedData(secretChatId, direction, seq) }, messageKey, plaintext),
    );

    const envelope = new Uint8Array(1 + IV_BYTES + sealed.length);
    envelope[0] = PROTOCOL_VERSION;
    envelope.set(iv, 1);
    envelope.set(sealed, 1 + IV_BYTES);
    return { chain: { chainKey: nextChainKey, seq }, seq, ciphertext: toBase64(envelope)
};
}

/**
 * Decrypts a message from the peer. The returned state is only valid if decryption
succeeded; on any
 * failure this throws and the caller keeps its old state, so a forged message cannot
move the chain.
 */
export async function decryptMessage(
    state: ReceiveState,
    secretChatId: number,
    direction: SecretDirection,
    seq: number,
    ciphertext: string,
): Promise<{ state: ReceiveState; payload: SecretPayload }> {
    if (!Number.isSafeInteger(seq) || seq < 1) throw new SecretCryptoError('Xabar tartib
raqami noto'g'ri.');
```

```

    const envelope = fromBase64(ciphertext);
    if (envelope.length < 1 + IV_BYTES + 16 || envelope[0] !== PROTOCOL_VERSION) {
        throw new SecretCryptoError('Xabar formati noto'g'ri.');
```

```

const skipped = new Map(state.skipped);
let chainKey = state.chainKey;
let chainSeq = state.seq;
let messageKey: CryptoKey;

if (seq <= chainSeq) {
  const key = skipped.get(seq);
  if (!key) throw new SecretCryptoError('Bu xabar allaqachon o'qilgan yoki juda eski.');
```

eski.');

```

  skipped.delete(seq);
  messageKey = key;
} else {
  if (seq - chainSeq > MAX_SKIP) throw new SecretCryptoError('Juda ko'p xabar o'tkazib yuborilgan.');
```

o'tkazib yuborilgan.');

```

  while (chainSeq < seq - 1) {
    const next = await step(chainKey);
    chainSeq++;
    skipped.set(chainSeq, next.messageKey);
    chainKey = next.nextChainKey;
  }
  const next = await step(chainKey);
  messageKey = next.messageKey;
  chainKey = next.nextChainKey;
  chainSeq = seq;
  // Oldest skipped keys go first once there are too many.
  while (skipped.size > MAX_SKIP) skipped.delete(skipped.keys().next().value!);
}

let plaintext: ArrayBuffer;
try {
  plaintext = await subtle().decrypt(
    { name: 'AES-GCM', iv: envelope.subarray(1, 1 + IV_BYTES), additionalData:
associatedData(secretChatId, direction, seq) },
    messageKey,
    envelope.subarray(1 + IV_BYTES),
  );
} catch {
  throw new SecretCryptoError('Xabarni ochib bo'lmadi: u buzilgan yoki soxta.');
```

throw new SecretCryptoError('Xabarni ochib bo'lmadi: u buzilgan yoki soxta.');

```

}

return { state: { chainKey, seq: chainSeq, skipped }, payload:
parsePayload(plaintext) };
}

export interface SecretFingerprint {
  /** Eight emoji; both devices must see the same row. */
  emoji: string[];
  /** Six groups of five digits, for reading out loud. */
  digits: string[];
}

/**
 * Key fingerprint shown on both devices. If a man in the middle swapped the public
keys, the two devices
 * would see different values.
 */
export async function computeFingerprint(initiatorPublicKey: string,
participantPublicKey: string): Promise<SecretFingerprint> {
  const initiator = publicKeyBytes(initiatorPublicKey);
  const participant = publicKeyBytes(participantPublicKey);
  const label = encoder.encode('nc-secret-fingerprint-v1');
  const input = new Uint8Array(label.length + initiator.length + participant.length);
  input.set(label, 0);
  input.set(initiator, label.length);
  input.set(participant, label.length + initiator.length);
  const hash = new Uint8Array(await subtle().digest('SHA-256', input));
}

```

```

const emoji: string[] = [];
let bits = 0;
let bitCount = 0;
for (let index = 0; emoji.length < 8; index++) {
  bits = (bits << 8) | hash[index];
  bitCount += 8;
  while (bitCount >= 6 && emoji.length < 8) {
    bitCount -= 6;
    emoji.push(FINGERPRINT_EMOJI[(bits >> bitCount) & 0x3f]);
  }
}

const digits: string[] = [];
for (let group = 0; group < 6; group++) {
  let value = 0;
  for (let offset = 0; offset < 5; offset++) value = value * 256 + hash[group * 5 + offset];
  digits.push(String(value % 100_000).padStart(5, '0'));
}
return { emoji, digits };
}

/** Decodes and checks a 32-byte X25519 public key in standard Base64. */
export function publicKeyBytes(value: string): Uint8Array<ArrayBuffer> {
  const bytes = fromBase64(value);
  if (bytes.length !== KEY_BYTES) throw new SecretCryptoError('Ochiq kalit noto`g`ri.');
  return bytes;
}

async function step(chainKey: CryptoKey): Promise<{ messageKey: CryptoKey; nextChainKey: CryptoKey }> {
  const messageBytes = new Uint8Array(await subtle().sign('HMAC', chainKey, MESSAGE_KEY_INPUT));
  const chainBytes = new Uint8Array(await subtle().sign('HMAC', chainKey, CHAIN_KEY_INPUT));
  try {
    const messageKey = await subtle().importKey('raw', messageBytes, { name: 'AES-GCM' }, false, ['encrypt', 'decrypt']);
    return { messageKey, nextChainKey: await importChainKey(chainBytes) };
  } finally {
    messageBytes.fill(0);
    chainBytes.fill(0);
  }
}

function importChainKey(bytes: Uint8Array<ArrayBuffer>): Promise<CryptoKey> {
  return subtle().importKey('raw', bytes, { name: 'HMAC', hash: 'SHA-256' }, false, ['sign']);
}

function associatedData(secretChatId: number, direction: SecretDirection, seq: number): Uint8Array<ArrayBuffer> {

```